
Image and Video Compression Lab Course

Lab Course Notes

Summer Term 2022



**FRIEDRICH-ALEXANDER
UNIVERSITÄT
ERLANGEN-NÜRNBERG**
TECHNISCHE FAKULTÄT

Preface

Nowadays, efficient video coding techniques play a decisive role in the rapid spread of diverse applications in multimedia communications (e.g. high resolution content, 360° content, screen content).

In this lab course, you will get to know the components of a video coder in detail. You will implement them in software and ultimately build a complete video coder. In the end, you will evaluate its quality. This is a challenging task which requires thorough preparation and concentrated work from the very beginning, as the results of each lab experiment will be further processed in the subsequent lab experiments.

The basic hybrid video coding structure, which will be implemented throughout this lab course, has not changed through the years and is still present in current video codecs. Indeed, most components have been improved over the last video codec generations. However, we focus on the basic concepts and ideas of the different coding elements, since they are first still valid, and second, the complexity and therewith the workload would exceed the lab course. Nevertheless, references to current coding techniques or adaptations that are used in state-of-the-art video codecs are provided at the end of some chapters.

We have decided on using MATLAB as a programming language for this lab course, since it provides its user with a multitude of elementary functions for processing image and video signals. Many students might already know about MATLAB from other lectures and lab courses. Elementary use of MATLAB will be practiced during the first lab experiment.

All lab experiments are designed in a way such that, in addition to the mandatory part, suggestions for possible extensions and improvements are provided. Depending on time and interest, individual groups may implement these suggestions – or parts thereof – during the lab experiments. In doing so, your video coder might achieve better results in the subjective quality test which will be conducted at the end of the semester.

This lab course supplements other lectures on this subject at the Chair of Multimedia Communications and Signal Processing (Lehrstuhl für Multimediakommunikation und Signalverarbeitung; LMS). To successfully carry out this lab course, you should have attended the lecture *Image and Video Compression* (IVC). You can also attend both the lecture and this lab course in the same semester. Since the lab course and the IVC lecture cannot be fully synchronized, it might be helpful to take a look at the lecture notes ahead of the lecture schedule. This is considered part of the preparation for each single lab

experiment and also taken into account in these lab course notes by providing exhaustive explanations and examples in each chapter.

The current lab course description covers seven lab experiments, each of which comprises one or more weekly sessions:

Lab Exp.	Name	Sessions
1	Introduction to MATLAB	1
2	Entropy Coding	2
3	Quantization	1
4	Motion Estimation and Compensation	1
5	Transform Coding and Intra-Coding Chain	2
6	Video Codec (+ Extensions)	2 (+1)
7	Subjective Video Test + Evaluation	2

The lab experiments have been designed by members of the LMS. With great dedication, they have not only conceived the lab experiments and drafted their descriptions, but also written many accompanying programs and functions for the execution of the lab experiments. For this, we thank Marcus Barkowsky, Jens Bialkowski, Ulrich Fecker, Markus Friebe, Jens Garbas, Achim Kuntz, Katrin Meisinger, Sascha Spors, Stefan Wehr, and Andy Regensky. The translation to English has been provided by Andrea Eichenseer. Various revisions and the adaption to the current structure have been performed by Muzammil Rasool, Andreas Weinlich, and Kristian Fischer.

The $\text{\LaTeX}$ layout is taken from the “Software for Mathematics” lab course script and has been made available to us by Robert Fischer.

We wish you a good time with building your first video codec!

Erlangen, March 2022

André Kaup

Christian Herglotz

Kristian Fischer

Contents

Preface	1
Introduction	7
Introduction to Video Coding	7
Outline of the Lab Experiments	10
Structure of the Experiment Descriptions	12
1 Introduction to MATLAB	13
1.1 What is MATLAB?	13
1.2 How do I learn MATLAB?	14
1.3 How do I work with MATLAB?	14
1.4 How do I get help on MATLAB?	16
1.5 Data Types in MATLAB	16
1.6 Images and Matrices	17
1.6.1 Displaying, Reading and Saving Images	18
1.6.2 Creating Matrices	20
1.6.3 Parts of Matrices	20
1.6.4 Luminance of Images	21
1.7 Scripts and Functions	22
1.8 Program Flow and Debugging	23
1.9 Plotting Curves	24
1.10 Block Processing	25
1.11 Image Sequences	26
1.12 Cell Arrays	26
1.13 Maintaining the Workspace	28
1.14 Summary	29

2	Entropy Coding	31
2.1	Basics on Information Theory	32
2.1.1	Symbol Probabilities	32
2.1.2	Information Content and Entropy	33
2.1.3	Decision Content	33
2.1.4	Average Code Word Length and Redundancy	34
2.2	Huffman Coding	35
2.2.1	Part One: Huffman Table	39
2.2.2	Part Two: Huffman Encoder and Decoder	46
2.3	Summary	49
2.4	Literature Recommendation	49
3	Quantization	53
3.1	Rate-Distortion Curves	54
3.2	Quantization	56
3.2.1	Uniform Quantization	56
3.2.2	Uniform Quantization Using Quantization Matrices	59
3.3	Optional Part	62
3.3.1	Additions to Section 3.1: Distortion Measures	62
3.3.2	Additions to Section 3.2: Quantization	63
3.4	Summary	64
3.5	Literature Recommendation	64
4	Motion Estimation and Compensation	65
4.1	The Principle of Motion Compensated Prediction	66
4.2	Block-based motion estimation	67
4.3	Motion Compensation	69

4.4	Optional Part	77
4.4.1	Sub-Optimal Motion Estimation	77
4.4.2	Sub-Pixel Accurate Motion Estimation	81
4.4.3	Macroblock-Based Intra/Inter Decision	83
4.4.4	Variable Block Sizes	84
4.5	Summary	86
5	Transform Coding and Intra Coding Chain	87
5.1	Discrete Cosine Transform (DCT) and Inverse Discrete Cosine Transform (IDCT)	87
5.1.1	One-Dimensional DCT/IDCT	88
5.1.2	Two-Dimensional DCT/IDCT	88
5.1.3	Block-Based Two-Dimensional DCT	89
5.2	Zigzag Scan	94
5.2.1	Basics	94
5.3	Run-Level Coding	99
5.3.1	Basics	99
5.4	Intra Coding Chain	104
5.4.1	Basics	104
5.4.2	Encoder	104
5.4.3	Decoder	105
5.5	Optional Part	107
5.5.1	Differential Coding of the DC Coefficient	107
5.6	Summary	108
5.7	Literature Recommendation	109

6	Video Codec	111
6.1	Layout of a Video Codec	111
6.1.1	General Layout	112
6.1.2	Simplifications in this Lab Course	114
6.2	Development of the Basic Video Codec	114
6.2.1	Coding and Decoding the I-Frame	114
6.2.2	Coding and Decoding P-Frames	117
6.2.3	Simulations with the Basic Codec	120
6.2.4	Submitting the Optimized Codec	121
6.3	Own Extensions	122
6.3.1	Modifications for Motion Estimation	124
6.3.2	De-Blocking Filter Within the Coding Loop	125
6.3.3	Extension for Coding Color Data	126
6.3.4	Prediction of the DC Coefficients	127
6.3.5	Dead-Zone Quantizer	127
6.4	Summary	128
6.5	Literature Recommendations	128
7	Subjective Video Test	129
7.1	Viewing Conditions	129
7.2	Subjective Video Test	130
7.3	Evaluation of a Subjective Video Test	130
7.4	Presentation and Discussion of Results	133
7.5	Literature Recommendation	134

Introduction

This lab course aims to provide a comprehensive introduction to video coding. For this purpose, different parts of a video coder will be implemented in MATLAB. This introductory chapter serves as an overview of the whole project, beginning with the presentation of a video coder. The role of each functional block will be briefly outlined. Afterward, the individual lab experiments will be described.

Introduction to Video Coding

To begin with, still image coding is considered. For this, only the information obtained from a single image (in this context also called a *frame*) is used; the preceding and subsequent frames do not have any effect. This is also referred to as *intra-frame coding*. The block diagram of a simple intra-frame coder is depicted in Figure 1.

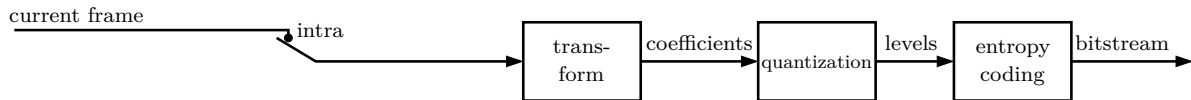


Figure 1: Block diagram of an intra-frame coder.

Most coding steps are performed in a block-based manner. This offers two significant advantages: First, image data is – in general – only stationary within a restricted local region. Consider the image of a human head, for example: The skin of the face has different statistics (low-frequency) than the surrounding hair (high-frequency). Stationarity can hence only be exploited in small image blocks as opposed to the whole image. The second advantage lies in the applied algorithms, whose processing times often increase squaredly or cubically with the data amount to be processed. Obviously, processing one block at a time is preferable to processing the whole image in one go.

From the block diagram, it is apparent that the image is first transformed into the frequency domain. This transformation step performs both a decorrelation and an energy compaction. To that end, the image is split into small blocks, generally of size 8×8 or 16×16 pixels. Each block is then transformed via the Discrete Cosine Transform (DCT). For natural images, it is known from experience that the low-frequency bands contain more energy than the high-frequency bands. Furthermore, discarding high-frequency

components merely results in a loss of sharpness, whereas the overall image information is essentially preserved. This fact is exploited in the next step, the quantization of the transform coefficients.

Quantization is the only lossy step in a video coder. It removes those parts of an image which can be considered negligible to the human observer. This task is also called irrelevancy reduction. As quantization maps the whole range of input values to much fewer output values, this step is irreversible and hence causes information to get lost. The higher the number of input values mapped to one and the same output value, the worse the quality of the video becomes; however, this significantly reduces the number of different values, the so-called “levels”, to be further coded, which is quite beneficial to the entropy coding step.

Entropy coding finally creates the actual data stream (the bitstream) into which individual code words of different lengths are written one after another. As already mentioned in the beginning, an image generally possesses only few high-frequency parts, which are often quantized to zero. Thus, the levels are not uniformly distributed — zeros, for instance, appear very frequently. Entropy coding therefore maps frequently occurring levels to short code words, whereas less frequent levels are mapped to longer code words. These code words of different lengths are then transmitted one after the other to the receiver in the form of a bitstream. The decoder extracts the single code words from the bitstream and transforms them back to the spatial domain, thus reconstructing the original data. Since the decoded image is also needed by the encoder, we reconstruct it here as seen in Figure 2. As entropy coding is completely reversible, we do not have to implement its corresponding inverse step in the encoder.

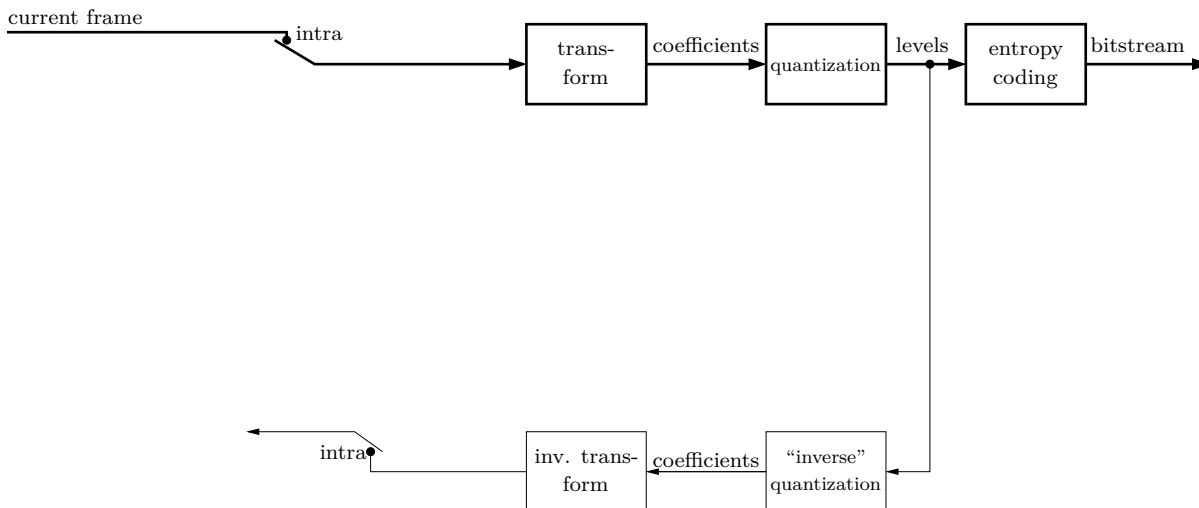


Figure 2: Block diagram of an intra-frame coder with built-in decoder.

The step entitled “inverse quantization” may be a bit confusing. Since different input values are mapped to one single output level, this procedure cannot be inverted in the mathematical sense. In fact, this “inverse” step just performs a mapping from the level indices to the corresponding (now quantized) coefficient values. The coefficient values are usually obtained by computing the mean value of the original input interval. Example: Assuming the original coefficient values 2, 3, 4, 5, 6 have been quantized to level 1, this level 1 is then mapped to the coefficient value 4 every time it is encountered.

The inverse transform reconstructs the spatial domain image from the modified coefficients. This reconstructed image then more or less resembles the original image.

As the exact same reconstructed image is available at both the encoder and the decoder, it can be used for predicting the next image. This is relevant for video coding, also referred to as *inter-frame coding*. The required steps are added in Figure 3.

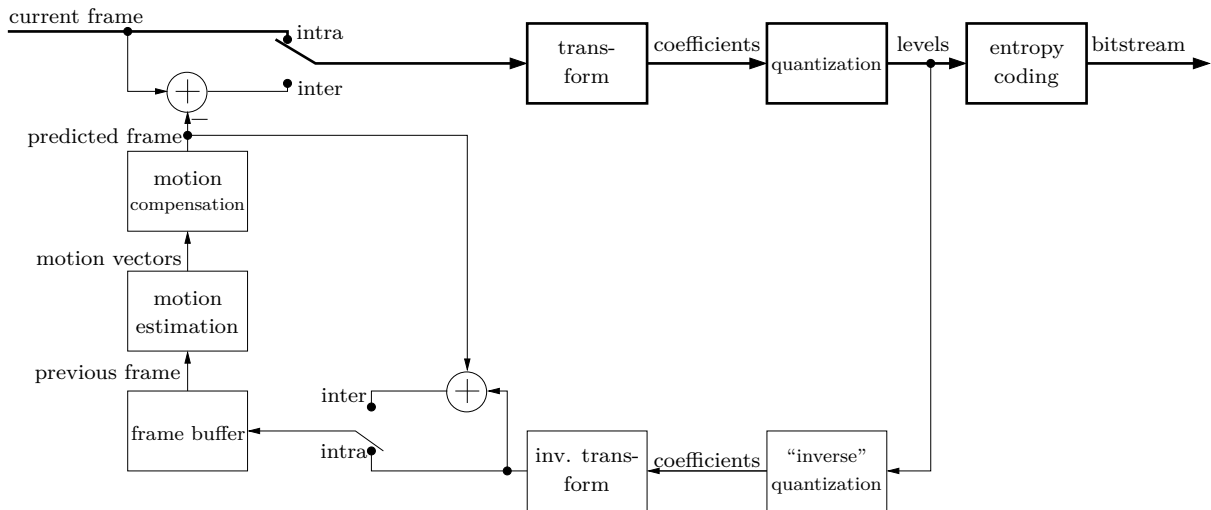


Figure 3: Block diagram of an inter-frame coder.

First of all, the image that has just been decoded is stored in the frame buffer and a new input frame is read.

Motion estimation now tries to find image areas that are identical in both the previous (stored) and the current frame. In doing so, the inter-image motion of objects is taken into account; e.g., the offset of a flying ball is searched for and the gained knowledge exploited during coding.

The subsequent motion compensation then creates (predicts) an image which resembles the current frame as closely as possible. The ball in the previous image, for example, is moved to a new position according to the found offset. This motion compensated image can be created by both the encoder and the decoder since it only consists of shifted parts of the previous frame.

By exploiting motion information, only the difference signal between the current frame and the predicted one has to be encoded. This is considerably more efficient than coding the original frames since it is known from experience that large parts of an image stay the same for many frames of a video sequence (e. g., the background) and thus do not have to be encoded anew in each frame. The individual steps for coding a video sequence do not differ from those of an intra coder. Note, however, that since only the difference image is encoded, the predicted image must be added again after the inverse transform in order to reconstruct the current video frame. This can also be seen in the diagram.

The structure as depicted in Figure 3 is also used for the currently used video codecs H.264 Advanced Video Coding (AVC) and its successor H.265 High Efficiency Video Coding (HEVC). The codec implemented during this lab course is similar but might be simplified at some points to facilitate the understanding and the implementation of a video codec. However, the basic principle of *hybrid video coding* is the same and gives a deep understanding on how a video codec works. Additional information and references on differences between the self-implemented codec and the currently used codecs are given at the end of some chapters.

Outline of the Lab Experiments

Lab Experiment 1 The first experiment of this lab course discusses the employed programming environment MATLAB. Basic capabilities of MATLAB are presented, e. g., computations with scalars and matrices and how to handle functions.

Lab Experiment 2 In the second lab experiment, entropy coding is introduced as the first method for data reduction. By performing entropy coding, redundant parts are removed, meaning that repetitions within the image data are exploited. This method is based on concepts from information theory; in particular, you will learn about Huffman coding in detail. This experiment comprises two sessions.

Lab Experiment 3 The third lab experiment introduces another method for data reduction. By means of quantization, irrelevant, i. e., imperceptible, parts of the video data are removed. You will work with distortion-rate ($D(R)$) curves and learn to treat important and less important parts of an image differently.

Lab Experiment 4 The previous experiments have only made use of spatial correlations between pixels of single images for redundancy reduction. Now you will

learn about methods that also exploit temporal correlations between consecutive frames of a video sequence. By employing motion estimation, the image content of the current image can be predicted from the previous one so that only the difference between the predicted image and the current image has to be further processed. This lab experiment will allow you not only to implement motion estimation and compensation, but also to work on your own improvements.

Lab Experiment 5 In the fifth lab experiment, a frequency-domain representation is used to exploit statistical properties and make entropy coding even more efficient. You will learn about the Discrete Cosine Transform (DCT) which transforms data from the spatial domain into the frequency domain. With suitable strategies, you will significantly reduce the amount of data required for these frequency-domain representations. At the end of this experiment, you will have implemented all components of a fully-functional image coder. This experiment comprises two sessions.

Lab Experiment 6 In this lab experiment, you will build a complete video codec. For this purpose, you will need the already implemented interfaces from the previous experiments. After creating a basic coder and decoder, you are then expected to improve them. The goal is to develop a codec which achieves a higher coding efficiency or works faster (or both) than the basic codec. Several options for improvements are described in the previous experiments as well as in the optional part of this experiment. Which of these options you choose to implement is entirely up to you.

At the end of Lab Experiment 6.3, you should have finished your work on your enhanced video codec and hand it in.

Lab Experiment 7 In Lab Experiment 7, the developed codecs of the individual groups are compared to one another. For this, you will have the chance to shortly talk about your work. Additionally, D(R) curves for unknown video material will be created and a subjective video test conducted using the codecs of a few selected groups. The overall results are finally presented by the tutors. This experiment will span two short sessions.

Structure of the Experiment Descriptions

Before each session, you are required to thoroughly read the description of the current experiment and familiarize yourselves with the theoretical concepts. You might also want to take a look at the lecture notes on *Image and Video Compression* [1] for further comprehension.

Each experiment of this lab course comprises a theoretical part, a part with exercises (to be prepared beforehand!), a practical part, and an optional part:

The **theoretical part** provides you with the basics for each lab experiment, general hints for the implementation, and information about the number of sessions scheduled for that experiment.

In the **preparatory part** you will find exercises which have to be worked on diligently before the actual lab experiment takes place. They serve to provide you with a better understanding and will make the later implementation of the functions easier.

The **practical part** specifies the functions to be written during a session and contains helpful hints for implementing them. The theoretical part of each experiment indicates the available time for this. The aim of this lab course is to realize a fully-functional video codec. Hence, the individual experiments are naturally correlated and it is essential to complete each practical part in order to be able to move on. For that reason, you are also provided with test functions with which you can verify your implementations.

Finally, most lab experiments include an **optional part**. By solving the assignments in this part, you can potentially improve the efficiency of your coder tremendously. So, depending on the time left at the end of each session, do try and realize a selection of these functions! In the second part of Lab Experiment 6, you will enhance the basic coder (which is made up of the functions of the practical part) by some optional functions and thus improve its performance. On the last day of this lab course, the improved coders of the different groups are then compared to one another.

Lab Experiment 1

Introduction to MATLAB

Implementing a video codec is a programming task. It is usually realized by means of low-level programming languages in order to guarantee the most efficient implementation possible. However, this requires a lot of experience with the corresponding programming environment and a good grasp of the hardware platform.

This lab course is not about building the fastest possible video coder or achieving minimal hardware cost. Instead, a high flexibility and being able to easily learn the employed programming language are of importance. Therefore, an introduction to the programming language MATLAB is given at the beginning of the lab course. After this first session, you should be able to solve simple tasks with regard to image representation and manipulation on your own.

1.1 What is MATLAB?

MATLAB is a simulation language which, because of its versatility and considerable functional range, represents the industry standard in this field. MATLAB is very much suited for testing new ideas and quickly obtaining first results. Many challenging programming tasks are first implemented in MATLAB so that their functionality may be tested on a conventional PC, before the results are ported to the desired target platform.

1.2 How do I learn MATLAB?

Because of said versatility, it takes a lot of time to gain a good overview of all MATLAB commands and other functions. Nevertheless, it is possible even for MATLAB beginners to quickly obtain first results and then broaden one's knowledge step by step. This is the way we will go about in this session.

There are different ways of familiarizing yourselves with MATLAB:

- Trial and Error

MATLAB is interactive and does not hold back on error messages. It also offers an extensive help function via the command line.

- MATLAB Documentation

The MATLAB environment provides a comprehensive documentation both as part of the installation and online via the manufacturer's websites. The latter can also be viewed on computers without MATLAB, e. g., on <http://www.mathworks.de/de/help/matlab/>

- Internet and Books

Entering 'MATLAB' into a search engine returns quite a lot of useful websites on MATLAB. In addition, there are also many books about MATLAB.

1.3 How do I work with MATLAB?

In order to open MATLAB, open a terminal and enter `matlab & disown`. The terminal can be closed afterwards. The MATLAB GUI (graphical user interface) similar to Figure 1.1 opens. The window of the MATLAB programming environment comprises three sub-windows:

1. Command Window

Here, commands are entered via the keyboard. Each command is interpreted immediately. Depending on the command, the result shows up in the command window or in a separate window. The command window is case-sensitive!

2. Workspace

Here, all variables that are created during a session appear by name, size, and data type.

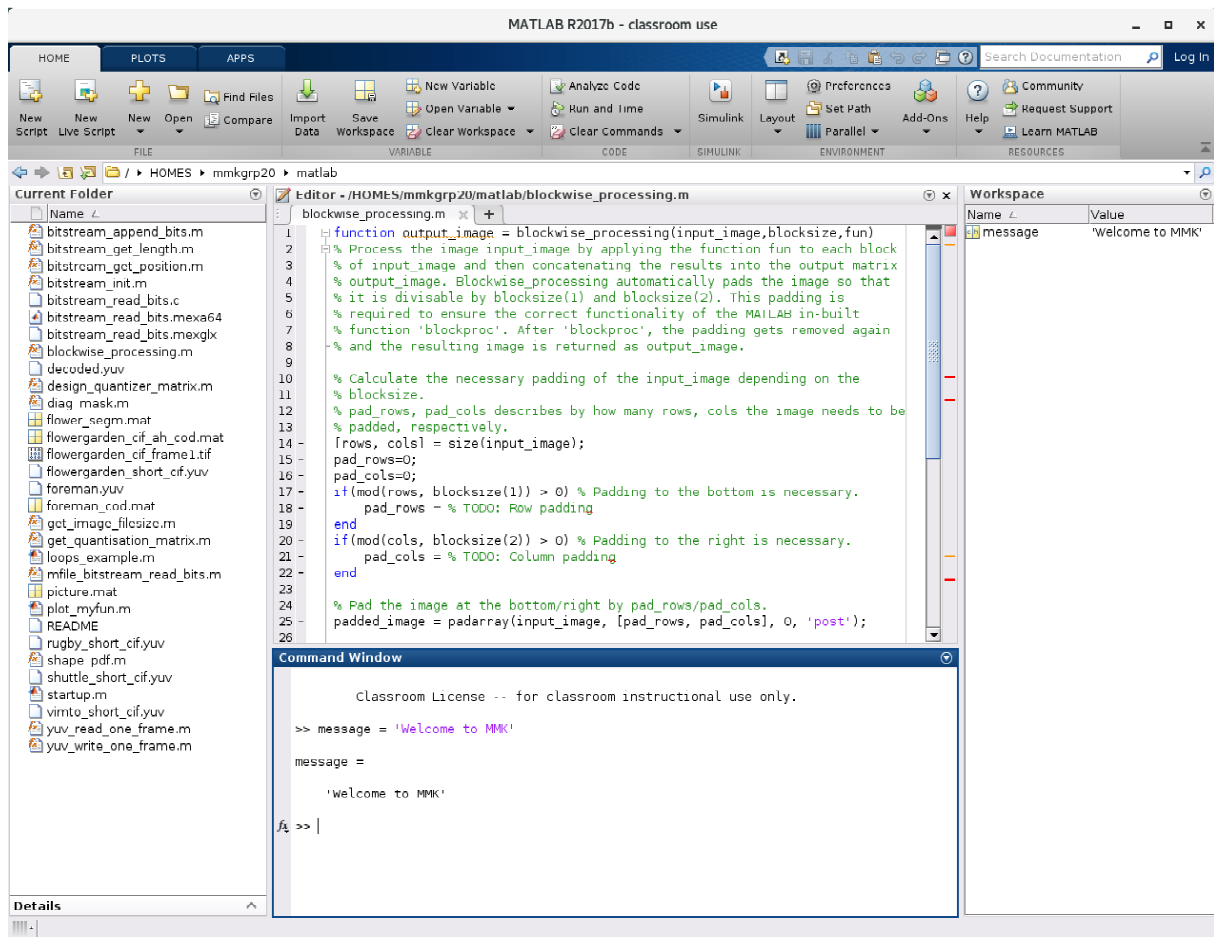


Figure 1.1: Window of the MATLAB programming environment.

3. Command History

Here, all commands of the current session are displayed. By clicking on them, they can be run again.

The MATLAB GUI can be exited by

- choosing `exit MATLAB` from the File menu, or
- entering `quit` or `exit` in the command window.

Now, type `1+1` in the command window and press Enter. If the expected result is shown, you have executed your first MATLAB command successfully.

1.4 How do I get help on MATLAB?

The help functions of MATLAB can be accessed by doing one of the following:

- Command Line

In the command window, enter `help` and a long list of help topics will appear. You can choose one by clicking on it. At the beginning, you will need very few of them. Enter `help ops` to learn about the operators, and `help elmat` for elementary matrix functions.

The `help` command will become your most important companion in this lab course!

- MATLAB Documentation

In the **Help** menu, choose **MATLAB Help**. Numerous examples can be found under **Demos**.

1.5 Data Types in MATLAB

The most important data types in MATLAB are:

- double

Numbers are stored in double-precision floating-point format. In MATLAB, *numbers* are real numbers, complex numbers, and vectors/matrices with real or complex elements.

- char

Strings are stored as vectors, whose elements are single characters. Strings are enclosed in single quotes (`'`).

- logical

Logical variables are the results of relational operators or logical operators (cf. `help ops`) and their values are either true (1) or false (0).

Practical Part: _____

Exercise: 1.1: In the command window, create two 2×2 matrices by entering

```
A = [ 1 2; 3 4]
```

and

```
B = [ 5 6; 7 8 ];
```

Note the semicolon at the end of the second assignment and compare the outputs of both command window and workspace.

Exercise: 1.2: In the command window, enter

```
File1 = 'showfile.m'
```

and

```
File2 = 'hidefile.m';
```

Compare the outputs of both command window and workspace.

Exercise: 1.3: In the command window, enter

```
TRUE = det(A) > det(B)
```

and compare the outputs of both command window and workspace. Is this the expected result? Explain! (Use `help det` to determine what `det` does.)

Exercise: 1.4: In the command window, enter

```
whos TRUE
```

```
whos
```

and compare the outputs of both command window and workspace.

1.6 Images and Matrices

Since MATLAB operates on matrices, images are treated as such, too. To address single pixels, rows, columns, and parts of images, the corresponding operations for matrices may be used.

1.6.1 Displaying, Reading and Saving Images

In your working directory, you can find the file `flowergarden_cif_frame1.tif` which contains an image in TIFF format. In the following exercises, you will make use of the commands:

<code>imshow</code>	displaying images
<code>imread</code>	loading images
<code>imwrite</code>	saving images

The TIFF format is only one of numerous image formats. In this lab course, images are represented as matrices (with data type `double`), the elements of which are real numbers ranging from zero to one. This is achieved by converting the TIFF-formatted image into data type `double`.

Practical Part: ---

Exercise: 1.5: In your working directory, open the file `flowergarden_cif_frame1.tif` with an image viewing program. Keep the window in the background in order to use it as a reference image for the exercises below.

Exercise: 1.6: In the command window of the MATLAB environment, enter the command `imshow('flowergarden_cif_frame1.tif')`. The same image is shown in a new window, this time by MATLAB.

The `imshow` command displays an image, but does not save it as a variable in the workspace.

Practical Part: ---

Exercise: 1.7: In the command window, enter

```
fg_tiff = imread('flowergarden_cif_frame1.tif') .
```

This command reads the image contained in the file `flowergarden_cif_frame1.tif` and saves it as a variable `fg_tiff` in the workspace. It is listed there along with its size `288×352` and a data type which will not be used in this lab course. At the same time,

the luminance values of the pixels are printed in the command window. Obviously, they range from 0 to 255.

Practical Part:

Exercise: 1.8: Convert the variable `fg_tiff` into a variable `fg` of format `double` with values ranging from 0 to 1. Use the command `double`. You can learn about its function by entering `help double`. Commands for arithmetic and other operations can be displayed by entering `help ops`. Look out for possible error messages!

If successful, a new variable `fg` should appear in the workspace. It is of the same size as `fg_tiff`, but its data type is `double`.

You will only find out if the conversion was successful by viewing the new image. You can again use the `imshow` command, as it is also able to display matrices containing double values ranging from 0 to 1 as images.

Practical Part:

Exercise: 1.9: To be able to compare the image in TIFF format to the new one, you first have to create a new figure with the command `figure`. You can thus prevent the `imshow` command from overwriting the contents of the first figure. Now display the contents of the variable `fg` as an image and compare it to the TIFF-formatted version.

If everything has worked out properly, you should not be able to notice any difference.

Finally, the image `fg` is to be saved again as a TIFF image in your working directory. The command `help imwrite` shows the numerous options. The easiest way to tell `imshow` about the desired TIFF format is the following:

Practical Part:

Exercise: 1.10: Save the image `fg` by writing it into a file using the command

```
imwrite({variable}, '{file name}.tif') .
```

Do not forget to enclose the file name in single quotes! Of course, you should name your file something other than `flowergarden_cif_frame1` so as not to overwrite the original image. In your working directory, a new file in TIFF format appears now. With an image viewing program, you can verify whether it still contains the desired image.

1.6.2 Creating Matrices

Using the data type `double` for images makes it possible to exploit MATLAB's extensive numerical computation capabilities for manipulating image content. In a first exercise, you will get to know some simple functions for creating certain kinds of matrices. They are:

<code>zeros(m,n)</code>	matrix of size $m \times n$ containing only zeros,
<code>ones(m,n)</code>	matrix of size $m \times n$ containing only ones,
<code>eye(n)</code>	identity matrix of size $n \times n$.

Practical Part: ---

Exercise: 1.11: Create a white image of size 288×352 .

Exercise: 1.12: Create a black image of size 352×288 .

Exercise: 1.13: Create a square white image with a black line from the upper left corner to the lower right corner.

1.6.3 Parts of Matrices

White or black images are not all that exciting. To be able to go into further detail with the image content, individual pixels must be accessible. This can be done by the following commands for addressing the individual elements of matrices. In MATLAB, indices are always counted from 1. This is different from languages like C, where the first element has index 0.

<code>A(m,n)</code>	element in row m and column n of matrix A ,
<code>A(m,:)</code>	all elements in row m of matrix A ,
<code>A(:,n)</code>	all elements in column n of matrix A .
<code>[M,N]=size(A)</code>	returns the number of rows and columns of A (cf. <code>help size</code>).

If indices m and n are vectors as well, several elements, rows, and columns can be addressed simultaneously.

Such index vectors are created either by

```
index = [1 2 3 4 5 6 7 8 9 10]
```

or even simpler by

```
index = 1:10 .
```

The step size from one index value to the next one can also be greater than 1 or even negative:

```
index1 = 1:10
index2 = 1:2:10
index5 = 1:4:10
indexm1 = 10:(-1):1
```

Practical Part:

Exercise: 1.14: Display the first row of image `fg` using `imshow`.

Exercise: 1.15: Display the last column of image `fg` using `imshow`.

Exercise: 1.16: Display the first ten rows of image `fg` using `imshow`.

Exercise: 1.17: Display a mirror-inverted version of image `fg` using `imshow`.

Exercise: 1.18: Display the upper left quarter of image `fg` using `imshow`.

Exercise: 1.19: Display a down-scaled version of image `fg` by addressing only every second pixel in both dimensions using `imshow`.

1.6.4 Luminance of Images

Until now, you have addressed parts of images, but you have not changed the luminance of the displayed pixels. To do so, the pixel values have to be re-computed using arithmetic operations. With the large amount of pixels, the capabilities of MATLAB help us formulate short and succinct calculation instructions. Useful operations for this purpose are:

`A*c` multiplication of a matrix `A` by a scalar `c`

`A*B` multiplication of two matrices `A` and `B`

`A.^c` raising each element of a matrix `A` to the power of a scalar `c`

When multiplying two matrices `A` and `B`, those two matrices must be compatible for matrix multiplication, i.e., the number of columns of matrix `A` must be equal to the number of rows of matrix `B`. Otherwise, MATLAB returns an error message.

The sole exception to this rule is the multiplication of a matrix by a scalar. Here, each element of the matrix is multiplied by the scalar. Note that apart from this, MATLAB does not differentiate between matrices and scalars. A scalar is a matrix of size 1×1 !

This is important when raising matrices to different powers. The operation $\mathbf{A}^2$ means $\mathbf{A}^2 = \mathbf{A} \cdot \mathbf{A}$ and works only for square matrices. Otherwise, $\mathbf{A}$ is not compatible with $\mathbf{A}$ with regard to a matrix multiplication. To square each individual element of $\mathbf{A}$ (as opposed to squaring the matrix $\mathbf{A}$), the command is changed to $\mathbf{A}.^2$. In general, a dot before the actual operator indicates an element-wise operation.

At the beginning, the difference between operations applied to matrices and operations applied to the *elements* of matrices might not be immediately obvious and result in mistakes.

With the help of these operations, the values of matrix elements can be manipulated in different ways. For these manipulations, MATLAB allows a compact notation so that it is also possible for them to be used in the argument of another function, e.g., in the argument of `imshow`.

Practical Part: ---

Exercise: 1.20: Multiply the image in the argument of `imshow` by a scalar. Vary the value of the scalar between 0 and 1.

Exercise: 1.21: Multiply the image in the argument of `imshow` by a scalar. Now also choose values greater than 1 and less than 0 for the scalar! Explain your observations!

Exercise: 1.22: Raise the luminance values of the image in the argument of `imshow` to the power of some scalar. Vary the value of the scalar and explain your observations.

1.7 Scripts and Functions

So far, you have been entering all commands into the command window. This is quite inefficient when a sequence of instructions is to be executed repeatedly. There are two ways to write reusable MATLAB code:

Script: A file with extension `.m` which is processed line by line as if typing those lines into the command window one after another. (cf. `help script`)

A script is called by its name, leaving out the extension.

Function: A file with extension `.m` and keyword `function` in the first line. Functions may expect input values and provide return values. In contrast to scripts, the variables used in functions are given local scope. Therefore, functions are more suited for writing reusable code as they prevent the undesirable accumulation of variables in the workspace. (cf. `help function`)

Just like the already available commands, self-written functions are called by their function names.

Henceforward, always use scripts and functions when writing your program code for the exercises below (and those following thereafter)!

Practical Part: _____

Exercise: 1.23: Write a MATLAB function `myfun` which computes $y(x, c) = x^c$ using the parameter c . Matrices should also be permissible as input x . (The function should thus have the same functionality as in **Exercise 1.22**.) Do not forget to write a suitable help text!

Test your function by using the `imshow` command to display the return matrix for different values of c and also call the help text of your function.

1.8 Program Flow and Debugging

To control the program flow, MATLAB provides the common conditional constructs `for`, `while`, `if`, `break`, etc. known from other programming languages. Details on how to use them can be obtained via `help`.

The MATLAB debugger is a very useful utility to find errors in your written function, but it can also be used to understand and analyze given implementations. You should always make use of the debugger if your implementations do not work as expected.

Practical Part: _____

Exercise: 1.24: Open the script `loops_example.m` using the MATLAB editor and give some thought to its functionality. Afterward, start the script and confirm whether

your thoughts were correct (you have to proceed with the next steps to be able to do that!).

The script contains a bug that makes MATLAB throw an error, so we need to debug it. Remove (use a comment) the `pause` command and save the file. Now set a break point at line 9 and restart the script. Use the *step* button from the MATLAB GUI or the *F10* key on your keyboard to step to the next line. Hover the mouse cursor over the variable names in the script code and explain the shown values.

Double-click on the `cols` variable in your workspace to open the variable editor. Hit *F10* again and watch how the vector changes. Click on the break point to remove it and set another break point at the `imshow` line. Click on the *continue* button to run all lines until the program flow again arrives at the `imshow` break point. In order to step into the `imshow` function, use the *step in* button—you can return to the script using *step out*. Proceed with *next* and *continue* and watch the variable values until the error is reached. What may be the problem?

When you have found the error, correct it, re-add the `pause` command, save the file, remove the break point and restart the script once again to test its functionality.

1.9 Plotting Curves

In this lab course, you will plot curves time and again in order to display image distortions as a function of a certain parameter, for instance. For this, the `plot` command is used.

Practical Part:

Exercise: 1.25: Open the script `plot_myfun.m` and adjust the `plot` line for a graphical output of your function `myfun`. Execute the script.

Exercise: 1.26: Now the curves for `c=(0.5:0.5:2)` shall be displayed graphically. For that purpose, extend the script `plot_myfun.m` by a `for`-loop. Use the commands `hold on` and `hold off` to prevent deletion of the previously plotted curves.

Compare the curves with your observations from **Exercise 1.22**.

1.10 Block Processing

Since our video coder works in a blockwise manner, we will now implement the MATLAB function `blockwise_processing` taking care of the application of a command onto each block of an image.

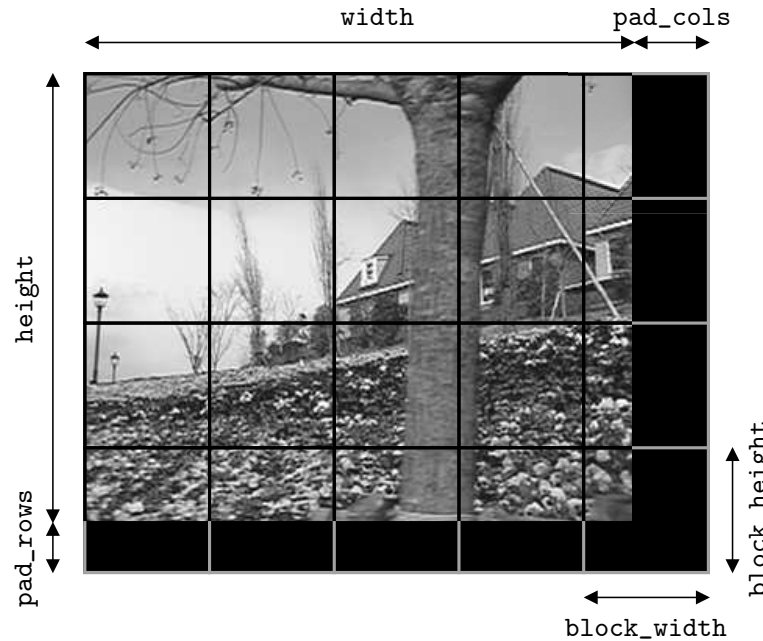


Figure 1.2: Image of size $[\text{height}, \text{width}]$ subdivided into blocks of size $[\text{block_height}, \text{block_width}]$. The necessary row/column padding is denoted as `pad_rows/pad_cols`, respectively.

Practical Part:

Exercise: **1.27:** In your working directory, you find the file `blockwise_processing.m` which contains the code skeleton for the MATLAB function `blockwise_processing(input_image, blocksize, fun)` that shall be implemented. Thereby, the `input_image` is the image that gets blockwisely processed, `blocksize` describes the size of one block as $[\text{height}, \text{width}]$ and `fun` is the function that gets executed on every block of the image. Familiarize yourself with the code skeleton, then implement the missing elements (marked with `% <- TODO`):

1. Row padding: Calculate the number of rows that need to be appended at the bottom of the image, so that the padded image can be divided by `blocksize(1)`.

2. Column padding: Calculate the number of columns that need to be appended to the right of the image, so that the padded image can be divided by `blocksize(2)`.
 3. Remove padding: Remove any padding from before.
-

Practical Part: ---

Exercise: 1.28: Apply the given function `diag_mask` to the example image and display the result by using `imshow(diag_mask(fg))`.

Now apply the function `diag_mask` block by block (using `blockwise_processing`) to 50×100 pixel blocks of the example image and display the result via `imshow`. *Hint:* Pass the function name `diag_mask` to `blockwise_processing` in single quotes.

1.11 Image Sequences

So far, we have been working with a single example image. In this lab course, functions for reading (`yuv_read_one_frame.m`) and writing (`yuv_write_one_frame.m`) single images (*frames*) from video sequences are provided.

Practical Part: ---

Exercise: 1.29: Write a program which displays the first 5 frames of the sequence 'flowergarden_short_cif.yuv'. Use `pause(0.5)` to display each frame for half a second. *Hint:* Use only the Y return values (= luminance channel) of the `yuv_read_one_frame` function. The image dimensions are the same as before!

1.12 Cell Arrays

In a matrix, all rows/columns possess the same number of elements and all elements possess the same data type. That is why matrices are well-suited for displaying the

luminance values of an image. For different purposes, e.g., a look-up table, more creative freedom is desirable.

To that end, MATLAB provides a data structure called *cell array*. A cell array is a rectangular arrangement of matrices, where each of those matrices may be of a different dimension and data type. In short, a cell array is a matrix of matrices.

A cell array can be built by one of two ways. Similar to matrices, the elements of a cell array are enclosed in curly braces:

```
cells = {zeros([2,3]), eye(3); pi, 'MMK'}
```

The elements are then matrices, scalars, or strings. An empty cell array can also be built via the command `cell`. The elements are filled by assignments.

As an example, we build a cell array which contains the names of several geometrical objects along with their vertices. Depending on the object (e.g., point, line, triangle), there are different numbers of vertices, though. Additionally, the vertices can be made up of either two (plane, 2D objects) or three (spatial, 3D objects) coordinates.

First of all, we build a cell array

```
D = cell(3,2)
```

which shall contain the name and the vertex matrix for each of three objects. Then, we fill in the names of the objects

```
D{1,1} = 'point'  
D{2,1} = 'line'  
D{3,1} = 'triangle'
```

as well as the matrices, in which each row contains the two-dimensional coordinates of a vertex

```
D{1,2} = [0 0]  
D{2,2} = [0 0; 0 1]  
D{3,2} = [0 0; 0 1; 1 0.5] .
```

With the commands `celldisp` and `cellplot`, the content and structure of a cell array can be viewed.

Practical Part: ---

Exercise: 1.30: Build the cell array from above and extend it such that it contains the names and the vertex coordinates of each of the geometrical objects

point, line, triangle, rectangle, cuboid.

Consider for each object the number of vertices it possesses and how many coordinates you need for describing these points. Use `celldisp` and `cellplot` to test the correctness of both the desired structure and the content of the cell array.

Exercise: 1.31: Now write a MATLAB function `plot_geometries(Cell_array)` which plots each of the two-dimensional geometrical objects (point, line, triangle, rectangle) in `Cell_array` in its own window! The function should automatically label each window with the name of the object (use `title('label')`!). To plot, use the function `plot(X,Y)`, which you already know from Section 1.9. Bear in mind that the last coordinate has to be equal to the first coordinate when plotting closed forms, and also pay attention to the order of the coordinates!

1.13 Maintaining the Workspace

Examine the MATLAB workspace. More likely than not, many unwanted variables have accumulated there. You may possibly want to save some of them for later use while deleting the ones that are not needed any longer.

For this, MATLAB offers the following commands (amongst others):

<code>whos</code>	displays the variables by name, type, and size
<code>save</code>	saves select variable in file
<code>clear</code>	deletes variable
<code>load</code>	loads variable from file

Practical Part: _____

Exercise: 1.32: Save the variables from **Exercise 1.1** in a file and clear the workspace.

1.14 Summary

With this short introduction, you have gained a first impression of MATLAB. In the following experiments, you will make use of your acquired knowledge, but you will also have to add to it by learning new things. Exploit the possibilities indicated in Section 1.2 to do so.

Lab Experiment 2

Entropy Coding

The task of a video codec is to reduce the amount of data when transmitting a video. To show the necessity for data reduction, consider the following example: With an uncompressed grayscale video signal in VGA resolution (640×480 pixels) and 25 frames per second, there is a data amount of $640 \cdot 480 \cdot 25 \cdot 8 = 61 \cdot 10^6$ bits/second, assuming the luminance values are digitized with 8 bits/pixel. The data cannot be transmitted efficiently at such high bitrates. However, using data reduction procedures, high bitrates are no longer necessary.

Data reduction is basically achieved by applying two principles:

- *Redundancy reduction* (this lab experiment) and
- *Irrelevancy reduction*

Irrelevancy reduction removes *unnecessary*, e. g., barely visible, information from the signal. This *lossy* procedure for data reduction will be dealt with in Lab Experiment 3.

In this lab experiment, we deal with *redundancy* reduction, which exploits repetitions or correlations within a signal. The coding methods which implement this principle work in a lossless manner and are hence reversible. In particular, we will focus on Huffman coding, which maps source symbols (in our case the image data) to code symbols. In principle, the frequently occurring symbols are mapped to short code words and the less probable symbols are mapped to longer code words. This is how ‘variable length codes’ (VLC) are created. Because of the variable length, those codes are very prone to errors

when transmitting them: one bit error results in coding failure due to synchronization loss.

At first, some basics on information theory are presented. Then, Huffman coding, which is normally used in a video codec to code different data such as quantized DCT-coefficients, motion vectors etc., is described. In the practical part, the principle of Huffman coding is first developed for coding grayscale values of an image. During the first session of this lab experiment, you will create a Huffman table for arbitrary input signals (Section 2.2.1). During the second session, you will write the functions which make reading/writing the resulting code words from/to the bitstream possible (Section 2.2.2).

Acknowledgment: The authors thank Prof. Dr.-Ing. habil. Guenter Soeder for making available his script [2], which provided a lot of suggestions for the design of Sections 2.1 and 2.2.

2.1 Basics on Information Theory

To clarify the basics on information theory, we consider a random *discrete-time information source*, which generates a sequence of source symbols

$$q_n = q_1, q_2, q_3, \dots, q_N \quad (2.1)$$

with sequence length N . $q_1, \dots, q_N$ can correspond to the pixels of an image to be coded, for example. We further assume that the information source is discrete-valued and that the individual symbols from the *symbol set* with elements S_m are taken from a finite pool of values. S_m represent the possible grayscale values of the pixels. This means

$$q_n = S_m \quad , \text{ where } n = 1, 2, 3, \dots, N \quad \text{and} \quad m = 1, 2, 3, \dots, M \quad (2.2)$$

M is the number of symbols in the *symbol set*. Reliable statistical assertions regarding the information source can only be made if the sequence length N is several times greater than the number of possible symbols M in the symbol set.

2.1.1 Symbol Probabilities

Assuming stationary signals, the *symbol probability* of the n -th symbol of the sequence being equal to the m -th element of the symbol set is given by

$$p(q_n = S_m) = p_m \quad , \quad (2.3)$$

where the probabilities are independent of the position n due to stationarity.

2.1.2 Information Content and Entropy

With the probability (2.3) that the n -th symbol of the sequence is equal to the m -th element of the symbol set, the *information content* of this symbol is defined as

$$I_m = \text{ld} \left(\frac{1}{p_m} \right) = -\text{ld}(p_m) . \quad (2.4)$$

$\text{ld}(\cdot)$ is the logarithm to the base of two (*binary logarithm*). When calculating (2.4) numerically, the pseudo unit “bit” is added.

The information content of a symbol is always positive or - in the case of the certain event with symbol probability 1 - equal to 0. The less probable a symbol, the greater the uncertainty of that symbol and the greater its information content.

The *entropy* of a digital source is determined by

$$H = \sum_{m=1}^M p_m \text{ld} \left(\frac{1}{p_m} \right) = - \sum_{m=1}^M p_m \text{ld}(p_m) . \quad (2.5)$$

This corresponds to the average information content of the source and hence possesses the unit $\frac{\text{bit}}{\text{symbol}}$. H is also a measure for the “randomness” of the source or the uncertainty about a source.

2.1.3 Decision Content

Independent of the number of symbols in the symbol set, the maximum value of the entropy is reached if all symbols have the same probability, i. e., if for all $m = 1, 2, \dots, M$

$$p_m = \frac{1}{M} . \quad (2.6)$$

With this assumption, applying (2.5) yields the entropy

$$H_0 = \sum_{m=1}^M \frac{1}{M} \text{ld}(M) = \text{ld}(M) . \quad (2.7)$$

This maximum value of the entropy is also called *decision content* in the literature.

If all M possible symbols are not equally probable, then the information source always possesses a smaller entropy than this limit $\text{ld}(M)$. Thus the range of the entropy is

$$0 \leq H \leq \text{ld}(M) . \quad (2.8)$$

2.1.4 Average Code Word Length and Redundancy

Given an M -ary source symbol sequence with statistically independent sequence elements, the entropy H_q and the decision content H_0 can be computed with (2.5) and (2.7), respectively:

$$H_q = - \sum_{m=1}^M p_m \text{ld}(p_m), \quad H_0 = \text{ld}(M) \quad (2.9)$$

If the M symbols are equally probable, both H_q and H_0 are the same and we speak of a redundancy-free source. For this case, entropy coding (also: source coding) is not required.

In the following, we assume sources with different symbol probabilities, whose M -ary original sequences are transformed into binary sequences via source coding:

$$\begin{aligned} S_m &= \{S_1, S_2, \dots, S_M\} \\ &\Downarrow \text{Source coding} \\ C_m &= \{C_1, C_2, \dots, C_M\} \end{aligned} \quad (2.10)$$

Each code word C_m is made up of l_m binary digits. This means that a symbol (grayscale value of a pixel) S_m is mapped to a code word C_m of length l_m . The mapping should be performed in a way such that the *average code word length* is kept as short as possible. The average code word length l_{av} can be calculated as a weighted average over the M possible code words:

$$l_{av} = \sum_{m=1}^M p_m l_m, \quad \text{where } l_m : \text{ number of binary digits of the } m\text{-th code word.} \quad (2.11)$$

In order to minimize this value, more frequently occurring symbols are represented by shorter code words and rarely occurring ones by longer code words.

According to the source coding theorem, the average code word length will never be less than the source entropy:

$$l_{av} \geq H_q \quad (2.12)$$

The difference between the average code word length and the actual entropy is called *redundancy*:

$$\Delta R = l_{av} - H_q \quad (2.13)$$

It describes that portion of a message that does not contain any information. If only one and the same symbol is being emitted, for example, then the whole message is redundant.

By combining several source symbols, the redundancy can be further reduced and the average code word length l_{av} decreased. A source with statistically independent, equally probable symbols is called redundancy-free.

2.2 Huffman Coding

In the following, we assume that the source symbols are taken from a set of 6 symbols and that they are statistically independent. In 1952, David A. Huffman developed an algorithm for the construction of optimal prefix codes for these conditions. Here, we only take binary codes into consideration.

In this context, *optimal* means that the average code word length defined in (2.11) is minimal for the given symbol probabilities. The property “*prefix*” declares that no code word may be the prefix (i.e., the beginning) of a longer code word. A prefix code is directly decodable.

The Huffman algorithm is given here without derivation or proof:

1. Sort the symbols, e.g., by descending probability.
2. Merge the two least probable symbols into one new symbol.
3. Repeat steps 1 and 2 until only two symbols are left.
4. Beginning with the last symbol pair, binary coding is done step by step by appending “1” to the partial code with lower and “0” to the one with higher probability.

Example: Without loss of generality, we presume that the symbols are already sorted according to their probabilities:

$$p(A) = 0.27, \quad p(B) = 0.26, \quad p(C) = 0.24, \quad p(D) = 0.10, \quad p(E) = 0.08, \quad p(F) = 0.05$$

By pairwise merging and sorting, we obtain the following symbol combinations in 4 steps (resulting probabilities are in parentheses):

1. A(0.27), B(0.26), C(0.24), D(0.10), EF(0.13)
2. A(0.27), B(0.26), C(0.24), DEF(0.23)

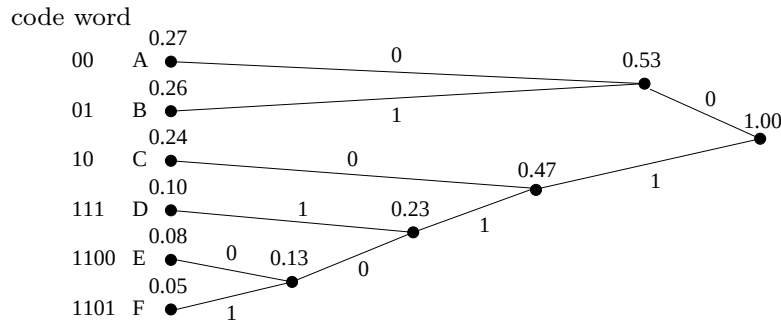


Figure 2.1: Huffman code tree.

3. A(0.27), B(0.26), CDEF(0.47)

4. AB(0.53), CDEF(0.47)

The actual assignment of the symbols is done backwards according to steps 4 through 1:

1. $AB \rightarrow 0x$; $CDEF \rightarrow 1x$;
2. $DEF \rightarrow 11x$; $A \rightarrow 00$; $B \rightarrow 01$; $C \rightarrow 10$;
3. $D \rightarrow 111$; $EF \rightarrow 110x$;
4. $E \rightarrow 1100$; $F \rightarrow 1101$;

In this representation, an x denotes that further bits have to be appended in the next step. In contrast, the final code assignments are underlined.

By comparing the average code word length defined in (2.11)

$$l_{av} = 2 \cdot (0.27 + 0.26 + 0.24) + 3 \cdot 0.10 + 4 \cdot (0.08 + 0.05) = 2.36 \text{ bit/Symbol}$$

to the entropy (here 2.36 bits/symbol), the efficiency of Huffman coding for the given probability distribution is evident.

The construction of Huffman codes is frequently done using a tree structure. Figure 2.1 shows one such tree for the example above.

Note that Huffman codes can also be used for data reduction of a binary information source if several source symbols are combined and thus coded together.

Preparation: _____

For the following exercises, an information source with statistically independent symbols is given. The number of symbols in the set is $M = 8$ and the occurrence probabilities are:

$$\begin{aligned} p(A) &= 0.01, & p(B) &= 0.15, & p(C) &= 0.05, & p(D) &= 0.2, \\ p(E) &= 0.02, & p(F) &= 0.02, & p(G) &= 0.05, & p(LZ) &= 0.5 \end{aligned}$$

Preparation: 2.1: Compute the source entropy H .

Preparation: 2.2: What is the average code word length of a binary code if the different occurrence probabilities of the source symbols are not considered for coding (i. e., if there is no actual coding done)?

Preparation: 2.3: Compute the redundancy of this “coding”.

Preparation: 2.4: Construct a Huffman code for the given occurrence probabilities. Draw the corresponding tree structure.

Preparation: 2.5: Compute the average word length of the Huffman code.

Preparation: 2.6: Compute the redundancy after the Huffman coding.

In this lab experiment, a Huffman encoder as well as a decoder are implemented. In the first part, a Huffman table is constructed based on the input symbols and the corresponding occurrence probabilities. In the second part, the Huffman encoder, which transforms the input signal into code words, is implemented. Afterward, the decoder, which transforms the transmitted code words back to the signal, is implemented.



Figure 2.2: Image from the “flowergarden” sequence. The small outlined block contains the luminance values that are to be coded via Huffman coding.

The Huffman coder is developed on the basis of a small block of an image taken from the “flowergarden” sequence (Figure 2.2).

The small outlined section contains the following 22 luminance values:

```
input_signal =
    244    244
    245    244
    243    244
    243    244
    244    244
    243    244
    244    244
    245    244
    243    244
    242    242
    240    241
```

Each pair of successive luminance values is mapped to one code word, thus exploiting the signal redundancy. The most frequently occurring symbols get assigned the shortest code words. At the end, we will compute the required amount of bits and compare it to the storage space required for the uncoded case.

Preparation: _____

Preparation: 2.7: Calculate the number of bits needed for the uncoded storage of the 22 luminance values in `input_signal` given above when one sample takes up 8 bits of storage space. Do not use hyper symbols. Also calculate the source entropy H and give the resulting minimum storage size for this signal if an optimal entropy coder were used.

2.2.1 Part One: Huffman Table

The function

```
huffman_table=create_huffman_table_from_signal(input_signal)
```

creates a probability table (`par_probability_matrix`) based on the occurrence probabilities of the `input_signal` symbols. In turn, this function calls the function

```
huffman_table=
create_huffman_table_from_probability(par_probability_matrix) ,
```

which creates the Huffman table (`huffman_table`) with the help of the probability table (`par_probability_matrix`).

Practical Part: _____

In the following exercise, you will first write the function

```
huffman_table=create_huffman_table_from_signal(input_signal) ,
```

which creates a probability matrix from an arbitrary input signal. K source symbols are combined into one hyper symbol, which is then mapped to a code word, i.e., in our case $K = 2$ source symbols are mapped to one code word. In general, K could be an arbitrary number. That is why the function is programmed such that the symbols of one row of `input_signal` make up a hyper symbol and are represented by a code word. In the subsequent exercises, you will then write the function

```
huffman_table=
create_huffman_table_from_probability(par_probability_matrix) ,
```

which is called by your function

```
huffman_table=create_huffman_table_from_signal(input_signal) ,
```

and create the actual Huffman table. Pay attention to your function calls: The return values of a function always have to be assigned to corresponding variables! In Figure 2.4 on page 51, you can once again observe how the two functions work on our example input signal.

Exercise: 2.1: Load the input signal into your working directory using `load('input_signal_huffman.mat')`.

Write the function

```
huffman_table=create_huffman_table_from_signal(input_signal) .
```

In order to construct the Huffman table later on, this function counts how often each unique hyper symbol occurs. For this purpose, use

```
[x,y,z]=unique(input_signal,'rows') .
```

What do the parameters `x,y,z` represent?

In order to count the hyper symbols, create a vector whose length is the same as the vector which contains each occurring hyper symbol exactly once. Initialize this vector with

```
symbol_count=zeros(size(??,1),1) .
```

You have to choose the vector `(??)` correctly from `x,y`, or `z`. Fill the vector `symbol_count` with the occurrence frequencies of the unique hyper symbols!

Now create the `probability_matrix`. Fill in the occurrence probabilities along with the corresponding source symbols according to the pattern

```
probability_matrix =
[probability source_symbol1_1 .. source_symbol1_K  %% Hypersymbol_1
 probability source_symbol2_1 .. source_symbol2_K  %% Hypersymbol_2
 ...] .
```

code word	symbol	Huffman table realized as a cell array	
1101	F	huffman_table{1,1} ans= 1101	huffman_table{1,2} ans= F
1100	E	huffman_table{2,1} ans= 1100	huffman_table{2,2} ans= E
111	D	huffman_table{3,1} ans= 111	huffman_table{3,2} ans= D
10	C	huffman_table{4,1} ans= 10	huffman_table{4,2} ans= C
01	B	huffman_table{5,1} ans= 01	huffman_table{5,2} ans= B
00	A	huffman_table{6,1} ans= 00	huffman_table{6,2} ans= A

Figure 2.3: Left: Huffman table of example from preparation. Right: Huffman table realized as a cell array.

Exercise: 2.2: In order to create the final Huffman table from the `probability_matrix` in `create_huffman_table_from_signal`, the function

```
huffman_table=
create_huffman_table_from_probability(par_probability_matrix)
```

has to be written. It is called by `create_huffman_table_from_signal`.

Input variable: `par_probability_matrix`

Return variable: `huffman_table`

The `par_probability_matrix` shall be sorted in ascending order so that the most probable symbol is written in the last row and the least probable one in the first row. Create a new matrix `sorted_probability_matrix` and use the command `sortrows`. Determine the number of possible code words (variable `nr_of_codewords`) by using the `size` command. The matrix is then sorted as follows:

```
sorted_probability_matrix =
0.0909 240.0000 241.0000
0.0909 242.0000 242.0000
```

```

0.1818  245.0000  244.0000
0.2727  244.0000  244.0000
0.3636  243.0000  244.0000

```

The Huffman table is stored in a cell array. In the first column, the code words are listed. In the second one, the corresponding hyper symbols made up of the respective source symbols are listed. In the cells of a cell array, data of varying length can be stored. For Huffman coding, this is necessary due to the variable length of each code word.

In Figure 2.3, the cell array (right) which corresponds to the Huffman table from the preparation exercises (left) is depicted.

Exercise: 2.3: Create a cell array

```
huffman_table=cell(m, n)
```

of size $m \times n$ for all code words `nr_of_codewords` for the Huffman table, with $n = 2$ (one entry for the code word, one entry for the hyper symbol per row). In the second column, the symbols to be coded are stored. Now fill them in using a `for` loop. An element of a cell array is addressed as follows: If, for example, the symbols `3:10` of row number `index` of `matrix` are to be passed to the cell array element `index,2`, then:

```
huffman_table{index,2}=[matrix(index,3:10)]
```

Check the contents of the created Huffman table with `show_huffman_table(huffman_table)`. You should obtain the following result:

```

Huffman_table
: 240 241
: 242 242
: 245 244
: 244 244
: 243 244

```

Next, the symbols have to be mapped to code words. To do so, a Huffman tree which contains the sorted probabilities has to be created. Remember: The least probable symbol is found at the top!

Exercise: 2.4: Create the cell array `huffman_tree`. Use a `for` loop, and for all `nr_of_codewords` code words, assign the probability from `sorted_probability_matrix` to the cells of the first column and the corresponding index `symbol_index` to the cells of the second column. This index is determined by the position of each probability value in the Huffman tree, i.e., the least probable symbol has the position 1 and the most probable one has the last position, which is `nr_of_codewords`. View the Huffman tree with the function

```
show_huffman_tree(huffman_tree) .
```

The Huffman tree should be structured as follows:

```
Huffman_tree
0.0909091 :1
0.0909091 :2
0.181818 :3
0.272727 :4
0.363636 :5
```

Exercise: 2.5: The next loop is the main processing loop: In each iteration, the Huffman tree (`huffman_tree`) is sorted anew, the two least probable symbols are combined and a '1' and a '0' is written into the Huffman table (`huffman_table`) for the two symbols. For this, realize the `while` loop

```
while (size(huffman_tree,1)>1)
show_huffman_tree(huffman_tree)
...
end
```

until all symbols are processed.

- In the first column of the Huffman table (`huffman_table`), the corresponding bits are assigned to the symbols. The least probable one gets a '1' and the most probable one a '0'. As seen in the Huffman tree from the preparation (Figure 2.1), the new entry of '1' or '0' is written BEFORE the possibly already existing code word. You can get the index of the row in the Huffman table via `huffman_tree{1,2}` and `huffman_tree{2,2}`, respectively.

For example

```
huffman_tree{1,2} = 1
huffman_tree{2,2} = 2
```

With the complete `huffman_tree`

```
0.0909091 :1
0.0909091 :2
0.181818 :3
0.272727 :4
0.363636 :5
```

Note: The first iteration yields a 1 and a 2, in the further iterations, this addressing returns a vector which contains the combined symbols. You can process this vector with the help of a `for` loop.

View the Huffman table with

```
show_huffman_table(huffman_table) .
```

In the first iteration (iteration 0 in Figure 2.4), it looks like this:

```
Huffman_table
1 : 240 241
0 : 242 242
: 245 244
: 244 244
: 243 244
```

- Then, the two least probable symbols are combined. For this, create a new cell array

```
new_huffman_tree
```

which possesses one row less than the current Huffman tree (`huffman_tree`).

- Now write the added probabilities of the first two symbols into the cell `new_huffman_tree{1,1}`. In the second cell of the first row and thus of the newly created symbol, write the corresponding indices of the symbols.

- Since the new tree `new_huffman_tree` consists of one row less due to the two combined symbols, each of the remaining entries from the old tree `huffman_tree` have to be shifted one row up. Realize this with a `for` loop.

- You can display the entries of the `new_huffman_tree` anytime with

```
show_huffman_tree(new_huffman_tree)
```

and check the result of the first iteration:

```
New_huffman_tree
```

```
0.181818 :1 2
```

```
0.181818 :3
```

```
0.272727 :4
```

```
0.363636 :5
```

```
new_probabilities =
```

```
0.1818    0.1818    0.2727    0.3636
```

- Verify both the new and the old tree by calling `show_huffman_tree(huffman_tree)` and `show_new_huffman_tree(new_huffman_tree)`.
- Lastly, the symbols have to be re-sorted by ascending probability. So, create another cell array for the Huffman tree (`huffman_tree`) with the size of the new tree `new_huffman_tree`.
- Store the probabilities [`new_huffman_tree{:,1}`] in a matrix. Sort this matrix with the `sort` command by ascending probabilities. Hint: Get more information about the possible return values of the `sort` function using the MATLAB help!
- Now transfer all entries from the `new_huffman_tree` to the Huffman tree (`huffman_tree`) so that it is sorted.

The loop now restarts from the beginning and does so until all symbols are processed.

Exercise: 2.6: Verify your own Huffman table by loading the Huffman table `huffman_table` with `load('huffman_table.mat')` and displaying it.

2.2.2 Part Two: Huffman Encoder and Decoder

The function `encode_signal_to_huffman_bitstream` is supposed to map each input symbol to a code word using the previously generated Huffman table. These code words are then to be written into a previously defined bitstream.

The function `decode_signal_from_huffman_bitstream` is supposed to recreate the original input signals from the generated bitstream with the help of the Huffman table. The following functions are **already available**:

The function `bitstream = bitstream_init()` initializes the required `bitstream`.

`bitstream = bitstream_append_bits(bitstream, vector)` appends the code words to `bitstream`. It returns the length of the bitstream in bits and a read and a write pointer, as well as the code words in `uint32` format.

`[vector, bitstream] = bitstream_read_bits(bitstream, nr_of_bits)`

reads `nr_of_bits` bits from the bitstream into `vector`. The rest of the bitstream is contained in `bitstream`.

`nr_of_bits = bitstream_get_length(bitstream)` returns the length of `bitstream` in bits.

Practical Part: _____

First of all, create the function

```
[bitstream]=
encode_signal_to_huffman_bitstream(bitstream, huffman_table, signal) ,
```

which stores the Huffman code words in a bitstream.

Its input variables are:

- `bitstream` (bitstream, in which the Huffman code words will be stored; has to be initialized first by the given function `bitstream_init()`, i.e., the init function must be called outside your function!)
- `huffman_table` (contains the Huffman table)
- `signal` (the signal)

Its return variable is:

- `bitstream` (Huffman coded bitstream)

Exercise: 2.7: Generate a Huffman look-up table matrix (`huffman_table_lookup`).

Here, all possible hyper symbols shall be represented uniquely.

Exercise: 2.8: Construct a `for` loop with index `idx`, which successively processes all hyper symbols in `signal`. Within the `for` loop:

- Use the search function `find.m` to look up that Huffman code word from `huffman_table` which corresponds to the currently processed hyper symbol `signal`. Hint: First search in column one `huffman_table_lookup(:,1)` for the symbol from the first column `signal(idx,1)`. With this result `find_col1`, search in the second column `huffman_table_lookup(find_col1,2)` for the corresponding symbol from the second column `signal(idx,2)` and obtain `find_col2`. You will get the position in the Huffman table by using one result and calling it with the other result as a parameter `position_in_huffman_table=find_col1(find_col2)`.
- With the function `bitstream=bitstream_append_bits(bitstream, vector)`, append the corresponding Huffman code word from `huffman_table` to the bitstream `bitstream`.

A description of the functions can be obtained with `help bitstream_append_bits` and `help find`.

In the following, write the function

```
[bitstream signal]=decode_signal_from_huffman_bitstream
(bitstream, huffman_table, nr_of_symbols) ,
```

which allows reading from the Huffman bitstream.

Its input variables are:

- `bitstream` (Huffman coded bitstream)
- `huffman_table` (contains the already created Huffman table)
- `nr_of_symbols` (number of hyper symbols to be decoded; if the whole bitstream is to be decoded, this variable may be omitted: use `nargin` and set it to the default value of 2^{63} .)

Its return variables are:

- `signal` (the signal after Huffman decoding)
- `bitstream` (Huffman coded bitstream with pointer to the next not yet decoded bit)

Exercise: 2.9: First, you have to initialize a few variables:

Compute the number of Huffman table entries using the `size` command and save it in `nr_of_huffman_entries`. Then, use a `for` loop to create a vector `huffman_length` which contains the length of each Huffman code word in bits.

Determine the minimum `min_bits` and maximum `max_bits` number of bits in `huffman_length`. Hint: This is not necessarily the last/first entry of `huffman_length`! Why?

Furthermore, you will need an empty array in which to store the output signal `signal`.

Exercise: 2.10: Now the bitstream is actually read:

In the following, pay attention to the indentation, as it conveys the structure of the program.

Create a `for` loop up until `nr_of_symbols` for decoding each hyper symbol. Within the `for` loop:

- Define the following start variables: `symbol_found=false` (when the search starts, no hyper symbol shall have been found yet) and `current_nr_of_bits=min_bits` (first look for the shortest Huffman code word).
- As long as `current_nr_of_bits <= max_bits` and no hyper symbol has been found yet (`symbol_found==false`), the following is executed (use a `while` loop):
 - Using `[this_huffman new_bitstream]=bitstream_read_bits(bitstream, current_nr_of_bits)`, read `current_nr_of_bits` Huffman coded bits from `bitstream` into `this_huffman`. `new_bitstream` is the not yet decoded part of the bitstream.
 - Check whether `this_huffman` is empty. In this case, the end of the bitstream is reached. A message shall be displayed, telling us that the end of the bitstream has been reached and the `while` loop has to be aborted with `break`.
 - For all Huffman code words of length `huffman_length==current_nr_of_bits`, use a `for` loop `for symbol_row=find(huffman_length==current_nr_of_bits)` ...
`end` ,
 to find out if one of the Huffman code words is equal to the `this_huffman` code word, obtained from the bitstream. Within the `for` loop:

- * If `this_huffman` is equal to a Huffman code word in the table, the corresponding hyper symbol `this_symbol` shall then be appended to the array `signal`.
- * Then assign the new bitstream `new_bitstream` to the old one `bitstream`.
- * Set `symbol_found=true` to indicate that a hyper symbol has been found and then exit the `for` loop with `break`.
- Increment `current_nr_of_bits` by 1.
- Exit the `for` loop with `break` if no hyper symbol has been found in this loop iteration. Furthermore, if less than `nr_of_symbols` hyper symbols have been decoded, an error has occurred when reading the bitstream. In this case, return an appropriate error message.

Test:

The two written functions `encode_signal_to_huffman_bitstream.m` and `decode_signal_from_huffman_bitstream.m` shall be tested for their functionality. In order to do so, call the given function `verify_encode_decode_huffman('name_of_huffman_encoder', 'name_of_huffman_decoder')`.

Exercise: 2.11: Compare the number of used bits before (cf. **Preparation 2.7**) and after Huffman coding. Use the function `bitstream_get_length.m`.

2.3 Summary

You have designed a Huffman encoder as well as a decoder for entropy coding, which reduces the amount of data of the source signal. After successful verification, the created functions can be integrated into the overall project.

2.4 Literature Recommendation

In current video coding, the so-called Context-based Adaptive Binary Arithmetic Coding (CABAC) is used for compressing the residual coefficients into the bitstream:

D. Marpe, H. Schwarz, and T. Wiegand, “Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 620–636, July 2003

"create_huffman_table_from_signal"

takes the input signal...

input_signal =

```

244 244
245 244
243 244
243 244
244 244
243 244
244 244
245 244
243 244
242 242
240 241

```



...extracts the individual hyper symbols...

unique_symbols =

```

240 241
242 242
243 244
244 244
245 244

```



...and creates the probability_matrix...

probability_matrix =

```

0.0909 240.0000 241.0000
0.0909 242.0000 242.0000
0.3636 243.0000 244.0000
0.2727 244.0000 244.0000
0.1818 245.0000 244.0000

```

..."create_huffman_table_from_probability"

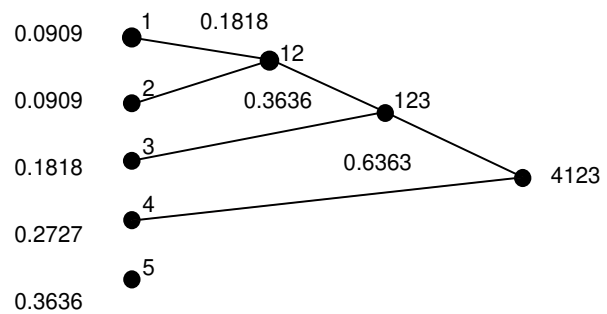
takes the probability_matrix, sorts it...

sorted_probability_matrix =

```

0.0909 240.0000 241.0000
0.0909 242.0000 242.0000
0.1818 245.0000 244.0000
0.2727 244.0000 244.0000
0.3636 243.0000 244.0000

```



...and creates the "huffman_tree" and the

"huffman_table" iteratively...

iteration 0:

huffman_tree	huffman_table		
0.0909091 : 1	1	:	240 241
0.0909091 : 2	0	:	242 242
0.181818 : 3	:	:	245 244
0.272727 : 4	:	:	244 244
0.363636 : 5	:	:	243 244

huffman_table: assign bit "0" and "1" to the first/second node of the huffman_tree
huffman_tree: merging nodes and sorting the tree anew

iteration 1:

huffman_tree	huffman_table		
0.181818 : 1 2	11	:	240 241
0.181818 : 3	10	:	242 242
0.272727 : 4	0	:	245 244
0.363636 : 5	:	:	244 244
	:	:	243 244

huffman_table: assign bit "0" and "1" to the first/second node of the huffman_tree
huffman_tree: merging nodes and sorting the tree anew

iteration 2:

huffman_tree	huffman_table		
0.272727 : 4	011	:	240 241
0.363636 : 1 2 3	010	:	242 242
0.363636 : 5	00	:	245 244
	1	:	244 244
	:	:	243 244

huffman_table: assign bit "0" and "1" to the first/second node of the huffman_tree
huffman_tree: merging nodes and sorting the tree anew

iteration 3:

huffman_tree	huffman_table		
0.363636 : 5	0011	:	240 241
0.636364 : 4 1 2 3	0010	:	242 242
	000	:	245 244
	01	:	244 244
	1	:	243 244

huffman_table: assign bit "0" and "1" to the first/second node of the huffman_tree
huffman_tree: finished when only 1 node left after merging

Figure 2.4: Example of creating a Huffman table .

Lab Experiment 3

Quantization

A video codec reduces the enormous amount of data of a video stream by making use of two principles:

- *Redundancy reduction* and
- *Irrelevancy reduction* (this session).

You have already learned about the principle of redundancy reduction in Lab Experiment 2 in the form of reversible entropy coding.

Irrelevancy reduction means the removal of irrelevant, i. e., unnecessary, information from a signal. As a result, the signal is altered or rather distorted and it is impossible to exactly reconstruct the original signal. Irrelevancy reduction thus is a *lossy* procedure.

Which parts of a signal are considered irrelevant and hence negligible depends on the signal receiver and the purpose of the signal, that is, the application. In this lab course, the transmission of video data is our application. We assume the receiver to be a human being who simply wants to watch the video.

The *quantization* discussed in this experiment is the most important tool for irrelevancy reduction. It exploits the properties of human perception by removing irrelevant — in our video-watching application: barely or not at all perceptible — information from the video data. Quantization is lossy.

Before we start, we need some basics on evaluating compression results for the further lab experiments. They will be introduced in the following section. In the subsequent section, you will then implement a uniform quantizer.

This lab experiment is to be completed in one session.

3.1 Rate-Distortion Curves

The higher the amount of information removed from a signal via lossy data reduction, the more the signal quality deteriorates. At a constant compression rate, different compression methods lead to different signal qualities. To be able to compare the efficiency of different lossy compression methods, we now introduce the *rate-distortion* curves as a suitable means of representation.

The signals to be considered shall be grayscale images. In this lab experiment, the undistorted or original image is denoted by $U(x, y)$, and the distorted one by $V(x, y)$. In order to explain the *rate-distortion* curve, two terms have to be defined first:

1. *Rate R*. In image processing, the rate denotes the required storage space per pixel. For an uncompressed grayscale image with 256 possible luminance values, the rate is $R = 8$ bits/pixel, since 8 bits can represent $2^8 = 256$ different luminance levels.
2. *Distortion D*. D is a means for measuring the deviation of the distorted image from the original image.

Common *distortion* measures D in image processing are:

- MSE (*mean squared error*): for an image, the MSE is given as

$$\text{MSE} = \frac{1}{X \cdot Y} \sum_{x=1}^X \sum_{y=1}^Y (V(x, y) - U(x, y))^2. \quad (3.1)$$

- SNR (*signal-to-noise ratio*): the ratio of the power of the original signal to the power of the error signal:

$$\text{SNR} = 10 \log_{10} \left(\frac{\sum_{x=1}^X \sum_{y=1}^Y U^2(x, y)}{\sum_{x=1}^X \sum_{y=1}^Y (V(x, y) - U(x, y))^2} \right) \text{dB}. \quad (3.2)$$

- PSNR (*peak signal-to-noise ratio*): analogous to SNR, but the maximum possible power of the signal, $\hat{S}$, is used for normalization:

$$\begin{aligned} \text{PSNR} &= 10 \log_{10} \left(\frac{\sum_{x=1}^X \sum_{y=1}^Y \hat{S}^2}{\sum_{x=1}^X \sum_{y=1}^Y (V(x, y) - U(x, y))^2} \right) \text{ dB} \\ &= 10 \log_{10} \left(\frac{\hat{S}^2}{\text{MSE}} \right) \text{ dB} . \end{aligned} \quad (3.3)$$

- MAD (*mean absolute deviation*): an absolute error measure:

$$\text{MAD} = \frac{1}{X \cdot Y} \sum_{x=1}^X \sum_{y=1}^Y |V(x, y) - U(x, y)| . \quad (3.4)$$

Rate-distortion curves ($D(R)$ curves) show the signal distortion obtained with a certain bitrate. As these curves are bijective functions, it is as well possible to read off the bitrate required for keeping the distortion below a given level. In image processing, PSNR is the most widely accepted measure for D(R) curves, which are also called PSNR curves in this case. In contrast to MSE and MAD, higher PSNR values mean lower distortion, so the rate-distortion curves should rather be called rate-quality curves. However, since this is uncommon in literature, we call them D(R) curves, too.

Practical Part:

Exercise: 3.1: Write a MATLAB function

`output_mse=mse_of_frame(original_image,distorted_image)` which returns the MSE as a result. (Hint: `help mean` or `help sum`)

Verify your function by calling `verify_mse_of_frame`.

Exercise: 3.2: Write a MATLAB function

`output_psnr=psnr_of_frame(original_image,distorted_image)` which returns the PSNR value as a result. The maximum possible power of the signal shall be assumed to be $\hat{S} = 1$. Test the function by calling `verify_psnr_of_frame`.

Exercise: 3.3: Write a MATLAB program which displays the $D(R)$ curve of the MATLAB JPEG image compressor for the first frame of the “flowergarden” sequence. JPEG is a lossy compression method which allows you to use different compression rates to achieve different image qualities. In order to load a frame of a sequence, the function `imread` is available. Now, in order to determine each point of the $D(R)$ curve, carry out the following steps:

1. Use the `imwrite` command to store the image in JPEG format with a certain JPEG quality.
2. Determine the file size of the stored image with `get_image_filesize` and use it to calculate the rate R in bits/pixel.
3. Load the stored image and determine the distortion D of the loaded image. As a distortion measure, use your function `psnr_of_frame` from the previous exercise.

Write a loop which performs these steps for all values of the JPEG parameter ‘Quality’ and display the result as a $D(R)$ curve (PSNR in dB as a function of R). Also show the original image and the decompressed image via `imshow`. (Hint: Use `pause` for observing the rise of the $D(R)$ curve.)

3.2 Quantization

Quantization is the process of mapping intervals of the input signal range u to quantization levels i . In the inverse process, the quantization levels i are then mapped to representative values v_i (Figure 3.1). The intervals of the input signal range are defined by decision thresholds $u_i, u_{i+1}, \dots$, and the *step size* of the quantizer is defined by the difference of consecutive representative values (e. g., $v_{i+1} - v_i$).

Here, we only want to deal with quantizers whose input-output characteristics – as depicted in Figure 3.1 – contain a representative value 0 (so-called *midtread* quantizers). The input values shall be quantized independently (*scalar* quantization).

The deviation of the reconstructed signal value from the input value is called quantization error

$$e = v - u . \quad (3.5)$$

The quantization error can be interpreted as a noise signal which is added to the wanted signal u .

3.2.1 Uniform Quantization

Uniform quantization describes the simplest kind of quantization: The step size of the quantizer is always the same and thus not dependent on the current signal value to be

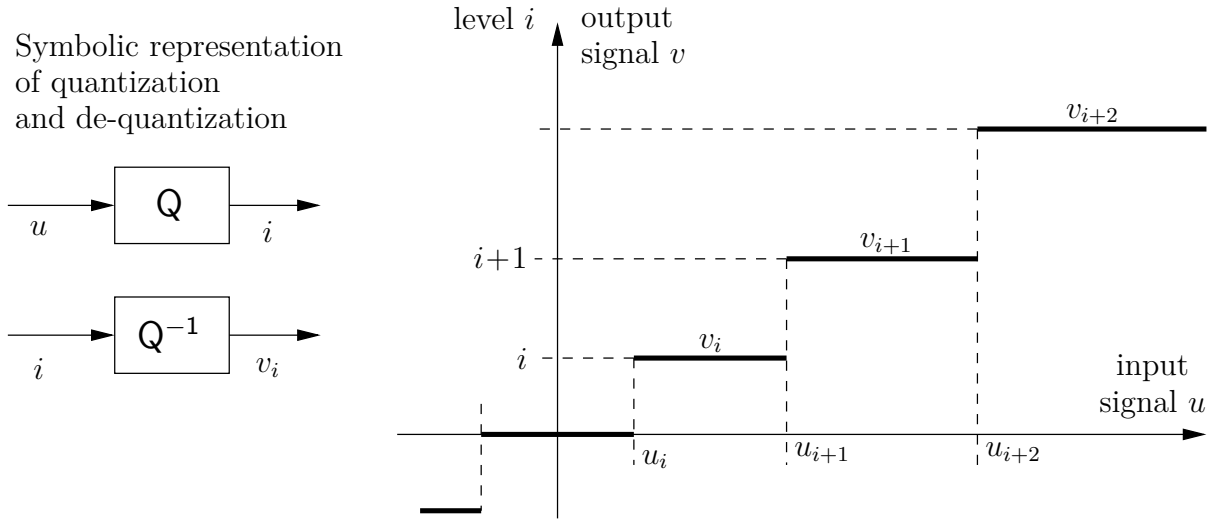


Figure 3.1: Quantization and de-quantization

quantized. This kind of quantization is also called *linear* quantization, although the term might be a bit confusing since quantization always describes a non-linear system.

Figure 3.2 shows the input-output characteristic of a uniform midtread quantizer as well as the quantization error e with respect to the input u . The step size Q of the quantizer is equal to the width of the quantization intervals.

The input signal values u are mapped to the quantizer levels i according to the instruction

$$i = NINT(u/Q) , \quad \text{where } NINT \text{ means rounding to the nearest integer.} \quad (3.6)$$

For this quantizer, signal reconstruction is accomplished by:

$$v = Q \cdot i \quad (3.7)$$

For estimating the signal power of the quantization error, we assume that this error is approximately uniformly distributed. Then, the probability density function of the error within an interval of width Q is

$$p_e(q) = 1/Q \quad (q = e) \quad (3.8)$$

and the variance of the quantization error in the interval around the origin is

$$\sigma_e^2 = \int_{-Q/2}^{Q/2} p_e(q) \cdot q^2 dq = Q^2/12 \quad . \quad (3.9)$$

The distribution of the quantization error is the same for the other intervals and the above-stated result thus applies to the whole range of quantization.

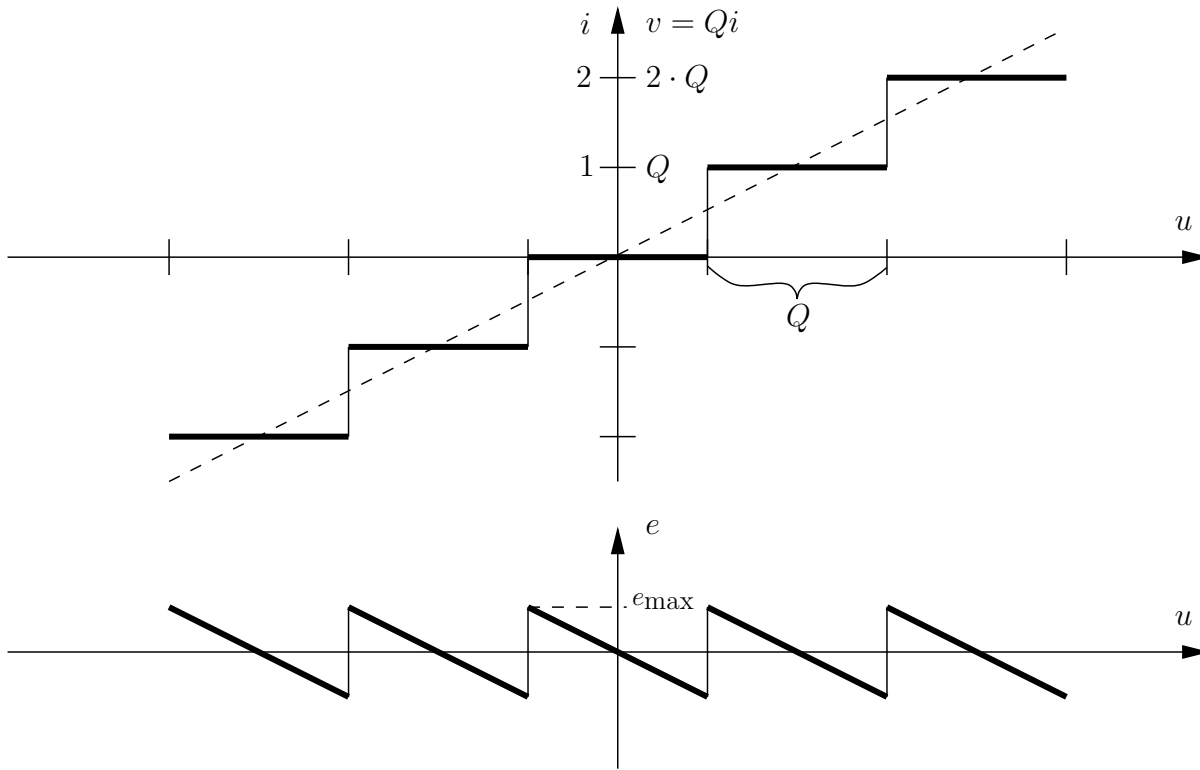


Figure 3.2: Uniform quantization (top) and quantization error (bottom).

Our quantizer has to quantize input values from range $[-1; 1]$ with N bits each. With N bits, we can assign each luminance value to one of 2^N intervals. That is why we choose the quantization interval width to be

$$Q = \frac{2}{2^N} = \frac{1}{2^{N-1}}, \quad \text{where } N \text{ is the number of bits/pixel (bit depth).} \quad (3.10)$$

Our images comprise non-negative luminance values, but in video coders, difference images or transformed images (Lab Experiment 5) with negative signal values are quantized as well. We thus “give away” 1 bit/pixel for the range $[0; 1]$. Furthermore, we ignore that, in fact, one additional quantization step would be required due to the midtread characteristic. To work around this problem, one might use the quantization interval $Q = \frac{1}{2^{N-1}-1}$ for instance.

Preparation: _____

Preparation: 3.1: What is the maximum error $e_{\max}$ that can occur with uniform quantization and step size Q ?

Preparation: 3.2: By which factor does the energy of the quantization error change

if you use one additional bit per signal value for representation? Express the result in dB.

Practical Part:

Now, both a uniform midtread quantizer for image signals as well as its corresponding de-quantizer are to be implemented.

Exercise: 3.4: Write a MATLAB function

`quant_levels=simple_quantizer(input_image,quantizer_stepsize)`
 which returns the quantization levels i of an input image with respect to the quantization step size Q . The value $i = 0$ shall here be equal to the representative value $v_0 = 0$.

The result shall be returned in the same form (matrix) as the input image, i. e., each luminance value of the image shall be replaced by the corresponding quantization level. (*Hint: help round*)

Exercise: 3.5: Write a MATLAB function with which to reconstruct the original values v from the quantization levels i according to (3.7)

`output_image=simple_dequantizer(quant_levels,quantizer_stepsize)` .

Test your uniform (de-)quantizer with different levels and step sizes.

Exercise: 3.6: Write a MATLAB script which quantizes and de-quantizes the first “flowergarden” frame with the bit depths $N = 1, 2, \dots, 8$ bits/pixel.

View both the non-quantized and the de-quantized images. Determine both the rate R and the distortion D (PSNR) for each case and display the results as a $D(R)$ curve (D in dB).

Determine the slope of the $D(R)$ curve and compare it to the result of preparation task **Preparation 3.2**. Also compare the curve to your result from **Exercise 3.3**.

3.2.2 Uniform Quantization Using Quantization Matrices

If certain signals are particularly important to the signal receiver, it is desirable to transmit those signals with less distortion than the less important signals. For this purpose, one can quantize the signals using different resolutions, e. g., uniformly with varying step sizes for the different signals.

This is accomplished by using a *quantization matrix*, the elements of which are the step sizes of a uniform quantizer for the different signal values.

Example: When transmitting a video, it is assumed that the most important parts of an image are located in its center. The central part of the image shall thus be quantized with a higher resolution than the boundary regions. So, we introduce a quantization matrix QM which features a smaller step size in the image center. This is depicted in Figure 3.3 for an image of 3×3 pixels.

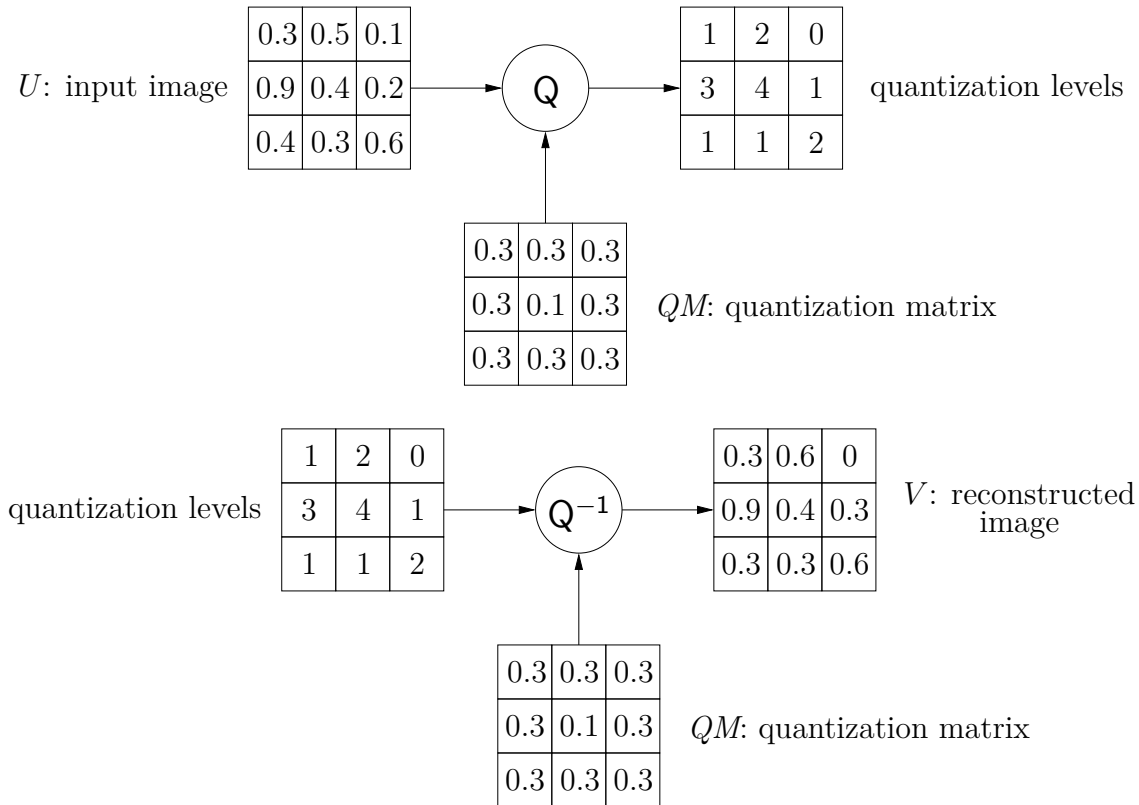


Figure 3.3: Quantization/de-quantization with quantization matrix

Attention: *Uniform* quantization with quantization matrices must not be confused with *non-uniform* quantization (step size dependent on current signal value) or with *vector quantization* (several signal values are described by one representative in a multi-dimensional space).

Practical Part: _____

Exercise: 3.7: Write a function

```
output_levels=scalar_quantizer(input_image, quantizer_matrix)
```

with which to compute the quantization levels of an image, given a quantization matrix. Verify your function by calling `verify_scalar_quantizer`.

Exercise: 3.8: Write a function

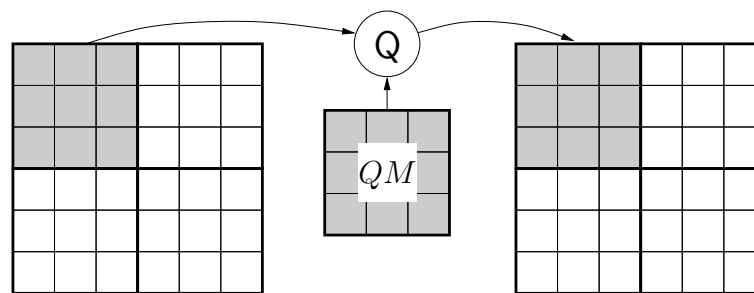
```
output_image=scalar_dequantizer(input_levels, quantizer_matrix)
```

with which to reconstruct the original image from the quantization levels and the quantization matrix. Test your function by calling `verify_scalar_dequantizer`.

Exercise: 3.9: Quantize the first “flowergarden” frame with the same quantization interval Q for all pixels. Choose N according to (3.10) in such a way that you achieve an acceptable image quality.

Now quantize the frame with different quantization intervals Q_0 for the flower bed and Q_1 for the rest of the image. To realize this, use the function `design_quantizer_matrix`. For one of the two segments, choose a coarser quantization (lower resolution) than before. Which segment is more suited for coarser quantization? Without significant quality loss, how much storage space can be saved for the quantized image by using a quantization matrix?

Exercise: 3.10: Our video coder is not supposed to quantize the luminance values of the image pixels but rather the values of a transformed image representation (cf. Chapter 5). Therefore, we need *block-based* quantization functions which divide the whole image into blocks and quantize each of those single blocks with the same quantization matrix QM . Example with blocks of 3×3 pixels:



Write the functions which implement block-based quantization and de-quantization for square blocks of dimension $blocksize \times blocksize$:

```
out_levels=blockbased_quantizer_to_levels(input_image,
quantizer_matrix)
and
out_image=blockbased_dequantizer_from_levels(input_levels,
```

```
quantizer_matrix)
```

Use the function `blockwise_processing` that you implemented in Lab Experiment 1, passing your functions `scalar_quantizer` and `scalar_dequantizer` as a so-called *function handle* (MATLAB pointer to a function). A *function handle* for the function `scalar_quantizer`, for instance, can be created as follows:

```
fun = @(input_image) scalar_quantizer(input_image,quantizer_matrix);
```

Alternatively, you can also pass the function name in single quotes!

Test your functions by calling `verify_blk_quant`.

3.3 Optional Part

3.3.1 Additions to Section 3.1: Distortion Measures

Practical Part:

The distortion measures introduced in Section 3.1 differ in both computation time and significance. A “good” distortion measure yields an objective result which matches the subjective qualitative impression of an image as accurately as possible. The significance of different distortion measures shall be examined here.

Exercise: 3.11: In **Exercises 3.1** and **3.2**, you have implemented functions for computing the distortion measures MSE and PSNR. Implement corresponding functions `mad_of_frame` and `snr_of_frame` for computing the MAD and SNR.

Exercise: 3.12: Load the first “flowergarden” frame and create two distorted images `noisy1` and `noisy2`: In the first case, add uniformly distributed noise $N1=0.1*(\text{rand}(\text{size}(\text{picture}))-0.5)$. In the second case, add a noise with a modified probability distribution $N2=\text{shape_pdf}(N1)$.

Display both distorted images side by side and assess the image quality subjectively. Now compute both the MAD and the MSE for both images (using your functions `mad_of_frame` and `mse_of_frame`) and compare the results with your subjective assessment. How can you explain the result? Also take the histograms of the noise signals (`hist(N1(:),100)` and `hist(N2(:),100)`) into consideration.

Exercise: 3.13: In information technology, the signal-to-noise ratio SNR is the most frequently employed measure for evaluating signal quality. In image processing, the

PSNR is often used instead.

Load the first “flowergarden” frame and create both a brightened up version (`picture1=picture.^0.4`) and a darker version (`picture2=picture.^1.4`). Then create two distorted images `noisy1` and `noisy2` by adding uniformly distributed white noise $N=0.2*(\text{rand}(\text{size}(\text{picture}))-0.5)$ to the bright and the dark images.

Display both distorted images side by side and assess the image quality subjectively. Now compute both the SNR and PSNR and compare the results with your subjective assessment. How can you explain the result?

3.3.2 Additions to Section 3.2: Quantization

Up until now, we have only considered uniform quantization in this lab course. For the video coder (and also for other applications in signal processing), *non-uniform* quantizers can also be of interest, though. When employing non-uniform quantization, the step size of the quantizer is dependent on the signal value.

Practical Part: _____

Exercise: 3.14: In a video coder, the luminance values of the input images are transformed suitably so that most of the image information can be described with few signal values. Less important image information is then represented by signal values close to zero.

Consequently, the compression rate of a video coder can be considerably improved by quantizing those small signal values to zero without having to suffer significant quality loss. This can be achieved with very little effort via a so-called *dead-zone quantizer*, whose input-output characteristic features a *dead zone* around the origin, hence giving the quantizer its name.

Implement a dead-zone quantizer

```
deadzone_quantizer(input_image,q_matrix)
deadzone_dequantizer(input_levels,q_matrix)
```

adhering to the instructions ($TRUNC$ =rounding towards zero):

$$i = TRUNC(u/Q), \quad v = (i + 0.5 \cdot \text{sign}(i)) \cdot Q. \quad (3.11)$$

Plot the input-output characteristic of your dead-zone quantizer by quantizing and reconstructing suitable input signals.

Implement a block-based version of your dead-zone quantizer:

```
blockbased_deadzone_quantizer_to_levels
blockbased_deadzone_dequantizer_from_levels .
```

3.4 Summary

In this lab experiment, you have written functions for quantizing and de-quantizing arbitrary two-dimensional signals using a uniform quantizer. In order to be able to quantize certain signal parts differently, you have learned about the so-called quantization matrix. In Lab Experiment 5, you will use the functions to quantize a transformed image representation. That is why you have adjusted your functions so that they can also be applied in a block-based manner to blocks of different sizes.

3.5 Literature Recommendation

In the previous experiments, the quality of the decoded video is mainly defined by the chosen quantization parameter. However, it is usually required for practical scenarios to run the encoder resulting in a fixed bitrate that fits to the available channel capacity. For that purpose, so-called rate control algorithms have to be implemented as it is proposed in the following reference for HEVC:

B. Li, H. Li, L. Li, and J. Zhang, “ λ domain rate control algorithm for high efficiency video coding,” *IEEE Transactions on Image Processing*, vol. 23, no. 9, pp. 3841–3854, Sep. 2014

Lab Experiment 4

Motion Estimation and Compensation

Motion estimation is an important component of most video processing systems. Video sequences consist of a series of individual frames. Each of these frames represents an image of a scene at a certain discrete point in time. Changes in a scene can be caused by, for example, movements of the camera, movements of objects, changes in illumination, and changes of the form and structure of objects. In multimedia applications, motion estimation is applied for, among other things, motion compensated interpolation for rendering views in-between two frames (e.g., 100Hz TV), analyzing video sequences (e.g., segmentation, computer vision), and motion compensated prediction for video compression (*inter-frame prediction*). The latter application constitutes the main part of this experiment.

This lab experiment comprises one session. The first half will deal with block-based motion estimation (Section 4.2, **Exercises 4.1–4.3**), the second one with motion compensation (Section 4.3, **Exercise 4.4**). Furthermore, if there is enough time left, you might also want to implement functions from the optional part. Here, there is a lot of potential for improving your coder! In order to get an impression of the overall implementation effort and the improvement possibilities, please read the whole description of the lab experiment thoroughly, including the optional part, and finish all the preparatory exercises (**Preparations 4.1–4.7**) before this session!

4.1 The Principle of Motion Compensated Prediction

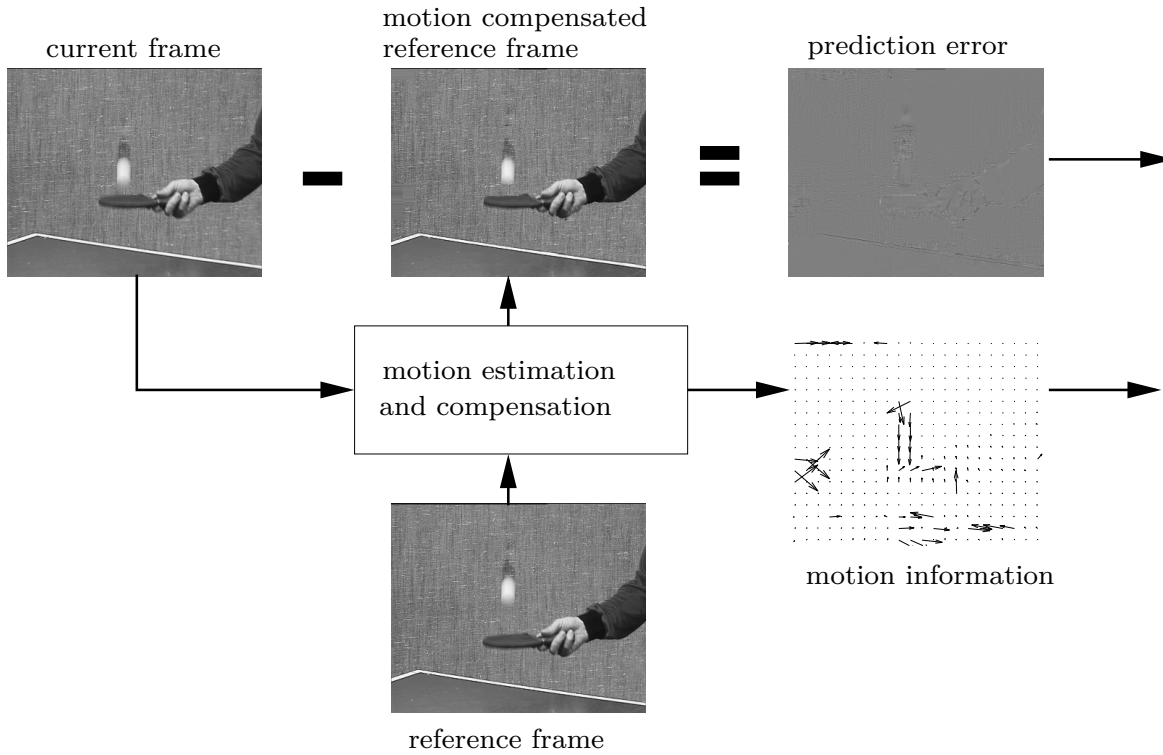


Figure 4.1: Principle of motion compensated prediction (cf. [1]).

Motion compensated prediction as depicted schematically in Figure 4.1 is applied in all common video coders (MPEG-1, MPEG-2, MPEG-4, H.263, H.264, H.265). It exploits the temporal correlation of consecutive video frames in order to reduce redundancy. To this end, an algorithm for motion estimation tries to estimate the motion of the image content in the current frame with respect to a reference frame (in most cases the temporally preceding frame). Thus the motion in the reference frame can be compensated and a motion compensated frame can then be used as a predictor for the current frame. To preserve the whole image content, it is sufficient to transmit the prediction error along with the motion information. In order to achieve as high a compression as possible, the criterion for motion estimation should be the minimization of the prediction error. Motion compensated prediction is also called *inter-frame coding*.

4.2 Block-based motion estimation

Most methods for motion estimation and compensation are block-based. Due to its low complexity, the so-called *block matching* is very often employed. This method is based on the assumption that the objects of a scene are usually only subjected to translational motion. This assumption is only justified as long as no rotation, deformation, camera zoom, or luminance alteration occur in a scene. Nevertheless, this method can accomplish quite good compression results, particularly since the amount of motion information to be transmitted is relatively small (one motion vector per block).

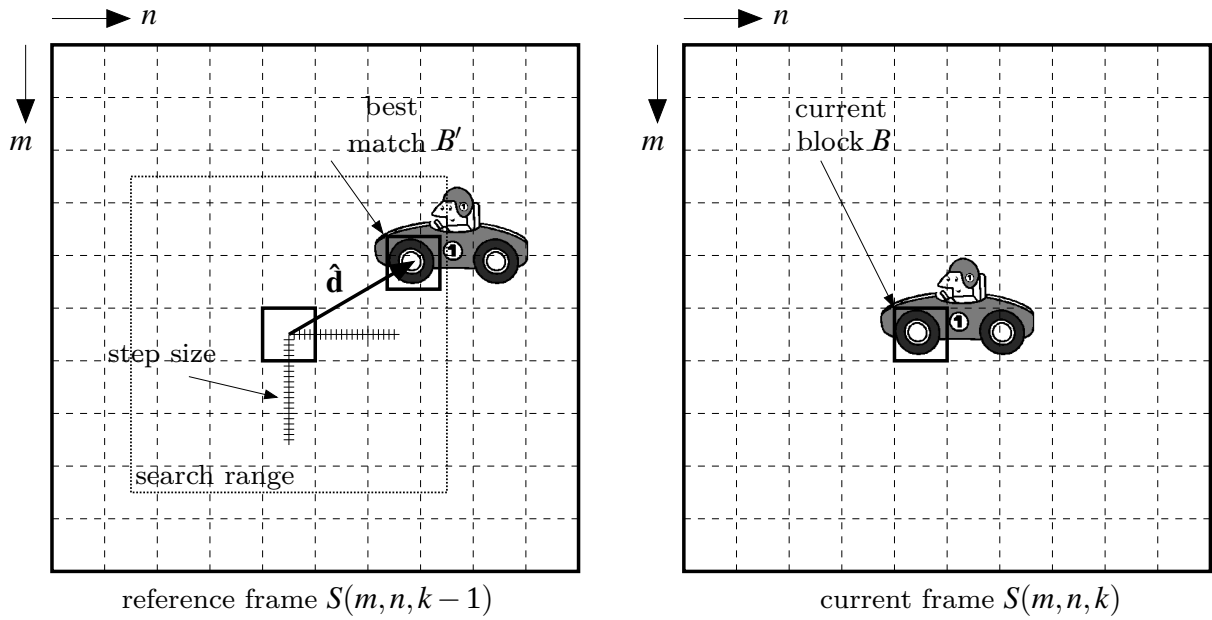


Figure 4.2: Block-based motion estimation (cf. [5]).

Figure 4.2 depicts the principle of block-based motion estimation. The pixel values of a sequence are denoted by $S(m, n, k)$ with the spatial coordinates m and n and the time index k . Here, the previous frame at time $k - 1$ has been chosen as the reference frame. Block-based motion estimation first divides the current frame into blocks of size $M_b \times N_b$. Then, for each block B , the reference frame is searched for a block B' with the greatest possible overlap. The vector $\hat{\mathbf{d}} = (\hat{d}_m, \hat{d}_n)^T$, denoting the translation of the best matching reference block B' relative to the current block B , is called the motion vector [5]. The motion vector always points from the current block to the reference block!

There exist a number of strategies for finding the best matching block in the reference frame. A criterion frequently employed is the minimization of the sum of absolute differ-

ences (SAD) [6]:

$$\text{SAD}(d_m, d_n) = \sum_{(m,n) \in B} |S(m, n, k) - S(m + d_m, n + d_n, k - 1)| \quad (4.1)$$

So, out of a large number of motion vector candidates $\mathbf{d}$ for block B , the block with the smallest SAD value is chosen, i. e.,

$$\hat{\mathbf{d}} = (\hat{d}_m, \hat{d}_n)^T = \arg \min_{(d_m, d_n)} \text{SAD}(d_m, d_n). \quad (4.2)$$

The number of motion vector candidates is dependent on the accuracy of the motion search, the search strategy, and the search range:

The accuracy of the motion estimation is defined by the step size with which the motion vectors are searched. In the simplest case, the candidate block in the reference frame is shifted pixel-wise. This is called *full-pixel* accuracy. In practice, a motion vector search with *subpixel* accuracy down to a step size of 1/4 pixel has proven its value. However, subpixels between the actual pixels have to be interpolated for this purpose (cf. Section 4.4).

The search strategy determines which of the motion vector candidates to evaluate for motion estimation. In case of the *full search*, this comprises all possible translational positions given by the grid of the used step size. *Subpixel* accurate motion estimation in particular results in a very time-consuming and computationally intensive search. That is why fast search strategies which evaluate only a subset of all possible motion vector candidates are often used for this purpose. This is done step by step: Starting with a selection of only few motion vector candidates with a high step size, this step size is decreased in each step, thus successively refining the motion estimation (e. g., *log-search*, *3-step search*; cf. Section 4.4).

The search range usually describes a symmetrical area surrounding the current block with R_x pixels to the left and right and R_y pixels above and below. This limits the motion search to a reasonable range. Since the dynamic range of vertical and horizontal motion is often the same, $R_x = R_y = R$ is usually chosen.

Preparation: _____

Given an image with a resolution of 288×352 pixels (CIF), let the block size be 16×16 pixels.

Preparation: 4.1: How many motion vector candidates have to be evaluated for one block when doing a *full search* with full-pixel accuracy, given a search range of $R = 32$ pixels?

Preparation: 4.2: Consequently, how many motion vector candidates have to be evaluated for the *whole* image?

Preparation: 4.3: How do the results of the previous two exercises change if

- a) the search range is doubled?
- b) the block size is halved?

4.3 Motion Compensation

Motion estimation is a key element in every video coder. Block-based motion estimation is used here for temporal block-based prediction. Each block in the current frame is predicted from the corresponding motion compensated block in the reference frame and merely the residual error signal is encoded. In this context, motion compensation describes a shift of block B' in the reference frame to the position of block B in the current frame, making use of the corresponding motion vector $\hat{\mathbf{d}}$.

A descriptive example for block-based motion estimation and compensation is depicted in Figure 4.3. In subfigures (a) and (b), two temporally consecutive frames of a test sequence can be seen. The sequence was recorded from a driving car so that the image content is basically shifted from right to left. In the current frame (Figure 4.3(b)), block $B = \{S(m, n, k) \mid 113 \leq m < 129 \wedge 193 \leq n < 209\}$ is highlighted.

Motion estimation now searches for the best match in the reference frame. In order to achieve this, the reference frame is first padded (extended) by the size of the search range by simply repeating the boundary pixels of the reference frame (cf. Figure 4.3(c)). Now the full search can also be performed on blocks at the image boundary without leaving the (padded) image. With that, usable prediction blocks are also obtained for image content which has newly appeared in the current frame.

In Figure 4.3(c), block $B' = \{S(m, n, k) \mid 109 \leq m < 125 \wedge 217 \leq n < 233\}$ is highlighted. During motion estimation, this block was found to be the most suitable prediction block for B . Motion compensation now copies it to the position at which B can be found in the current frame.

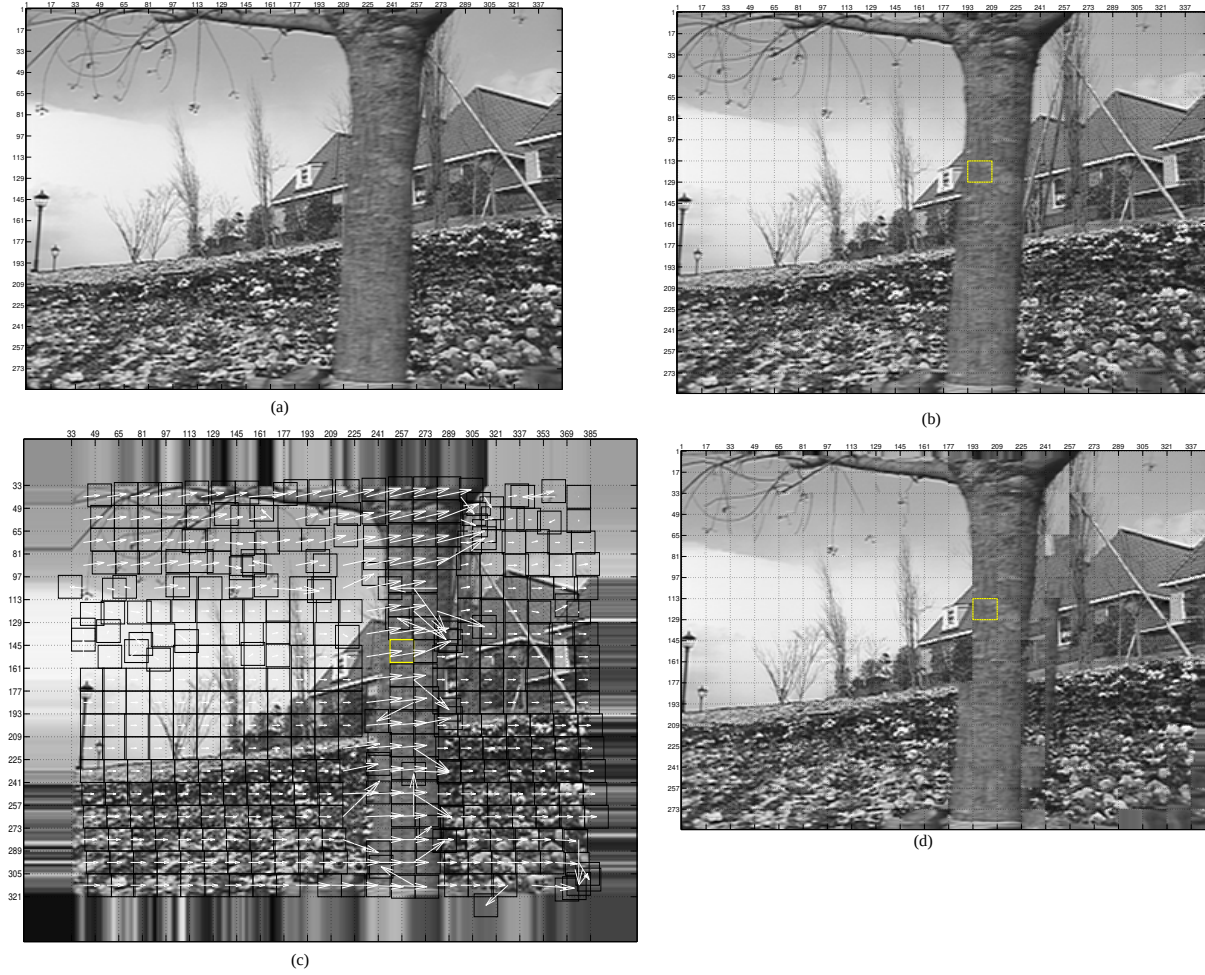


Figure 4.3: Example for block-based motion estimation and compensation.

(a) Reference frame at time $k - 1$. (b) Current frame at time k , divided into blocks (block size 16×16). (c) Reference frame padded by replicating the boundary pixels. Black squares denote the blocks found by motion estimation and arrows denote the corresponding motion vectors. (d) Motion compensated reference frame as prediction signal for the current frame.

The same happens for all other blocks. Block-based motion estimation does not make sure that the actual motion in the image is exactly determined but rather that the signal difference between the frames with respect to the error criterion (here: SAD value) is kept as small as possible. For this reason, Figure 4.3(c) also contains many block overlaps and uncorrelated motion vectors.

For block $B = \{S(m, n, k) \mid 113 \leq m < 129 \wedge 193 \leq n < 209\}$ and block $B' = \{S(m, n, k) \mid 109 \leq m < 125 \wedge 217 \leq n < 233\}$ in the example, the following motion vector can be computed: $\hat{d} = B' - B = (-4, 24)$.

Figure 4.3(d) shows the result of the motion compensation. It can be observed clearly that the motion of the tree in the foreground from right to left, for example, was estimated very well. Directly to the right of the tree and at the right image border, there are areas which have been newly revealed. For those areas, the motion can naturally not be estimated very well as they were not present in the reference frame.

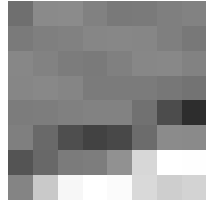
In such cases, one possibility for improvement is offered by using the methods for adaptive *intra prediction* or by choosing a reference frame which succeeds the current frame temporally (*bi-predictive motion estimation*). These methods are shortly addressed in Section 4.4.

Generally, though, it can be observed that the motion compensated reference frame in Figure 4.3(d) already realizes a fairly good prediction for the current frame in Figure 4.3(b) and that the energy of the residual error image will thus be very small. This, in turn, leads to a high compression rate.

Preparation:

The following two images of a video sequence are given along with their grayscale matrices (pixel range $0 \dots 255$):

$$Y_{\text{reference_frame}} = \begin{bmatrix} 99 & 106 & 136 & 142 & 131 & 120 & 124 & 126 \\ 107 & 118 & 126 & 129 & 131 & 132 & 135 & 128 \\ 117 & 136 & 132 & 122 & 119 & 125 & 132 & 134 \\ 122 & 133 & 136 & 132 & 125 & 125 & 121 & 117 \\ 133 & 125 & 124 & 125 & 131 & 130 & 118 & 89 \\ 138 & 134 & 113 & 88 & 69 & 63 & 91 & 130 \\ 69 & 76 & 93 & 115 & 125 & 132 & 188 & 248 \\ 97 & 113 & 169 & 228 & 255 & 255 & 230 & 208 \end{bmatrix}$$


$$Y_{\text{current_frame}} = \begin{bmatrix} 111 & 138 & 139 & 129 & 121 & 122 & 127 & 130 \\ 122 & 127 & 129 & 134 & 135 & 134 & 127 & 120 \\ 135 & 130 & 124 & 122 & 128 & 135 & 136 & 130 \\ 134 & 136 & 133 & 123 & 123 & 123 & 119 & 115 \\ 124 & 125 & 128 & 130 & 130 & 113 & 76 & 45 \\ 127 & 105 & 78 & 66 & 72 & 108 & 139 & 138 \\ 84 & 104 & 123 & 125 & 145 & 215 & 255 & 254 \\ 132 & 201 & 247 & 255 & 251 & 217 & 206 & 211 \end{bmatrix}$$


One image is to be predicted from the other by using block-based motion estimation. The motion search shall be performed on blocks of size 2×2 .

Preparation: 4.4: Compute the SAD value for the block in the 2nd block column (i.e., starting at the 3rd pixel column) and 3rd block row for the motion vector candidates $(0, 0)$, $(0, 1)$, and $(0, 2)$. Which motion vector is to be preferred?

An incomplete pseudo code of a function which performs a full motion search for **one** block of the current frame is given. The following parameters shall be passed to the function:

- Current frame in matrix form
- Reference frame in matrix form
- Starting indices (upper left corner) of the current block: x_0, y_0
- Block size
- Size of the search range

The incomplete pseudo code for the motion vector search looks like this:

```
• current block = current frame(rows from ...to ...,
                                columns from ...to ...)

• for all  $|d_m, d_n| \leq \dots$ 

    candidate block = reference frame(rows from ...to ...,
                                      columns from ...to ...)

    ...

    motion vector = ...
```

Preparation: 4.5: Complete this pseudo code reasonably!

Practical Part: _____

In the practical part, functions for motion estimation and compensation shall now be implemented. They are later used in the video coder.

Exercise: 4.1: Write a MATLAB function for computing the SAD of two blocks. The function call shall have the syntax

```
sad_value = calculate_sad(input_block1,input_block2);
```

The input parameters `input_block1` and `input_block2` denote the two image blocks. In the function, they are represented by two matrices of the same size. Implement the function according to (4.1) for arbitrary block sizes.

Test:

To test your implementation, use the image blocks from **Preparation 4.4** and compare your results to those of the preparation. The matrices $Y_{\text{reference_frame}}$ and $Y_{\text{current_frame}}$ can be created by using the provided function

```
[y_reference,y_current] = create_mv_test_matrix;
```

Exercise: 4.2: Implement a MATLAB function which determines the optimal motion vector for a given image block. The function call shall have the syntax

```
motion_vector = get_motion_vector_for_block(padded_input_image,
padded_previous_image,blockstart_x,blockstart_y,blocksize,searchrange)
```

with the parameters:

- **motion_vector:**
optimal motion vector represented by 1×2 vector
 $\text{motion_vector} = [\hat{d}_m, \hat{d}_n]$, i.e., the y-component first!
- **padded_input_image, padded_previous_image:**
padded current frame and padded reference frame in matrix form, respectively
- **blockstart_x, blockstart_y:**
pixel coordinates (left, top) of the current block for which the optimal motion vector is to be determined
- **blocksize:**
block size in pixels
- **searchrange:**
search range in pixels

For your implementation, use your description of the algorithm from **Preparation 4.5**. The function shall realize a full search with integer-pixel accuracy based on the criterion (4.2). For the calculation of the SAD, use your function `calculate_sad` from the previous exercise.

Test:

Determine the optimal motion vector for the block in the second column and third row of the frame $Y_{\text{current_frame}}$ (block size 2×2 , search range 2). Use $Y_{\text{reference_frame}}$ as reference frame.

Hint: Before calling the function, the padded frames must be created. For this, you may use the MATLAB function

```
b = padarray(a,padsize,method,direction) .
```

Display both frames and verify the result. The optimal motion vector for the above-mentioned block is: $\hat{\mathbf{d}} = (xx, xx)$.

Exercise: 4.3: Now use the function from the previous exercise to compute the motion vectors for a whole frame. The function call shall have the syntax

```
motion_vectors = blockbased_motion_search(input_image,  
previous_image,blocksize,searchrange)
```

with the parameters:

- **motion_vectors:**
All motion vectors as a $N \times 2$ matrix. Here, N denotes the number of motion vectors. This matrix can be created by concatenating the motion vectors within the function, for instance. The individual motion vectors are to be taken from the image row by row from left to right/top to bottom (raster scan order).
- **input_image, previous_image:**
current frame and reference frame in matrix form
- **blocksize:**
block size in pixels
- **searchrange:**
search range in pixels

The function could be realized as follows: First create the padded frames. For this, you can use the MATLAB function `padarray` again. Then, in a loop, proceed from block to block in the padded current frame and determine the optimal motion vectors using your function `get_motion_vector_for_block`.

Test:

For verifying your function, synthetically generated test images are used at first. These images can be created with the provided MATLAB script

```
[img1,img2]=create_synthetic_images(height,width,dx,dy) .
```

The function generates two images with noise-like background and a noise-like block in the foreground. The block in image `img2` (the reference image) is shifted by `dx, dy` with respect to the block in `img1`. The shift can be freely chosen within the image size. Now create two test images (size 288×352) and perform the motion vector search (block size 16×16 , search range 8) with your implementation. For displaying the motion vectors, use the provided script

```
plot_motion_vectors(height,width,blocksize,searchrange,motion_vectors)
```

Do your specifications in the script `create_synthetic_images` match your displayed motion vectors?

Now, the motion vectors of a frame from the “flowergarden” sequence are to be computed. For this purpose, load frames 1 and 2 from this sequence. Determine the motion vectors (block size 16×16 , search range 8) of the second frame using the first one as reference frame. Plot the current frame, the reference frame, the difference image of both, and the motion vectors. Do the motion vectors depict the motion in the image in each case?

Load the file `flower_motion.mat` and compare your results to the motion vector field `motion_vectors_flower`!

Exercise: 4.4: Since now the motion vectors are known, they can be used for compensating the motion of the image. Therefore, implement the function

```
output_image = blockbased_motion_compensation(input_image,  
blocksize,searchrange,motion_vectors)
```

with the parameters:

- `output_image`:
motion compensated frame
- `input_image`:
reference frame
- `blocksize`:
block size in pixels
- `searchrange`:
search range in pixels

- `motion_vectors`:
motion vectors of the current frame.

In this function, an approximation – or prediction – of the current frame is to be created from the padded reference frame and the motion vectors. Use the motion vectors to copy the corresponding blocks in the padded reference frame into the approximated current frame (the motion compensated frame).

Test:

For verifying your function, use the “flowergarden” sequence. Compute a motion compensated frame for frame 2 (block size 16×16 , search range 8). Frame 1 of the sequence shall serve as the reference frame. Display the two frames, the motion compensated frame, and the difference image between the original frame and the motion compensated frame. Calling the function

```
[motion_compensated_frame2] = verify_motion_compensation;
```

provides you with the motion compensated frame. Compare your results to this standard solution. Calculate the PSNR of the motion compensated frame with respect to the original frame 2. Use the function `psnr_of_frame` from Lab Experiment 3.

Also determine the PSNR for different search ranges. Using a loop, compute both the motion compensated frame 2 and the PSNR for search ranges $sr = [1 \ 2 \ 4 \ 8 \ 16 \ 32]$. Plot the PSNR values as a function of the search range. How does the search range affect motion compensation?

4.4 Optional Part

4.4.1 Sub-Optimal Motion Estimation

The *full search* on all pixels of the search range results in a high computational effort for motion estimation. In order to accelerate motion estimation, there exist a number of methods for fast motion search. As a general approach, the amount of search candidates is limited reasonably. This leads to a sub-optimal motion estimation, but it can reduce the computing time enormously. If the algorithm works in a way such that it only discards

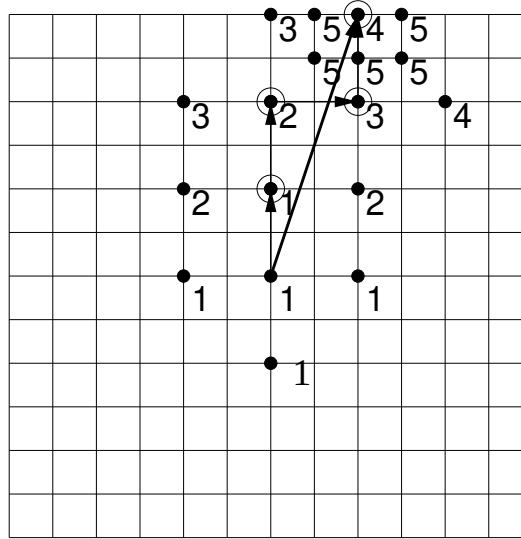


Figure 4.4: Logarithmic search. The evaluated motion vector candidates are depicted by dots, the best match in each step by a circled dot. Here: $(-2,0)$, $(-4,0)$, $(-4,2)$, $(-6,2)$, and $(-6,2)$ (cf. [5]). The origin $(0,0)$ is the center of the grid.

those motion vector candidates which are highly likely not to result in a low SAD value, then the loss in efficiency is very low. Three common methods for fast motion search are *logarithmic search*, *3-step search*, and *octagonal search*.

Logarithmic search:

The procedure of the *logarithmic search* is depicted in Figure 4.4 for a search range of ± 6 pixels. The search starts at position $(0,0)$, the case for which no motion occurred. In each step, five search points are evaluated in a diamond-shaped grid. In the next step, this diamond-shaped evaluation is repeated, but the center is moved to the best match of the previous step. The step size (i. e., the radius of the diamond) is halved if the best match lies in the center of the diamond or on the boundary of the search range. The algorithm terminates as soon as the step size is equal to one pixel. In the last step, the search is performed on the eight neighboring points and the center. The result of the *logarithmic search* is identical to that of the *full search* if no motion vector candidate coincides with a local minimum of the cost function (SAD value). The step size with which the algorithm starts has a certain influence on how probable it is to find a local minimum at a great or small distance from the origin. If it is chosen to be small (e. g., 2, as in Figure 4.4), then the probability of finding a local minimum in the vicinity of the origin is greater than it would be for a larger initial step size.

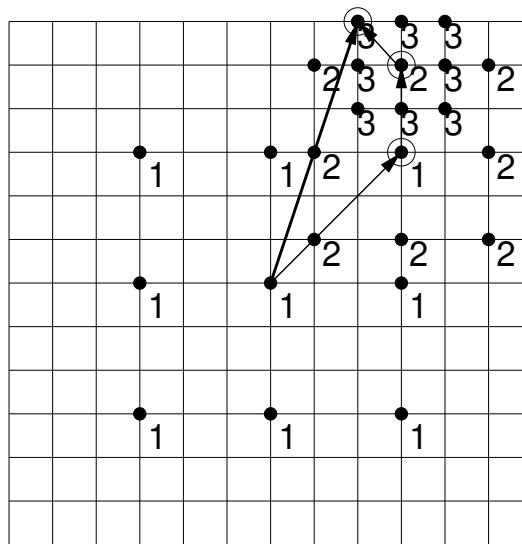


Figure 4.5: 3-step search. The evaluated motion vector candidates are denoted by dots, the best match in each step by a circled dot. Here: $(-3,3), (-5,3), (-6,2)$ (cf. [5]).

Preparation: _____

Preparation: 4.6: Is it possible to predict the number of motion vector candidates to be evaluated when using the *logarithmic search*?

3-step search:

The *3-step search* starts with a step size which is equal to half the maximum search range. In the first step, nine search points are evaluated, as can be seen in Figure 4.5. In the next step, the search center is moved to the best match of the previous step and the step size is halved. The search terminates if the step size is equal to one pixel. In the example in Figure 4.5, exactly three steps are needed for a search range of ± 6 pixels. Hence this algorithm has been named *3-step search* by its inventors. However, in the case of a larger search range, more than three steps are needed and thus one should correctly speak of a *multi-step search*.

Preparation: _____

Preparation: 4.7: How many motion vector candidates have to be evaluated when performing a *multi-step search* with full-pixel accuracy for one block if the search range is $R = 32$ pixels and the initial step size is equal to half the search range? Compare your result to exercise **Preparation 4.1!**

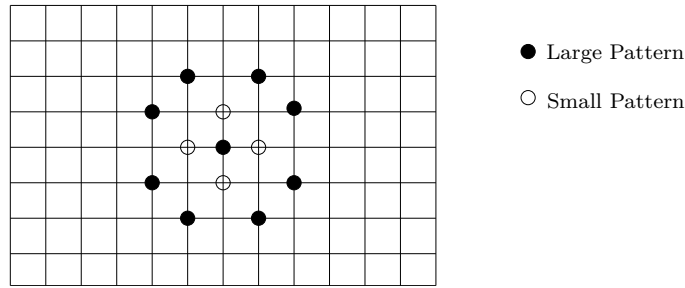


Figure 4.6: Search pattern of the octagonal search.

Octagonal search:

The octagonal search starts with a large octagonal pattern with nine search points as shown in Figure 4.6. If the center point possesses the lowest SAD value, the pattern is reduced to the four surrounding points. If this does not apply, the center of the octagon is moved to the point with the lowest computed SAD value. In the following, only those values which have not yet been calculated and which are no neighbors to previous search points have to be considered. The search continues until the smallest SAD value is found at the center of the octagon. Then the octagon is scaled down and only the four surrounding points are evaluated and compared to the previous SAD value. The point with the smallest value represents the final point of the octagonal search. If a chosen point is on or outside the boundary of the search range, only the four surrounding points are evaluated. In Figure 4.7, the search algorithm is exemplified.

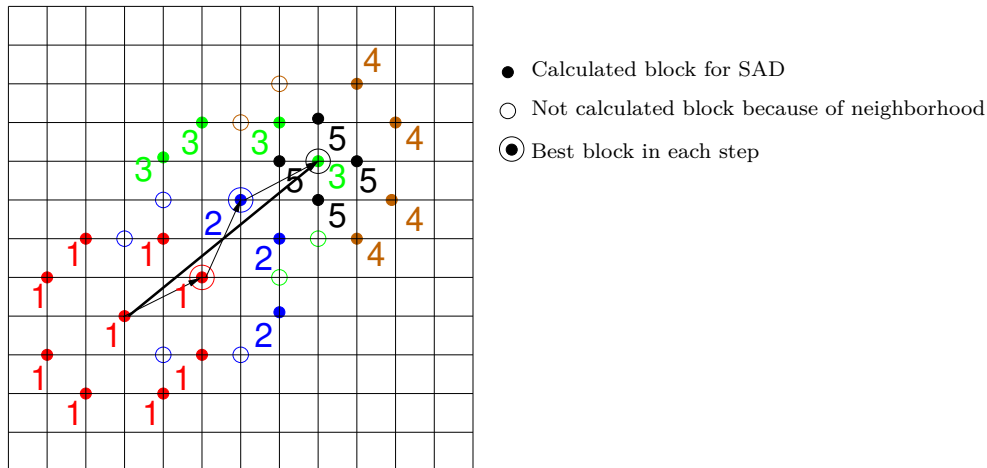


Figure 4.7: Octagonal search demonstrated by an example. In this case, the motion vector $(-4, +5)$ is found.

Practical Part:

Exercise: 4.5: Copy and paste the function `get_motion_vector_for_block` into the new file `get_motion_vector_for_block_logsearch.m`. Then make the necessary alterations in this function to obtain the motion vector for the block by performing a *logarithmic search*. Use an initial step size that is equal to half the maximum search range.

Also copy and paste the function `blockbased_motion_search` into the new file `blockbased_motion_search_logsearch.m`. In this function, you only have to substitute the function call `get_motion_vector_for_block` with the new name `get_motion_vector_for_block_logsearch`.

Exercise: 4.6: Copy and paste the function `get_motion_vector_for_block` into the new file `get_motion_vector_for_block_3stepsearch.m`. Then make the necessary alterations in this function to obtain the motion vector for the block by performing a *multi-step search*. Use an initial step size that is equal to half the maximum search range.

Also copy and paste the function `blockbased_motion_search` into the new file `blockbased_motion_search_threestep.m`. In this function, you only have to substitute the function call `get_motion_vector_for_block` with the new name `get_motion_vector_for_block_threestep`.

Exercise: 4.7: Copy and paste the function `get_motion_vector_for_block` into the new file `get_motion_vector_for_block_octagonsearch.m`. Then make the necessary alterations in this functions to obtain the motion vector for the block by performing a *octagonal search*.

Also copy and paste the function `blockbased_motion_search` into the new file `blockbased_motion_search_octagon.m`. In this function, you only have to substitute the function call `get_motion_vector_for_block` with the new name `get_motion_vector_for_block_octagonsearch`.

4.4.2 Sub-Pixel Accurate Motion Estimation

In order to achieve better motion estimation, common video coders usually use subpixel accurate motion estimation. This means that the step size is reduced to half or even quarter pixel distances when searching for the most suitable motion vector. The motion vectors can thus also take non-integer values such as (2.5, 1.75). The advantage of such

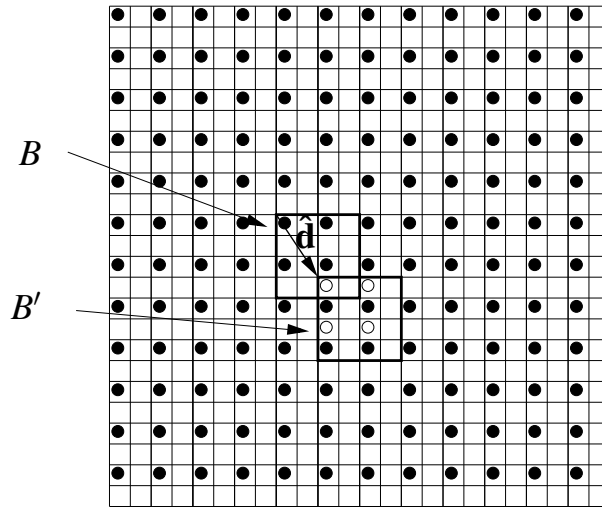


Figure 4.8: Subpixel accurate motion estimation. Computation of the SAD value is done on interpolated pixel positions of the reference frame.

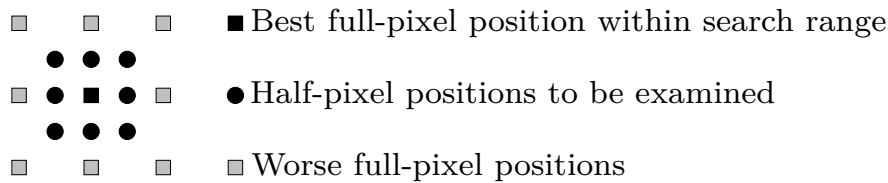


Figure 4.9: Re-estimation at half-pixel positions.

an increased resolution of the motion information is a residual error signal with a low energy, which, in turn, results in a higher coding efficiency.

Motion estimation with subpixel accuracy makes it necessary to interpolate pixel values at the subpixel positions. This is depicted in Figure 4.8. Therefore, new pixel values between the original pixels are created for the whole reference frame by applying bilinear interpolation (MPEG-2, MPEG-4, H.263) or by filtering with a 6-tap-filter (H.264/AVC). To increase the computation time no more than necessary for performing motion estimation, only the region around the best full-pixel position is generally taken into consideration when searching for the most suitable subpixel position. This is shown in Figure 4.9.

Practical Part: _____

Exercise: 4.8: The functions for motion estimation and compensation must be modified. For this purpose, copy and paste the functions `blockbased_motion_search`, `blockbased_motion_compensation`, and `get_motion_vector_for_block` into the new files `subpixel_motion_search.m`, `subpixel_motion_compensation.m`, and

`get_subpixel_motion_vector_for_block.m`, respectively!

Exercise: 4.9: Write the function

`interpolated_image=interpolate_image(previous_image)`, which interpolates the reference frame at the half-pixel positions! Hint: MATLAB already provides two-dimensional bilinear interpolation via the function `interp2`. In order to also be able to interpolate boundary pixels, it is sufficient to duplicate the last row and the last column of the already interpolated image.

Exercise: 4.10: Modify the function `get_subpixel_motion_vector_for_block!`

This function differs from `get_motion_vector_for_block` in that it is passed the *interpolated* reference frame as well as the *full-pixel accurate* motion vector from the full-pixel accurate motion search for this block. The function call thus is

```
subpixel_motion_vector=get_subpixel_motion_vector_for_block
(padded_input_image, padded_interpolated_previous_image,
blockstart_x, blockstart_y, blocksize, integer_motion_vector) .
```

The function is now supposed to perform a motion search at the half-pixel positions surrounding the most suitable full-pixel position – given by `integer_motion_vector` – in the range $(y, x) = (\pm 0.5, \pm 0.5)$.

Exercise: 4.11: The function `subpixel_motion_search` shall be modified such that it performs a full-pixel accurate motion estimation on each block as a first step. The resulting motion vector is then to be passed as an input parameter for the half-pixel accurate estimation.

Exercise: 4.12: Finally, complete the function for motion compensation by adding the interpolation of the input image (i.e., the reference frame) as well as the prediction at the interpolated subpixel positions!

4.4.3 Macroblock-Based Intra/Inter Decision

If motion estimation is not able to find a suitable reference block in the reference frame for the current block of the current frame, it is often better to code the current block as an *intra block*. This means that motion compensated prediction is not performed on this block. The block is instead transmitted directly and unchanged. When deciding whether a block is to be *intra* coded or not, the costs ($\sim$ the number of needed bits) for both

inter/intra coding have to be balanced against each other. An estimation for the *inter* cost is given by the minimum SAD value from motion estimation. The cost for *intra* coding a block can be calculated via the absolute differences between the pixels of the block and its mean value, for instance (H.263). If the *intra* cost is less than the minimum SAD value for the block minus a scaling offset, then the block is *intra* coded. A suitable scaling offset has been determined empirically in the H.263 standard and is here chosen to be 500 (pixel range $0 \dots 255$). It makes sure that *inter* coding is preferred over *intra* coding since it is usually more efficient.

Practical Part: ---

Exercise: 4.13: Write the function `iblock_cost=get_iblock_cost(input_block)`, which returns the *intra* cost for `input_block`!

Exercise: 4.14: Enhance your function `get_motion_vector_for_block` so that the minimal SAD is compared to the *intra* cost for each block. If the *intra* cost is less than the minimum SAD $- 1.95$, an invalid motion vector (greater than the search range) is to be returned for this block in order to signal the *intra* mode. Save the new function into a file named `get_intradecision_motion_vector_for_block.m`! Change the function call within the function `blockbased_motion_search` accordingly and save it as `intradecision_motion_search.m`!

Exercise: 4.15: Enhance the function for motion compensation so that a block with an invalid motion vector is filled with zeroes. Save the new file as `intradecision_motion_compensation.m`!

4.4.4 Variable Block Sizes

In many cases of *inter frame* coding, it is more beneficial to adjust the block size for motion estimation and compensation adaptively to the image content. Along the boundary of two objects which are exposed to different motions (e.g., at the edge of the tree in Figure 4.3), it may occur that part of the block is better predictable from a different direction. Therefore, the standards H.263 and MPEG-4, for example, offer the possibility to divide a 16×16 block for motion estimation (called macroblock) into 4 blocks of size 8×8 and to determine a motion vector for each of these sub-blocks. Using this approach, much better predictors for areas of inconsistent motion can be created.

If one were to divide every macroblock into four sub-blocks (this is equal to halving the original block size for motion estimation), the prediction error would definitely be reduced, but at the same time, the necessary bitrate for the motion information would be quadrupled. Because of that, splitting the macroblocks into sub-blocks is done adaptively for each block by comparing the minimal SAD value of the macroblock to the sum of the minimal SAD values of the four sub-blocks plus a scaling offset. With the offset, the rate increase caused by additional motion information is taken into account when deciding for or against macroblock division. In the H.263 standard, the value 200 (pixel range $0 \dots 255$) has been determined empirically as the offset. That is, a division of the macroblock into sub-blocks only happens if

$$\text{SAD}_{\text{MB}} > \text{SAD}_{\text{SB1}} + \text{SAD}_{\text{SB2}} + \text{SAD}_{\text{SB3}} + \text{SAD}_{\text{SB4}} + 200 \quad (4.3)$$

where SAD_{MB} and SAD_{SBx} denote the SAD values of the macroblock and the four sub-blocks, respectively.

When employing adaptive macroblock division, it must be taken into account that the block division has to be transmitted as additional side information and that the number of motion vectors per frame is no longer constant.

Practical Part:

Exercise: 4.16: Modify the function `get_motion_vector_for_block` such that it returns not only the motion vector but also the minimum SAD and save it as `get_motion_vector_min_sad_for_block.m`!

Exercise: 4.17: The function `blockbased_motion_search` has to be modified such that it performs motion estimation on every macroblock as well as on the four sub-blocks of a macroblock. It shall also store the corresponding motion vectors and the minimum SAD value.

In a further loop, it must be decided according to (4.3) which block division is the most beneficial one for each macroblock. Keep in mind the pixel range $[0 \dots 1]$ of our coder!

The function is to return the motion vector(s) for each macroblock in an array of the same form as in **Exercise 4.3**. The number of rows can now be different for each frame. In order to still be able to assign each motion vector to a block in the frame without ambiguity, the function must additionally return an array `macroblock_partition` which contains the decision on the block division for each macroblock: 0 = no division, 1 = division into sub-blocks. Save the new function so that its function call is

```
[motion_vectors macroblock_partition]=variable_block_motion_search  
(input_image, previous_image, blocksize, searchrange) .
```

Exercise: 4.18: Extend your function for motion compensation by the input parameter containing the macroblock division so that the new function call is

```
output_image=variable_block_motion_compensation  
(input_image, blocksize, searchrange, motion_vectors,  
macroblock_partition) .
```

Modify the function body such that the prediction for variable block sizes with the passed parameters is made possible. For this purpose, it is necessary to determine the corresponding motion vector(s) for each macroblock from the motion vector array using the side information `macroblock_partition`.

In Lab Experiment 6, you will build a complete video coder and decoder by combining your previously implemented functional blocks. If you also want to implement the optional enhancement introduced in this section, you will have to keep in mind that the side information `macroblock_partition` must be transmitted from the encoder to the decoder. Since this side information is a binary array, Huffman coding is not necessary. Thus the array can be stored without prior coding.

The decoder has to be modified such that it decodes the side information first. The number of motion vectors is now variable, but can be determined from this array.

4.5 Summary

In this lab experiment, you have written all the functions for block-based motion estimation and compensation. The current frame can thus be predicted from the previous frame, resulting in a difference image with low signal energy that can then be encoded very efficiently.

Lab Experiment 5

Transform Coding and Intra Coding Chain

In this experiment, you will create the functions which are necessary – in addition to the already implemented quantization and entropy coding – for realizing the intra coder. First, the image data is transformed into the frequency domain. The data in the frequency domain is then re-sorted by the so-called zigzag scan. With the help of run-level coding, it is then transformed into a representation which is convenient for entropy coding. In the end, you can combine the functions you have created into a fully-functional intra image coder.

In part one of this lab experiment, you will work on Chapters 5.1 and 5.2. Part two comprises Chapters 5.3 and 5.4 plus an optional part.

5.1 Discrete Cosine Transform (DCT) and Inverse Discrete Cosine Transform (IDCT)

Real photographic image data has a low-pass characteristic, i.e., the low frequencies contain most of the image information. To exploit that fact, the image data is transformed into the frequency domain via the Discrete Cosine Transform (DCT). In this section, the forward and backward transforms are introduced for which you will implement the corresponding MATLAB functions later on.

5.1.1 One-Dimensional DCT/IDCT

Given the input sequence $f(n)$ of length N , where $n = 0, \dots, N-1$, the one-dimensional DCT (1D-DCT for short) of $f(n)$ is denoted as $F(k)$, where $k = 0, \dots, N-1$:

$$F(k) = \gamma_k \sum_{n=0}^{N-1} f(n) \cos\left(\frac{\pi(2n+1)k}{2N}\right) \quad \forall k = 0, \dots, N-1 \quad (5.1)$$

The values $F(k)$ are also called DCT coefficients. Using the one-dimensional IDCT (1D-IDCT for short), the input sequence $f(n)$ is perfectly reconstructible from the DCT coefficients $F(k)$ for $n = 0, \dots, N-1$:

$$f(n) = \sum_{k=0}^{N-1} \gamma_k F(k) \cos\left(\frac{\pi(2n+1)k}{2N}\right) \quad \forall n = 0, \dots, N-1 \quad (5.2)$$

In both cases, the coefficient γ_k is defined as follows:

$$\gamma_k = \begin{cases} \sqrt{\frac{1}{N}}, & k = 0 \\ \sqrt{\frac{2}{N}}, & 0 < k < N \end{cases} \quad (5.3)$$

Figure 5.1 shows the basis functions of the one-dimensional DCT $\left(\cos\left(\frac{\pi(2n+1)k}{2N}\right)\right)$ for a block length of $N = 8$.

5.1.2 Two-Dimensional DCT/IDCT

The two-dimensional DCT (2D-DCT for short) is a separable linear transform. Due to its separability, the two-dimensional transform can be decomposed into two consecutive one-dimensional transforms of the single dimensions.

The 2D-DCT of an $M \times N$ input matrix $A(m, n)$ is defined as:

$$B(k, l) = \gamma_k \gamma_l \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} A(m, n) \cos\left(\frac{\pi(2m+1)k}{2M}\right) \cos\left(\frac{\pi(2n+1)l}{2N}\right) \quad (5.4)$$

The inverse two-dimensional DCT (2D-IDCT for short) is defined as:

$$A(m, n) = \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} \gamma_k \gamma_l B(k, l) \cos\left(\frac{\pi(2m+1)k}{2M}\right) \cos\left(\frac{\pi(2n+1)l}{2N}\right) \quad (5.5)$$

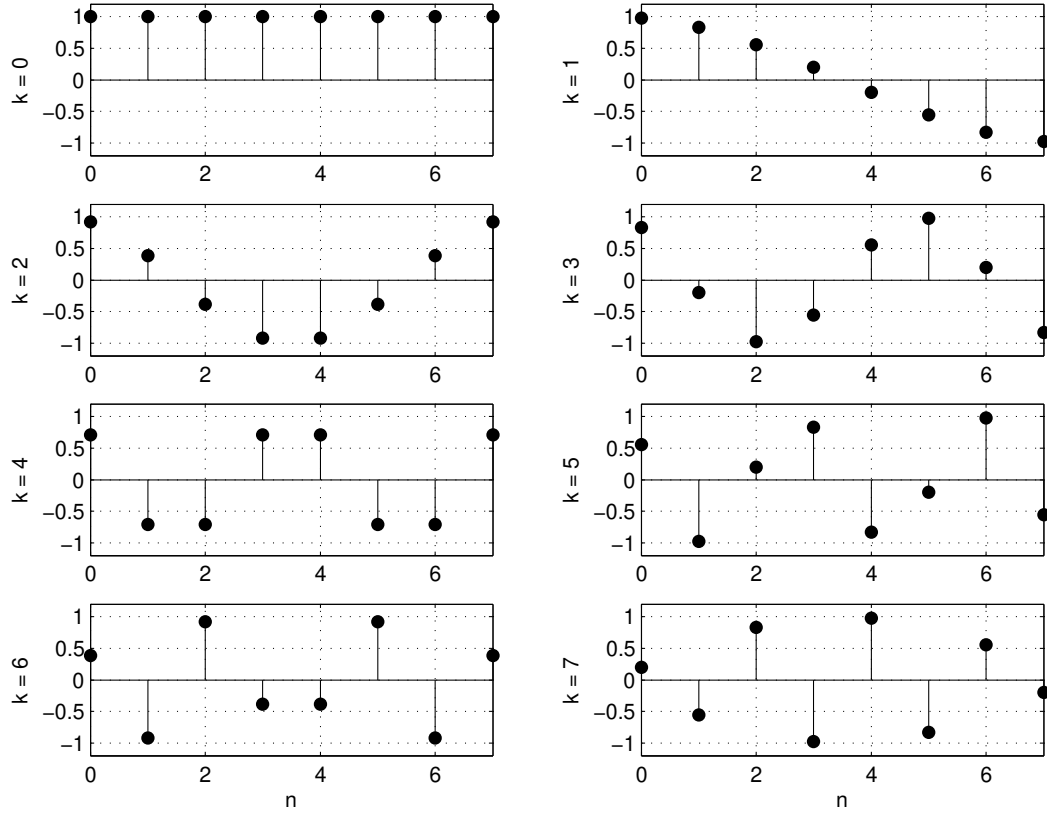


Figure 5.1: Basis functions of the 1D-DCT: $\cos\left(\frac{\pi(2n+1)k}{2N}\right)$, for $N = 8$.

In both cases, the following applies:

$$k, m = 0, \dots, M - 1 \quad (5.6)$$

$$l, n = 0, \dots, N - 1 \quad (5.7)$$

$$\gamma_k = \begin{cases} \sqrt{\frac{1}{M}}, & k = 0 \\ \sqrt{\frac{2}{M}}, & 0 < k < M \end{cases} \quad (5.8)$$

$$\gamma_l = \begin{cases} \sqrt{\frac{1}{N}}, & l = 0 \\ \sqrt{\frac{2}{N}}, & 0 < l < N \end{cases} \quad (5.9)$$

The basis functions of the two-dimensional DCT are depicted in Figure 5.2 for $M = N = 8$ pixels (square block).

5.1.3 Block-Based Two-Dimensional DCT

Given an input image (in general: an input matrix) of height H and width W , let the block size be M (vertically) and N (horizontally). Figure 5.3 illustrates the method described

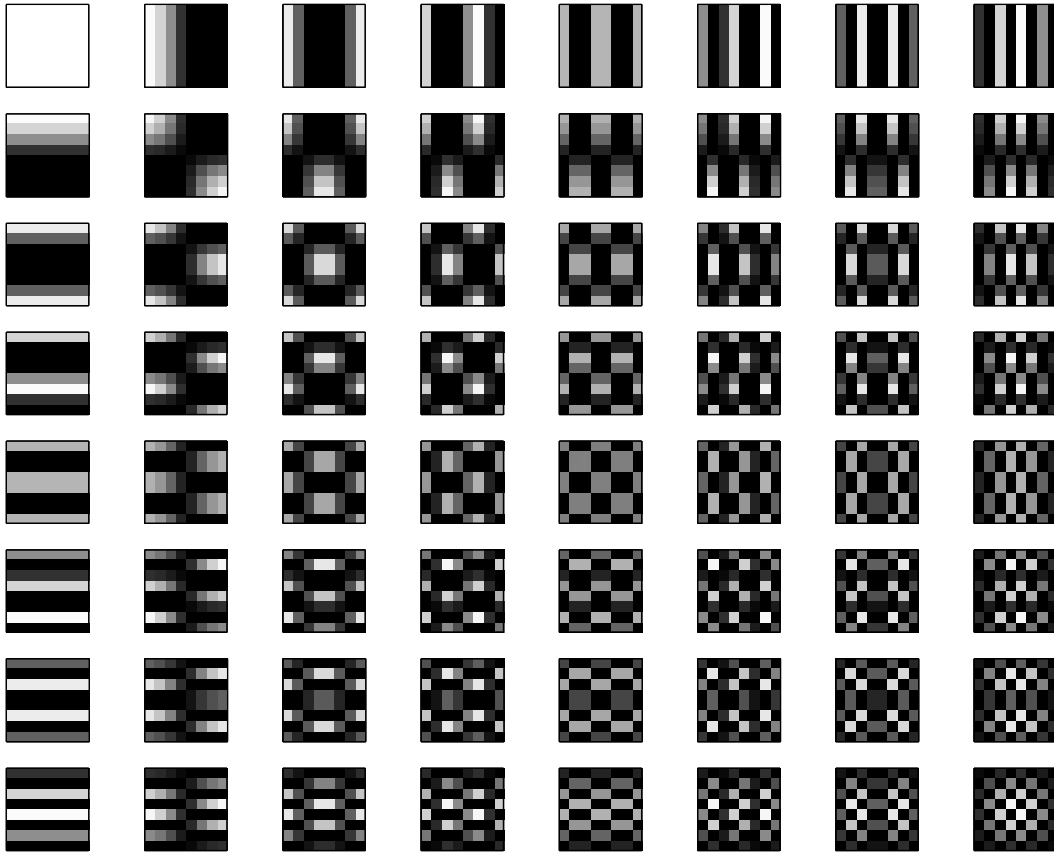


Figure 5.2: Basis functions of the two-dimensional DCT ($M = N = 8$).

in the following for the block-based two-dimensional DCT for $H = 8$, $W = 7$, $M = 3$, and $N = 2$.

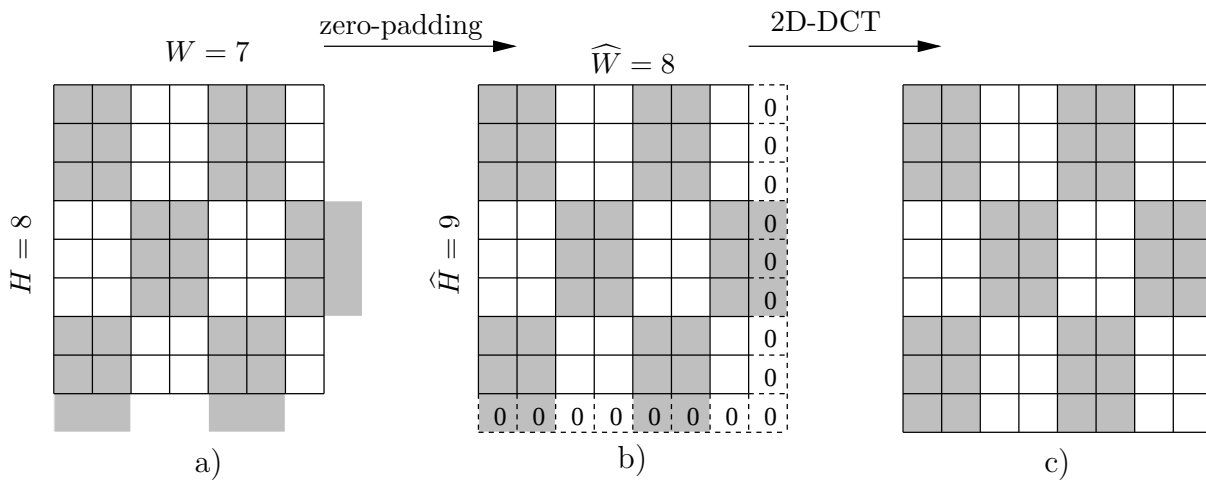


Figure 5.3: Block-based 2D-DCT. Image size: 8×7 , block size: 3×2 . a) Original image, b) zero-padded image, c) DCT-coefficients.

In the first step, the input image is expanded by zeros (zero-padding) such that each image dimension is a whole-number multiple of the corresponding block dimension (cf. Figure 5.3: a)→b)).

$$\widehat{H} = m \cdot M, \quad m \in \mathbb{N} \quad (5.10)$$

$$\widehat{W} = n \cdot N, \quad n \in \mathbb{N} \quad (5.11)$$

In the second step, the image is divided into individual blocks of size $[M \times N]$. The 2D-DCT is applied to each of these blocks (cf. Figure 5.3: b)→c)). The output image (cf. Figure 5.3 c)) consists of blocks with DCT coefficients. The position of these blocks within the output image is the same as the position of the corresponding image blocks in the input image.

Preparation: _____

In Figure 5.4, you can see the first frame of the flowergarden sequence. The two-dimensional DCT has been applied to each of the four marked blocks A, B, C, and D. Unfortunately, the figure lacks the assignments of the DCT coefficients to their corresponding image blocks.

Preparation: 5.1: Assign each of the image blocks A, B, C, and D to one DCT coefficient block 1,2,3, or 4 and give reasons for your choice!

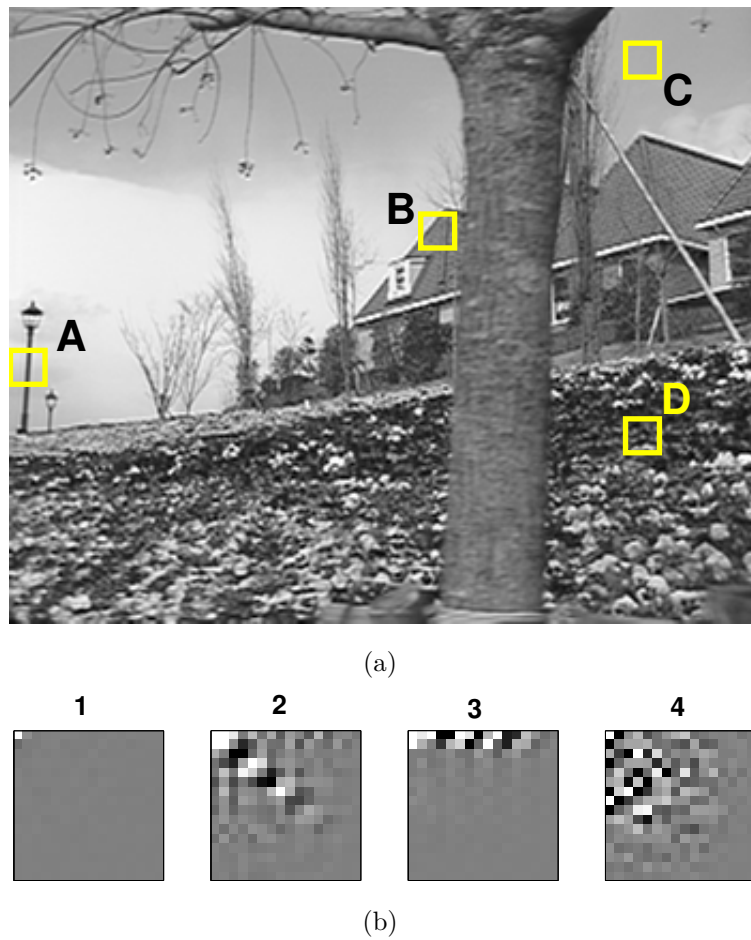


Figure 5.4: Example of block-based two-dimensional DCT. a) Original image with four marked blocks, block size 16×16 . b) 2D-DCT coefficients (magnified and scaled).

Practical Part:

Exercise: 5.1: Implement the block-based two-dimensional DCT described above for input images `input_image` of arbitrary size and for square blocks of size `[blocksize×blocksize]`:

```
output_image = blockbased_dct_on_image(input_image,blocksize)
```

Use the MATLAB function `dct2`. Consider whether or not image padding may be useful.

Test:

Run the test file

```
verify_blockbased_dct_on_image.
```

With your implementation of the block-based 2D-DCT, the DCT coefficients of different input test images are calculated. Your results are compared to the expected DCT coefficients both graphically and numerically.

Exercise: 5.2: Now implement the MATLAB function

```
output_image = blockbased_idct_on_image(input_image,blocksize),
```

which reconstructs the original image (`output_image`) from the DCT coefficients of `input_image`. Let the block size be `[blocksize×blocksize]`. Both height and width of `input_image` are a whole-number multiple of `blocksize`.

Test:

Run the test file

```
verify_blockbased_idct_on_image.
```

With your implementation of the block-based 2D-IDCT, the original images are reconstructed from the DCT coefficients. Your results are compared to the expected image pixels both graphically and numerically.

5.2 Zigzag Scan

As already mentioned, the low-frequency parts of natural images contain most of the information. In contrast, the high-frequency parts contain little to no image information. The DCT coefficients can be coded particularly efficiently via run-level coding (cf. section 5.3) if many zeros appear consecutively. Therefore, we now introduce the so-called zigzag scan, which sorts the DCT coefficients by ascending frequencies of the corresponding basis functions.

5.2.1 Basics

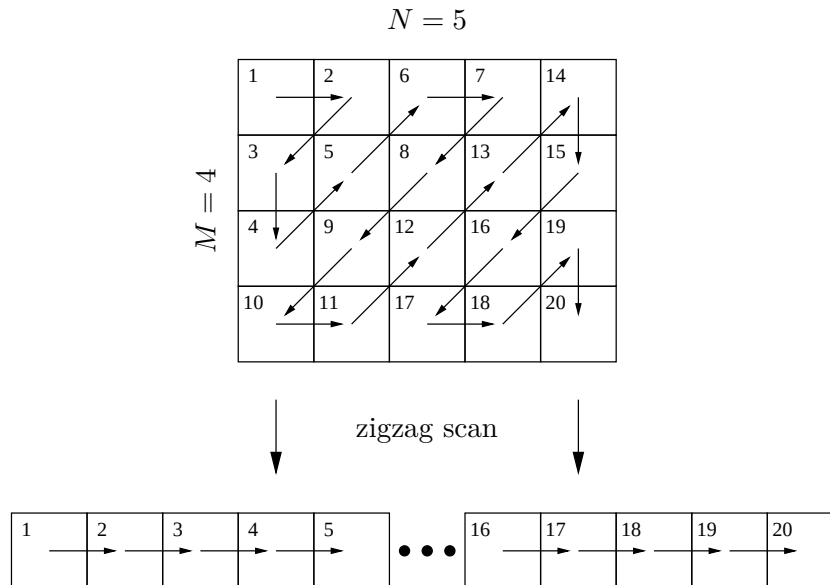


Figure 5.5: Zigzag scan of a 4×5 matrix.

Figure 5.5 shows the zigzag scan for a 4×5 matrix. In general, the elements of an arbitrary $M \times N$ matrix are numbered according to the shown zigzag pattern and then re-sorted into a $1 \times MN$ vector. For an $M \times N$ block of DCT coefficients, this means sorting the DCT coefficients from low-frequency to high-frequency basis functions.

Preparation: _____

Preparation: 5.2: Read the function description in **Exercise 5.3** thoroughly. Think about how you would realize the function and write down your ideas either as a

data structure diagram or in a structured pseudo code! (Your diagram or pseudo code are then used for the implementation of the zigzag scan.)

Practical Part:

Exercise: 5.3: Write the MATLAB function

`permutation_matrix = generate_zigzag_permutation_matrix(width,height)`
 which builds a permutation matrix of size $[M \times N]$. The elements of the matrix correspond to the indices of the elements as yielded by the zigzag scan in Figure 5.5 ($\text{width} \hat{=} N$, $\text{height} \hat{=} M$).

Example: `generate_zigzag_permutation_matrix(3,2)` yields

$$\text{permutation_matrix} = \begin{bmatrix} 1 & 2 & 5 \\ 3 & 4 & 6 \end{bmatrix}. \quad (5.12)$$

Test:

Run the test file

`verify_generate_zigzag_permutation_matrix.`

Pre-computed permutation matrices are compared to the permutation matrices built by your implementation.

Exercise: 5.4: Implement the MATLAB function

`zigzag_scanned = encoding_to_zigzag_scanned(input_image)`

which transforms the block `input_image` of size $[\text{height} \times \text{width}]$ into the $1 \times MN$ vector `zigzag_scanned` via a zigzag scan. Perform this transform with the help of a permutation matrix.

Test:

Create permutation matrices with the dimensions given in Table 5.1.

width	2	4	8	3	6
height	2	4	8	7	4

Table 5.1: Dimensions of the permutation matrices.

Use each permutation matrix as `input_image` and create the output vectors `zigzag_scanned` with your MATLAB function `encoding_to_zigzag_scanned`. The elements of the output vectors `zigzag_scanned` should then be, of course:

$$\text{zigzag_scanned} = [1 \ 2 \ 3 \ \cdots \ \text{width} \cdot \text{height}] \quad (5.13)$$

Exercise: 5.5: Write the MATLAB function

```
output_image=decoding_from_zigzag_scanned(zigzag_scanned,width,height)
```

which transforms the input vector `zigzag_scanned` into the image `output_image` that it is based on. As parameters, the image dimensions `height` and `width` are also passed. For your implementation, you can assume consistent input data `zigzag_scanned`, `width`, and `height` i.e., the requirement

$$\text{length}(\text{zigzag_scanned}) = \text{width} \cdot \text{height} \quad (5.14)$$

is met. Again, use a suitable permutation matrix!

Test:

For the input data given in Table 5.2, create the corresponding output images `output_image`. `output_image` should correspond to the permutation matrix of size `[height×width]` in all cases.

zigzag_scanned	width	height
= 1:4;	2	2
= 1:16;	4	4
= 1:64;	8	8
= 1:21;	3	7
= 1:24;	6	4

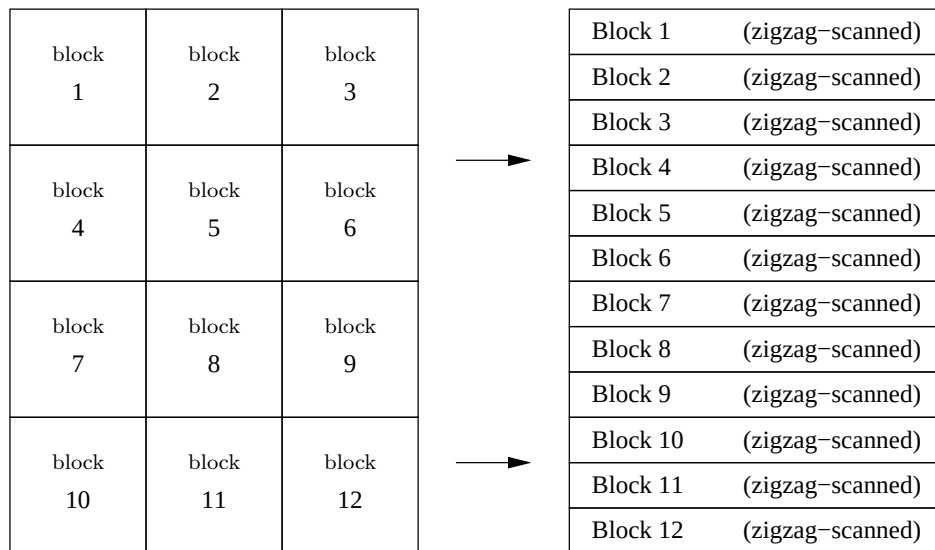
Table 5.2: Input data for testing `decoding_from_zigzag_scanned`.

Exercise: 5.6: Write the MATLAB function

```
zigzag_scanned =
    blockbased_encoding_to_zigzag_scanned(input_image,blocksize)
```

which transforms the input image `input_image` block by block via a zigzag scan. Figure 5.6 illustrates the procedure:

1. `input_image` of size $[\text{height} \times \text{width}]$ is divided into blocks of size $[\text{blocksize} \times \text{blocksize}]$.
2. Each of these blocks is transformed into vectors of size $[1 \times \text{blocksize}^2]$ using `encoding_to_zigzag_scanned`.
3. The resulting row vectors are combined into a matrix. The size of the matrix is $[\frac{\text{width} \cdot \text{height}}{\text{blocksize}^2} \times \text{blocksize}^2]$. The order of the row vectors in the output matrix `zigzag_scanned` corresponds to the block numbering in Figure 5.6.



left: input image divided into 12 square blocks

right: matrix of the zigzag-scanned blocks

Figure 5.6: Block-based zigzag scan.

Test:

Run the test file

```
verify_blockbased_encoding_to_zigzag_scanned.
```

The input variables `input_image` and `blocksize` along with the expected output matrix `zigzag_scanned` are generated. The matrix `input_image` is transformed with your implementation. The result of this transformation is compared to the expected result.

Exercise: 5.7: Write the MATLAB function

```
output_image = blockbased_decoding_from_zigzag_scanned
```

`(zigzag_scanned,blocksize,width,height)`

which transforms the matrix `zigzag_scanned` of size $\lceil \frac{\text{width} \cdot \text{height}}{\text{blocksize}^2} \times \text{blocksize}^2 \rceil$ (corresponds to the output matrix of `blockbased_encoding_to_zigzag_scanned`) into the original image `output_image` of size $[\text{height} \times \text{width}]$. The block size is $[\text{blocksize} \times \text{blocksize}]$. For your implementation, you can assume consistent input data `zigzag_scanned`, `blocksize`, `width`, and `height`, i.e., the following requirements are met:

$$\text{size}(\text{zigzag_scanned}, 1) = \frac{\text{width} \cdot \text{height}}{\text{blocksize}^2} \quad (5.15)$$

$$\text{size}(\text{zigzag_scanned}, 2) = \text{blocksize}^2 \quad (5.16)$$

$$\text{blocksize} \cdot m = \text{height} \quad m \in \mathbb{N} \quad (5.17)$$

$$\text{blocksize} \cdot n = \text{width} \quad n \in \mathbb{N} \quad (5.18)$$

Use your function `decoding_from_zigzag_scanned` for transforming the rows of the matrix `zigzag_scanned` into the corresponding image blocks. The assignment of the rows of the matrix `zigzag_scanned` to the image blocks is given in Figure 5.6.

Test:

Start the test file

`verify_blockbased_decoding_from_zigzag_scanned.`

The input variables `zigzag_scanned`, `blocksize`, `width`, and `height` along with the expected output image `output_image` are generated. The matrix `zigzag_scanned` is transformed with your implementation. The result of this transformation is compared to the expected result.

5.3 Run-Level Coding

5.3.1 Basics

In Section 5.1.3, we have seen how the input image of an image or video coder is divided into blocks of size $[M \times N]$ and how the two-dimensional DCT is applied to the individual blocks. The result is a matrix with the same dimensions as the original block, but the content is now made up of transform coefficients.

Before further processing is done, those coefficients are quantized. In the following, it is assumed that a uniform midread quantizer is used as described in Section 3.2. The result is a matrix with – depending on the applied quantization matrix – comparatively small integer values. An example of how a block of size 8×8 might look like after performing DCT and quantization (and de-quantization!) is depicted in Figure 5.7.

42	-5	1	0	0	0	0	0
4	-3	0	0	0	0	0	0
0	0	-1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Figure 5.7: Example of an $[8 \times 8]$ block after DCT and quantization.

Generally, many of the coefficients are quantized to zero. Only values that had an original absolute value greater than half of the quantizer step size are now non-zero. They have a higher probability of occurrence in the low frequency range, i. e., in the upper left part of the DCT coefficient matrix. After the zigzag scan, which we already examined in the

previous section, our example yields the following vector of length $M \cdot N = 64$:

$$\text{zigzag_scanned} = \begin{bmatrix} 42 & -5 & 4 & 0 & -3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & \dots & 0 \end{bmatrix} \quad (5.19)$$

In this representation, one typically obtains longer sequences of consecutively occurring zeros. This consecutive occurrence of zeros in the sequence is also called *run*. *Run-level coding* now transforms the vector obtained from the zigzag scan into symbols of the form $(run, level)$. Here, *level* denotes the value of a non-zero coefficient and *run* the number of zeros preceding it. For example, run-level coding generates the following symbols for the vector from (5.19):

$$(0, 42), (0, -5), (0, 4), (1, -3), (0, 1), (6, -1), EOB$$

EOB denotes an “end-of-block” mark, i.e., from this position onwards, all further coefficients are zero and do not have to be transmitted.

Practical Part:

Exercise: 5.8: Write the MATLAB function

```
runlevel_representation =
    encoding_to_runlevel_representation(input_vector)
```

which transforms `input_vector` into its run-level representation. The output `runlevel_representation` shall be a $[U \times 2]$ matrix, where U denotes the number of non-zero entries contained in `input_vector`:

$$\begin{aligned} \text{runlevel_representation} = & \begin{bmatrix} \text{run1}, & \text{level1}; \\ & \text{run2}, & \text{level2}; \\ & \vdots & \vdots \\ & \text{runU}, & \text{levelU} \end{bmatrix}; \end{aligned} \quad (5.20)$$

Since MATLAB knows the number of rows of `runlevel_representation` and thus the number of runs, it is not necessary to append an EOB mark.

Test:

Start the test file

```
verify_encoding_to_runlevel_representation.
```

With your implementation, input vectors of different lengths are transformed into their run-level representations and compared to the expected results. If the results do not match, an error is returned and the input/output data is shown.

Exercise: 5.9: Write the MATLAB function

```
output_vector= decoding_from_runlevel_representation  
                (runlevel_representation, blocksize)
```

which transforms `runlevel_representation` back into the original vector `output_vector` of size $[1 \times \text{blocksize}^2]$.

Test:

Start the test file

```
verify_decoding_from_runlevel_representation.
```

With your implementation, different run-level representations generated from the input data with different block sizes are decoded. The returned output vector is compared to the expected vector. If the results do not match, an error is returned and the input/output data is shown.

Exercise: 5.10: Write the MATLAB function

```
runlevel_representation=  
    blockbased_encoding_to_runlevel_representation(zigzag_scanned).
```

The input matrix `zigzag_scanned` is the result of the block-based zigzag scan from **Exercise 5.6**. Each row of the input matrix thus represents a block of the input image after the zigzag scan (cf. Figure 5.6 on page 97).

Use your already written function `encoding_to_runlevel_representation` in order to transform each of those rows into its corresponding run-level representation. The output matrix `runlevel_representation` shall then be of the following for-

mat:

$$\begin{aligned}
 \text{runlevel_representation} = & \begin{bmatrix} \text{block_1_run1}, & \text{block_1_level1}; \\ & \text{block_1_run2}, & \text{block_1_level2}; \\ & \vdots & \vdots \\ & \text{EOB}, & \text{EOB}; \\ & \text{block_2_run1}, & \text{block_2_level1}; \\ & \text{block_2_run2}, & \text{block_2_level2}; \\ & \vdots & \vdots \\ & \text{EOB}, & \text{EOB}; \\ & \text{block_3_run1}, & \text{block_3_level1}; \\ & \text{block_3_run2}, & \text{block_3_level2}; \\ & \vdots & \vdots \\ & \text{EOB}, & \text{EOB}; \\ & \vdots & \vdots \end{bmatrix} \quad (5.21)
 \end{aligned}$$

Contained is the run-level representation for all blocks of an image, where each block ends with an EOB mark. As an EOB mark, the value -1 shall be used since $(-1, -1)$ does not describe a valid run and hence can be unambiguously identified.

Test:

Start the test file

```
verify_blockbased_encoding_to_runlevel_representation.
```

For two different input data sets which represent block-based zigzag scans, your function computes the run-level representations. They are then compared to the expected values.

Exercise: 5.11: Write the MATLAB function

```
zigzag_scanned= blockbased_decoding_from_runlevel_representation
                (runlevel_representation, blocksize)
```

which reverses the procedure of **Exercise 5.10** and reconstructs the original zigzag representation from an input matrix of the format depicted in (5.21). You can make use of your already written function `decoding_from_runlevel_representation`

for the individual blocks.

Test:

Start the test file

```
verify_blockbased_decoding_from_runlevel_representation.
```

For two different run-level representations, the original zigzag-scanned data is reconstructed with the help of your function and compared to the expected values.

5.4 Intra Coding Chain

5.4.1 Basics

In the previous lab experiments, you have learned about different individual aspects used in image and video coding. Meanwhile, you have developed all the tools needed to code and decode images in a lossless or lossy manner. Therefore, in this lab experiment, these individual components will be combined into an intra coding chain. You will thus obtain a fully-functional image encoder and decoder. Single image coding of this kind is nowadays used for storing and transmitting images of all sorts. The procedure most commonly used by far is the JPEG standard, without which practically no web page or digital camera can go anymore.

5.4.2 Encoder

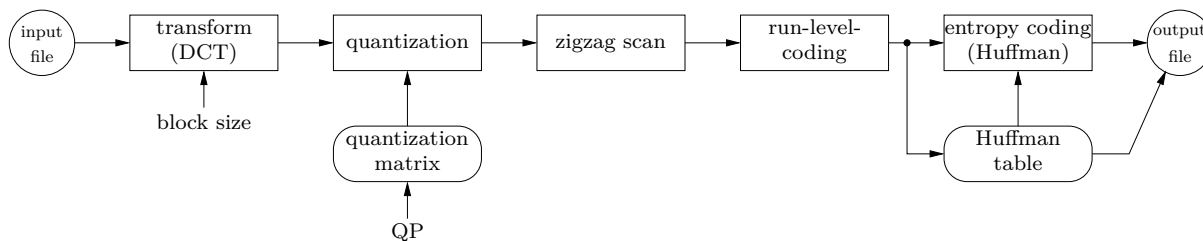


Figure 5.8: Layout of the intra coder.

Figure 5.8 schematically depicts how the intra coder is to be implemented in this lab experiment. At the encoder side, the input image is read from a YUV file as a first step. If the file contains more than one frame, the first one of them shall be read. For this purpose, the already implemented function

```
[output_image_Y, output_image_U, output_image_V] =
    yuv_read_one_frame(filename, image_number, width, height)
```

is available. Since only grayscale images are considered here, you will need only the luminance component `output_image_Y` of the returned data for further processing.

In the next step, both DCT and quantization are performed. For the quantization, a quantization matrix is needed. You can make use of the provided function

```
quantisation_matrix= get_quantisation_matrix(qp, transform_blocksize) .
```

This function creates a quantization matrix in which the DC coefficient, which is important for the image quality, is always quantized at a high resolution and all other step sizes are dependent on the so-called quantization parameter `qp`. The greater `qp` is chosen, the coarser the transform coefficients are quantized, which ultimately leads to lower data rates but also to a lower image quality. The range of reasonable values is: $qp = \{1, 2, \dots, 35\}$.

Following the quantization step, both zigzag scan and run-level coding are performed. The values of the run-level representation are then Huffman coded, after a Huffman table has been computed from the same values first. The resulting bitstream along with the Huffman table can then be stored in an output file.

This output file shall contain all the parameters which the decoder later requires in order to be able to decode the data stream. Therefore, storing only the Huffman table and the bitstream is not enough: The width and height of the original image, the block size of the DCT, and the quantization parameter `qp` have to be stored as well. You can achieve this by using the following command, for instance:

```
save(coded_file, 'width', 'height', 'transform_blocksize',  
    'qp', 'huffman_table', 'bitstream')
```

5.4.3 Decoder

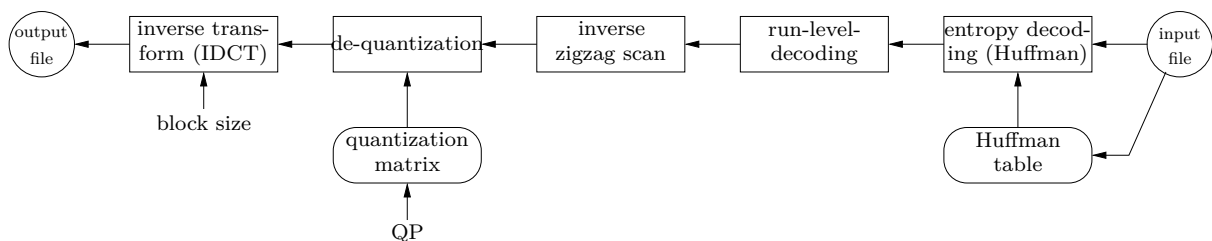


Figure 5.9: Layout of the intra decoder.

Figure 5.9 shows the layout of the decoder. Both the compressed data and the stored parameters have to be read from the source file first. Then the individual steps of the encoder are inverted and the decompressed result is written into a YUV file. For this purpose, the already implemented function

```
retval= yuv_write_one_frame(filename, frame_no, Y, U, V)
```

is available. It writes the grayscale image containing the luminance values Y into the output file `filename`. The parameters U and V are optional and need not be passed in the case of grayscale images. The parameter `frame_no` should always be set to 1 here since only one image is written.

Preparation: ---

Preparation: 5.3: Make a list of all the MATLAB functions that have been implemented so far and are required for the *intra coder*. Put the functions into the order in which they have to be executed.

Preparation: 5.4: Make a list of all the MATLAB functions that have been implemented so far and are required for the *intra decoder*. Put the functions into the order in which they have to be executed.

Practical Part: ---

Exercise: 5.12: Write the MATLAB function

```
encode_intra(input_yuv_file, coded_file, width, height,  
             transform_blocksize, qp).
```

It is supposed to read the first frame from the file `input_yuv_file`, compress it via the described coding chain, and write it into the output file `coded_file`. Make use of the components already implemented in the previous lab experiments.

How many bytes are used for storing the luminance values of the uncompressed input file? Compare this to the size of the output file! What effect has a change of the quantization parameter (`qp`)?

Exercise: 5.13: Write the MATLAB function

```
decode_intra(coded_file, decoded_yuv_file)
```

which reads the compressed image stored in `coded_file` and decodes it according to the described intra decoding chain. Make use of your list of already implemented functions.

Since the decoder works with quantized coefficients, there might be luminance values outside the range of $[0; 1]$ after the inverse transform. Therefore, you have to limit

the range after the inverse transform by searching those values in the matrix and replacing them with 0 or 1.

Test:

Start the test file

```
verify_intra_coding(qp)
```

which encodes a frame from the flowergarden sequence with your function `encode_intra` and decodes it again with the function `decode_intra`. You may freely choose the quantization parameter. The decoded frame is shown along with the original frame. In addition, the MSE, the PSNR, the data rate in bits per pixel, and the code rate are displayed. Hint: The resulting PSNR must not be lower than 20 dB for `qp` values smaller than 35.

If the test has been successful, display a D(R) curve by using a vector with several values for the parameter `qp`. The coding procedure is then applied to all given values and the results are combined into a D(R) curve.

5.5 Optional Part

5.5.1 Differential Coding of the DC Coefficient

After the transform, the DC coefficient of each block, i.e., the constant component, is usually quantized with a high resolution. This coefficient corresponds to the average luminance value of the block. Alterations of this coefficient lead to severe blocking artifacts which affect the image quality considerably. The large values left for the DC coefficient after quantization are not beneficial to entropy coding later on, though.

It is more efficient to encode the DC coefficient differentially so that only the coefficient of the first image block is transmitted as an absolute value. For all other blocks, only the difference between the current DC coefficient and the DC coefficient of the previous block is stored. You can achieve this by processing the first column of the data (which corresponds to the DC coefficients) after the zigzag scan accordingly.

Practical Part: _____

Exercise: 5.14: Write the MATLAB function

```
zigzag_scanned= optional_dc_prediction(zigzag_scanned)
```

which transforms the DC coefficients (the first column of the matrix `zigzag_scanned`) into a differential representation. Adjust your intra coder `encode_intra` so that the new function is called at the appropriate position.

Exercise: 5.15: Write the MATLAB function

```
zigzag_scanned= optional_dc_prediction_inverse(zigzag_scanned)
```

which reverses the procedure of differential coding. Adjust your intra decoder `decode_intra` so that this function is called at the appropriate position.

Test:

Executing the test program

```
verify_intra_coding(qp)
```

anew, verify whether encoding and decoding still lead to the desired result. Make sure that the achieved PSNR for a given quantization parameter is equal to the PSNR obtained without DC prediction. Compare the achieved data rates with and without DC prediction for different quantization parameters and calculate how many percent are saved.

5.6 Summary

In the first part of this lab experiment, you have dealt with transform coding and the zigzag scan. For this purpose, you have written functions with which to transform the image data block by block into the frequency domain. Furthermore, you have realized the zigzag scan, which makes efficient coding of the DCT coefficients possible.

In the second part of this lab experiment, you have implemented run-level coding. At that point of the lab course, you had written all functions needed for building a fully-functional intra coder, which was then realized at the end of part two of the lab experiment. In the next lab experiment, you will enhance your intra coder with the functionality of inter coding by incorporating the functions implemented in Lab Experiment 4.

5.7 Literature Recommendation

In the implemented intra coding chain as depicted in Figure 5.8 the first frame of the input video is transformed, quantized, and entropy coded. However, this usually results in higher bitrates since high transform coefficients have to be submitted. Thus, intra prediction is performed that tries to predict the current block from the blocks that have been coded before and which are available at the decoder side. Hence, the residual image between the prediction and the original can be transmitted which requires less bits. Further information can be found for HEVC intra prediction in:

J. Lainema, F. Bossen, W. Han, J. Min, and K. Ugur, “Intra coding of the HEVC standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1792–1801, Dec 2012

Lab Experiment 6

Video Codec

During the past weeks of the Image and Video Compression Lab Course, you have learned about the fundamental techniques on which many current multimedia applications are based. The goal of this lab experiment is to make use of the learned methods and build a fully-functional system for digital video compression. In the first session, each group will thus build a basic coder (Section 6.2). It will serve as a reference with which to compare your own improvements implemented during the remaining sessions (Section 6.3). On the last day of the lab course, in the second session of Lab Experiment 7, the obtained results will then be discussed and compared to those of the other groups. That is why it is necessary to appropriately store all the results obtained in this lab experiment. It is also very important to thoroughly work on all preparation exercises in order to be able to finish the practical part in time.

6.1 Layout of a Video Codec

In the following section, the layout of a block-based video codec is summarized, similar to how it is defined in the standards MPEG-1, MPEG-2, MPEG-4 Part 2 (also called MPEG-4 Visual), and MPEG-4 Part 10 (also called MPEG-4 AVC, or more commonly H.264/AVC). The basic relationships have already been described in the introduction and examined in-depth in the previous Lab Experiments 2 through 5. So here, only the overall layout of a video codec is described.

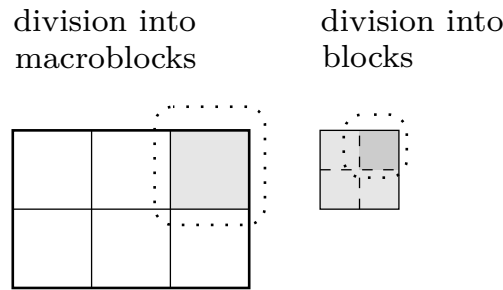


Figure 6.1: Division of a frame into macroblocks and transform blocks.

6.1.1 General Layout

At first, an image is coded without temporal prediction. This kind of image is called an intra frame (I-frame), which you already know about from Lab Experiment 5. The decoding of a bitstream can always start anew at an I-frame since it does not need any other frame as a reference. The I-frame can then be followed by several temporally predicted frame types.

Prediction usually uses block-based motion compensation. The predicted frame types are divided into P-frames, which are predicted from the most recently decoded frame, and B-frames, which are predicted from the two most recently decoded frames. In the standards used nowadays, it is additionally decided for each macroblock in a P- or B-frame which mode (I, P, or B) is used for coding the respective macroblock. Thus it is possible for an intra-coded macroblock to occur in a P-frame, for example. In B-frames, it is even possible to choose between forward prediction, backward prediction, bidirectional prediction, or intra-coding for each macroblock. In order to limit the scope of the lab course, the basic coder is limited to I- and P-frames.

For block-based methods, a frame is first divided into blocks of some given size, the so-called macroblocks. The MPEG standards up until MPEG-4 AVC define a macroblock size of 16 pixels in each dimension. The macroblocks of a frame are processed row by row (raster scan order). Usually, a macroblock is further divided into square blocks with an edge length of 8 pixels. The division of the frame into macroblocks and the further division of macroblocks in blocks is depicted in Figure 6.1. In this lab course, the division will be simplified.

The coding of an **I-frame** is depicted in Figure 6.2. For intra-frame coding, only those functional blocks are required which are not highlighted in gray. The intra frame is first transformed block by block via the DCT. The DCT coefficients of each block are then quantized and re-sorted into vector form by using zigzag scanning. The result is further

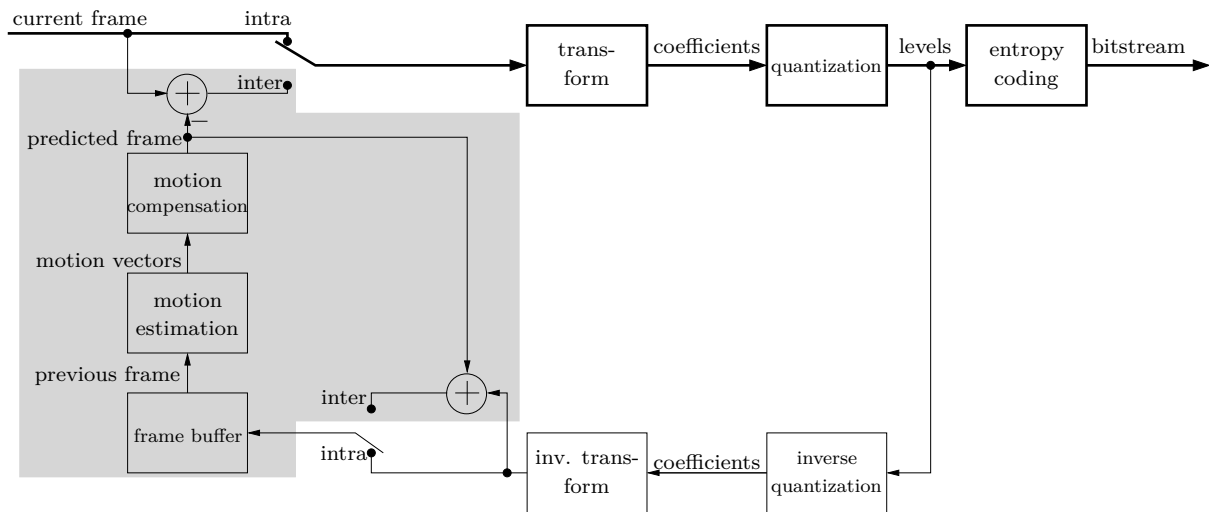


Figure 6.2: Overview of the basic coder; functional blocks needed for coding P-frames are highlighted in gray.

transformed into a run-level representation and then stored in the bitstream via Huffman coding. To make available the same reference frame for the subsequent temporally predicted frame types at both the encoder and the decoder, the stored blocks have to be de-quantized and inversely DCT transformed at the encoder (decoder loop). The frame data thus obtained is then stored in the frame buffer and available for predicting the following frame.

For **P-frames**, the frame is predicted block by block from the previously coded frame via motion estimation and compensation before the DCT transform is performed. The predicted frame is then subtracted from the input frame to be coded. The resulting residual error signal is transformed via the DCT, quantized, transformed into vector representation and run-level coded – just like an I-frame. The run-level representation as well as the motion vectors still contain redundancy which is further reduced via Huffman coding. The Huffman code words for the run-levels and the motion vectors are then written into the data stream. Just like with I-frames, the *reconstructed* P-frame has to be stored as a reference for the subsequent frames. For this purpose, both the quantization and the DCT transform steps of the residual error signal are inverted and the frame predicted from the motion vectors is added. This reconstructed frame is again stored as a reference in the frame buffer. By using this procedure, the reference frame data becomes identical at both the encoder and the decoder.

6.1.2 Simplifications in this Lab Course

In order to keep matters simple for this lab course, the layout described above is slightly modified in some aspects. Coding is limited to grayscale images (luminance component only). The chrominance components of video data are often spatially downsampled. This is due to psycho-visual effects which state that the human visual system is more perceptive to luminance variations than it is to color information. In principle, the coding of the chrominance components is done like the coding of the luminance information. However, it makes the implementation more error-prone and more costly in terms of time. By limiting ourselves to grayscale frames, the focus here lies on experimenting and improving the individual functional blocks.

Furthermore, the division into macroblocks and transform blocks is simplified. In this lab experiment, the frame to be transformed is arranged beforehand as a whole image. Thus the effects of the individual functional blocks can be measured in a better way. Additionally, the modularity of the layout increases, leading to an easier expandability. This allows changing the parameters of the motion estimation independently from the transform or adding a preceding intra/inter decision.

Finally, Huffman coding is performed for the whole set of run-level data and motion vectors after the last frame has been coded. Hence only one Huffman table is required for every transmitted information, which results in an easier implementation. In comparison to that, existing standards make use of a fixed Huffman table and each macroblock is written into the bitstream immediately. As a result, each macroblock can be decoded separately, minimizing delays and making the sequential processing of the data stream possible.

6.2 Development of the Basic Video Codec

The goal of this section is the development of the basic encoder and decoder. The basic encoder is supposed to always code the first frame as an I-frame. All remaining frames are to be coded as P-frames. At first, an encoder/decoder pair for I-frames is developed based on Lab Experiment 5. Then the program is extended by P-frame coding.

6.2.1 Coding and Decoding the I-Frame

You have already learned about and implemented a coder for an I-frame in Lab Experiment 5. That program will now be extended. In contrast to Lab Experiment 5, the

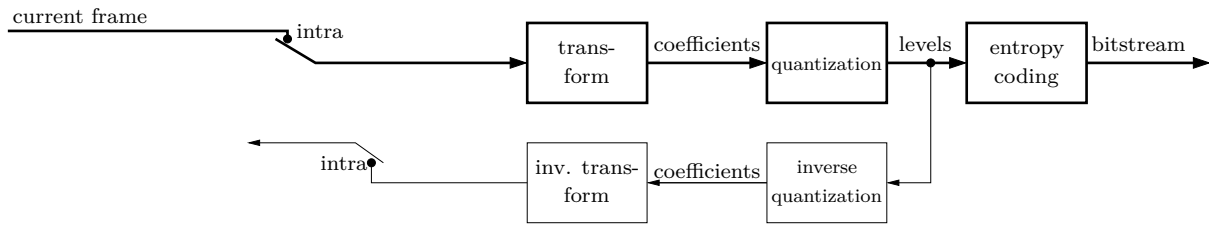


Figure 6.3: Components of the intra coder.

reconstruction of the current frame from the quantized DCT coefficients must be developed additionally for the encoder. The necessary components are depicted in Figure 6.3. The components marked in bold have already been implemented in Lab Experiment 5. The decoder from Lab Experiment 5 does not have to be changed for decoding a single frame.

This intra coder shall be extended in a second step such that it is possible to encode sequences of I-frames. To that end, an extra loop over all individual frames to be encoded is needed. When decoding multiple frames, the decoder first decodes all run-level representations from the Huffman code. For the remaining decoding steps, it is then necessary to search for the run-level boundaries of the coded transform blocks by means of end-of-block marks.

Preparation: _____

Preparation: 6.1: Think about how to extend your listing from Lab Experiment 5, **Preparation-5.3**, to encode several frames of a file. Note that all run-level data is Huffman coded only after the last frame has been processed. The number of frames to be coded shall be adjustable.

Preparation: 6.2: On which parameters does the number of coded blocks per frame depend?

Preparation: 6.3: Given a list containing all the block boundaries (= row numbers) within the run-level array, how can you cut out the part needed for decoding frame k from the complete list of run-level data?

Hint: It is necessary to know the number of blocks per frame. Think about which indices from the EOB-list you need to use.

Preparation: 6.4: Give a break condition for the decoder which is dependent on the current index of the run-level block data!

Practical Part:

Exercise: 6.1: Copy and paste both intra encoder and decoder from Lab Experiment 5 into the new files “encoder_basic_intra.m” and “decoder_basic_intra.m”, respectively. Do not forget to change the function names within the files accordingly. The function definition should then be:

```
[coded_bits, mse_per_frame]=encoder_basic_intra(input_yuv_file,...
coded_file, width, height, transform_blocksize, qp)
```

The return values of the function, `coded_bits` (number of bits of the coded data stream) and `mse_per_frame` (array with the MSE value of each frame), can be determined by the provided function `bitstream_get_length` and by the function `mse_of_frame` from Lab Experiment 3.

Exercise: 6.2: Extend the intra coder from Lab Experiment 5 according to Figure 6.3 such that the decoded image data of the current frame is made available for the subsequent P-frames. For testing, use the sequence “flowergarden_short_cif.yuv”.

Exercise: 6.3: Quantization can cause the pixel values in the decoded frame to exceed the range $[0; 1]$. Therefore, clipping to this range has to be performed after reconstructing the frame. For clipping to value 0, you can use the following command:

```
reconstructed_image(reconstructed_image<0)=0;
```

Clipping to value 1 is done analogously.

Add this procedure of clipping to the range $[0; 1]$ to your coder and decoder.

Exercise: 6.4: Now extend the coder in such a way that several frames can be coded successively. Huffman coding shall only be performed after the last frame has been processed. Extend your function body as follows:

```
[coded_bits, mse_per_frame]=encoder_basic_intra(input_yuv_file,...
coded_file, width, height, number_of_frames, transform_blocksize,
qp)
```

The parameter “number_of_frames” shall here denote the number of frames to be encoded.

Exercise: 6.5: Now the intra decoder has to be modified in order to gain the capability of decoding several frames successively. The run-level data array now contains the data for all blocks of all coded frames. To be able to use the function `blockbased_decoding_from_runlevel_representation(...)`, the run-level data of the current frame has to be extracted. Moreover, a loop must be included with which to process all the frames.

1. Implement the decoder by making use of your considerations from preparations 6.2 through 6.4.
2. For verification, use `imshow` to display each decoded frame and also save it via the function `yuv_write_one_frame(filename, frame_nr, decoded_frame)` to a file.

Exercise: 6.6: Now create a D(R) curve for your encoder by intra coding the sequence “flowergarden_short_cif.yuv” using quantization parameters $qp = \{6, 8, 10, 15, 20\}$ and a transform block size of 8. Use the function

```
[rate,psnr,enc_time,dec_time]=get_dr_result_intra(encoder_function,...
decoder_function, input_yuv_file, qp_values, transform_blocksize).
```

The function simulates your coder with the given parameters and saves the results into a file. Furthermore, a diagram is created which you can save for later comparison with the complete video coder. While the simulation is in progress you can already proceed with the next steps.

6.2.2 Coding and Decoding P-Frames

Now the intra coder shall be extended by temporally predicted P-frames. You can see the configuration in the form of a block diagram in Figure 6.4. Assume that all frames, with the exception of the first one, are coded as P-frames. The basic layout corresponds to that

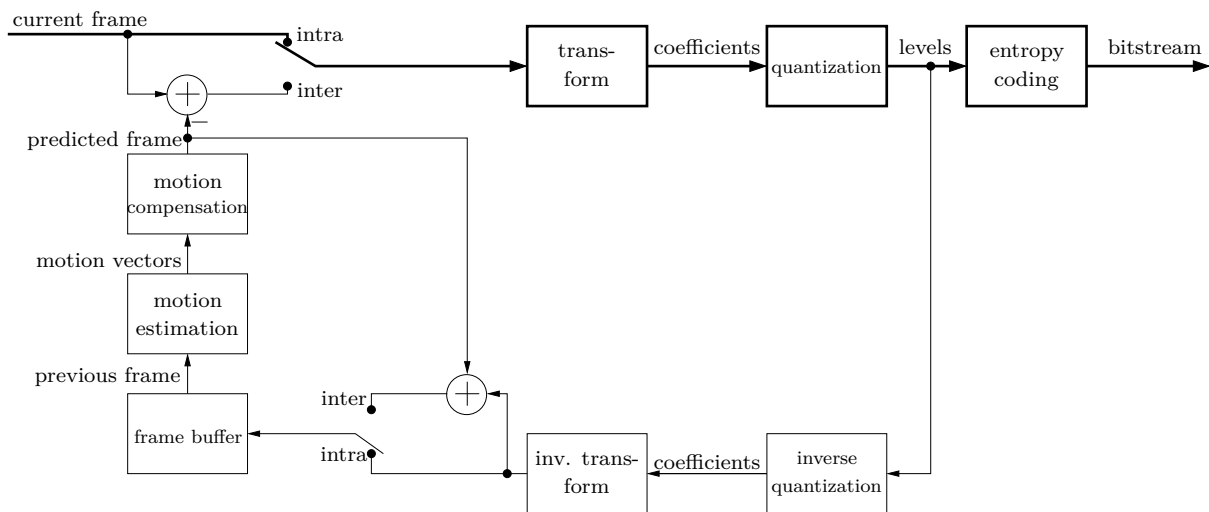


Figure 6.4: Block diagram of the coder for P-frames.

of coding I-frames. But now, the prediction loop for motion estimation and compensation must be introduced additionally. Only the coefficients of the computed residual error signal and the motion vectors are then encoded. In this lab course, Huffman coding is performed only after processing the last frame, just like for I-frames.

In order to be able to use the coded P-frames as references for the subsequent frames and to have the same reference as the decoder, the coding of the coefficients has to be reversed and the predicted frame be added.

Preparation: _____

Preparation: 6.5: Think about how to solve the following issues for the P-frame coder. Use the functions from the previous experiments.

1. Determining the frame type to be coded (I-frame or P-frame)
2. Which additional side information has to be stored for P-frames?
3. Write the program for coding and subsequently decoding a P-frame within the encoder. Mark those parts that form the decoding loop.
4. Which information do you now have to save in the file “coded_sequence.mat” so that the sequence can be completely reconstructed?

Practical Part: _____

Exercise: 6.7: Copy both the intra coder and decoder to new files “encoder_basic.m” and “decoder_basic.m”, respectively. Change the function definitions within the files as follows:

```
[coded_bits,mse_per_frame]=encoder_basic(input_yuv_file, coded_file,
width, height, number_of_frames, transform_blocksize, qp,
me_blocksize, me_searchrange);

decoder_basic(coded_file, decoded_yuv_file);
```

Exercise: 6.8: Basic coder: Modify the function `encoder_basic(..)` such that it encodes all but the first frame as P-frames. Use the following steps:

1. Include an if-condition which allows deciding on the frame type (I or P). Use the previously written source code for the intra coding part.

2. Now complete the inter coding part. Make use of the results of **Preparation 6.5**. For verification, display the following images: predicted frame, residual error frame before the quantization, coefficients, de-quantized residual error frame after the IDCT, reconstructed frame (each with `figure(1 through 5);imshow(frame);`)
3. Make sure that the data necessary for Huffman coding is also available later on.
4. As described for the intra coder, each reconstructed frame must be clipped to the range $[0; 1]$. Perform the necessary clipping operations! Note: Clipping is not to be applied to the difference images! Why?
5. Compute the MSE of the reconstructed frame and use the reconstructed frame as a reference for the subsequent frame.
6. After processing all frames, Huffman coding has to be applied. The result is saved into a file along with the required side information.

Exercise: 6.9: Basic decoder: Modify the function `decoder_basic(..)` so that the first frame is decoded as an I-frame and the subsequent frames as P-frames:

1. After reading the coded data, the first thing to do is to generate the motion vector data from the corresponding Huffman code.
2. Include a query for identifying the frame type. Use the previously written source code for decoding the I-frame!
3. Now complete all the steps necessary for decoding P-frames. Display the generated frames after each step and verify their correctness!
 - (a) Start by extracting the required motion vectors and run-level data of the current frame
 - (b) Generate the predicted frame using the motion vectors and the reference frame
 - (c) From the run-levels, generate the residual error frame and add it to the predictor
 - (d) As with the intra coder, each reconstructed frame must be clipped to the range $[0; 1]$. Perform the necessary clipping operations!

- (e) Save the decoded frame using the function
`yuv_write_one_frame(...)`
- (f) Use the reconstructed frame as a reference for the subsequent one!

Exercise: 6.10: Make sure that the coder and the decoder work together. For this purpose, encode three frames of the sequence “flowergarden_short_cif.yuv”. Then, delete the variables in your workspace via

```
clear all;
```

and finally decode the sequence from the encoded data.

Exercise: 6.11: Create a D(R) curve with the parameters used for the intra coder! For this purpose, you can use the function

```
[rate,psnr,enc_time,dec_time]=get_dr_result(encoder_function,...
decoder_function, input_yuv_file, qp_values, transform_blocksize,...
me_searchrange, me_blocksize);
```

This function visualizes the result and stores it in a mat-file along with the coding parameters. Compare the coding efficiency of the intra coder to that of the predictive coder! Choose a search range of 8, a transform block size of 8, and a motion estimation block size of 16!

6.2.3 Simulations with the Basic Codec

In this section, you will examine the basic coder just developed. The goal is to analyze the impact of the free parameters like the transform or motion estimation block size. For this purpose, use the sequences `vimto_short_cif.yuv`, `rugby_short_cif.yuv`, `shuttle_short_cif.yuv`. Make use of the function `get_dr_result`.

Practical Part: _____

Exercise: 6.12: At first, compare the coding efficiency of the basic coder for the sequences “`vimto_short_cif.yuv`”, “`rugby_short_cif.yuv`”, “`shuttle_short_cif.yuv`” using the settings of **Exercise 6.11**. Create the D(R) diagrams for these sequences.

Exercise: 6.13: Now use the sequence “`vimto_short_cif.yuv`”. Examine how different motion estimation block sizes of 8, 16 affect the hybrid coder by making use of D(R) diagrams. Choose a search range of 8 and a transform block size of 8.

Exercise: 6.14: Now compare the coding efficiency for different motion estimation search ranges 4, 8, 16. Choose a motion estimation block size of 16 and a transform block size of 8. Compare the results to those from Lab Experiment 4.

6.2.4 Submitting the Optimized Codec

First of all, many congratulations on creating your first self-programmed video codec! If you have correctly followed the lab course description up until today, you should now have a fully-functional video codec modeled after the successful MPEG standards.

In the penultimate lab session, we will compare the performance of some of your codecs in a subjective video test. In order to facilitate the selection, several diagrams and pieces of data are to be made available. On the last day of the lab course, the results of the subjective video test will be presented.

Practical Part:

For computing the coded sequences and the D(R) curves, it is necessary to guarantee a uniform interface for all groups. For this purpose, two functions are defined:

Exercise: 6.15: Write the functions

```
final_encoder(input_sequence_yuv,coded_file, width,...  
height, nr_of_frames, qp)  
and  
final_decoder(coded_file, decoded_sequence_yuv) .
```

You have already learned about the function parameters. All the other parameters like transform block size or search range can be defined within the function by yourselves.

Limitation:

The execution time with $qp = 8$ for coding 5 frames of size CIF **must not exceed 150 seconds!** You can measure the execution time with

```
tic;  
final_encoder('rugby_short_cif.yuv','rugby_q12.mat',352,288,5,8);  
toc .
```

You can influence this execution time by adjusting the parameters for both transform and motion estimation.

Exercise: 6.16: In your home directory `/HOMES/mmkgpXX`, you will find the directory `/HOMES/mmkgpXX/final/`. It contains a text file `codec_description.txt`. Please fill in the missing data and list any optional extensions that you have integrated into your codec! You might also want to have a look at the optional extensions proposed in Section 6.3.

Note: Before actually submitting your codec, remove (or comment) any `imshow` and `figure` commands you have included in your code for verification purposes! Furthermore, all of your files should be in the same folder, i. e., you should not put your files into dedicated lab experiment folders!

Important note: If you have enhanced your codec such that it is also able to code the chrominance components, please remember to save the related information into a separate file. This is required so that the corresponding information is not taken into consideration for the bitrate calculations, thus preventing any disadvantage. For this purpose, use the following command in the encoder, for example:

```
save([ coded_file '_color'], 'bitstream_u', 'bitstream_v', ...
     'huffman_table_u', 'huffman_table_v');
```

Exercise: 6.17: In order to generate comparable data sets with your final encoder/decoder pair, please use the function `get_final_result(sequence)`. This function generates both a PSNR plot as well as a plot which depicts the encoding and decoding times for several quantization parameters as shown in Figure 6.5. The results and the decoded sequences are automatically stored in the directory `/HOMES/mmkgpXX/final/`. First, execute the function for the sequence `rugby_short_cif.yuv` and again make sure that the encoding time for QP=8 does not exceed 150 s!

If you do not close the window created just now, you can extend the diagrams by calling the function again for the sequences

`vimto_short_cif.yuv` and `shuttle_short_cif.yuv`, respectively, and thus add the corresponding curves!

6.3 Own Extensions

Now that you have a fully-functional coder and decoder at your disposal, you can integrate your own extensions. At the end of the second session, you are supposed to hand in an

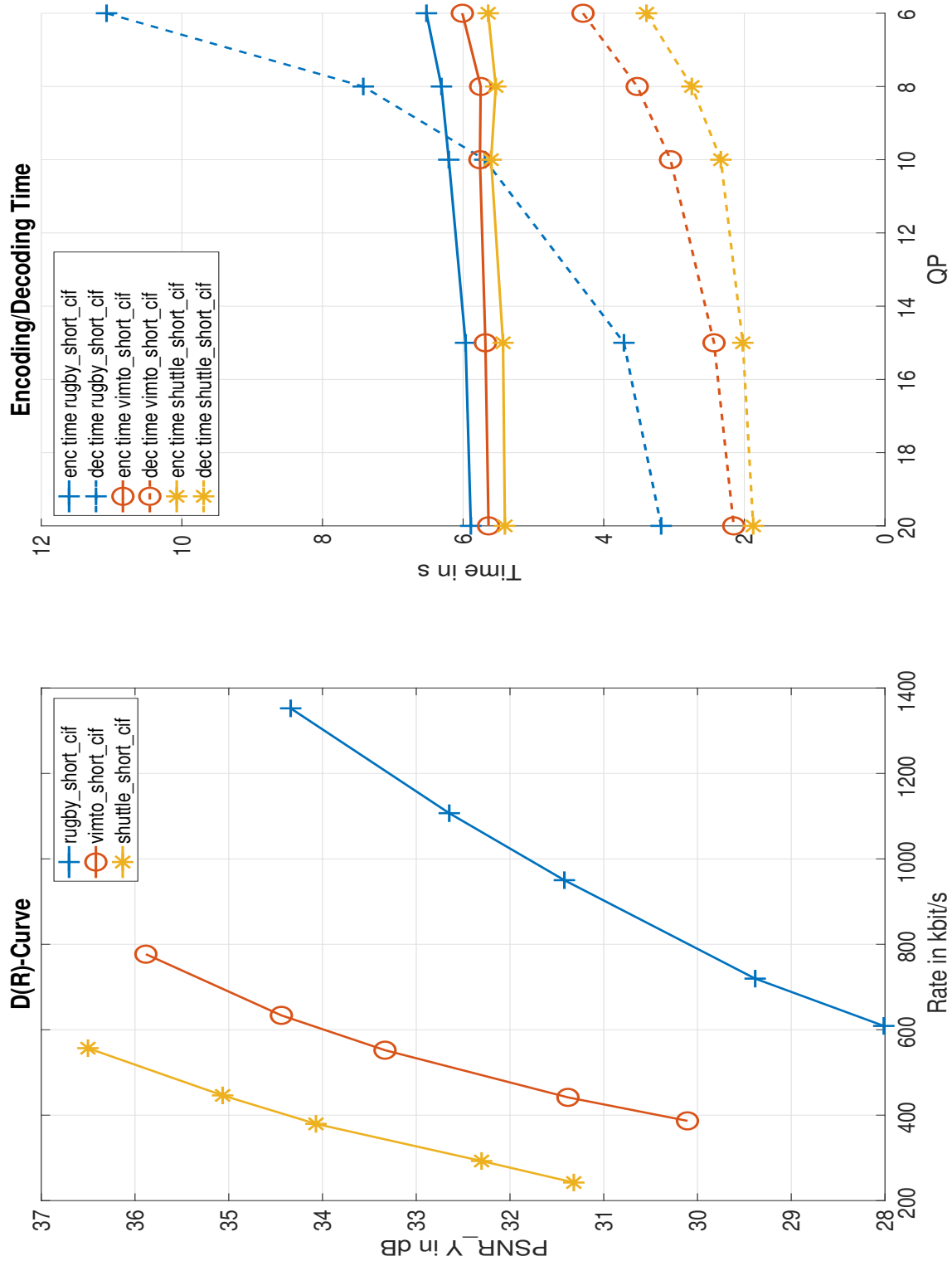


Figure 6.5: D(R) curves and coding times for 3 test sequences.

improved coder/decoder pair, which is then compared to the results of the other groups in the subjective video test. Approaches for some of the most common enhancements are given below. Note that, in general, those enhancements can be combined arbitrarily.

Before you start working on such an enhancement, copy and paste your functioning coder and decoder into new MATLAB files and change the function names accordingly! By doing so, you can later revert to already working functions.

6.3.1 Modifications for Motion Estimation

The major contribution to coding efficiency and execution time is usually made by motion estimation. Thus it seems obvious to improve this functional block.

In order to decrease the execution time, several suboptimal estimation procedures, such as 3-step-search or log-search, have been developed. Usually, they do not decrease the coding efficiency by much.

From experience, it is known that the search on full-pixel positions can lead to high temporal aliasing, which then considerably derogates the coding efficiency. In order to compensate for this disadvantage, one can either use a de-blocking filter (MPEG-1) or estimate and transmit the motion with subpixel accuracy. Furthermore, the energy of the residual error frame can be better compensated by using variable block sizes.

In case no good prediction is possible for the macroblock despite these procedures, it is recommended to provide an additional decision possibility that allows the use of intra coding for each individual block.

The de-blocking filter is described in Section 6.3.2. Instructions for implementing the other enhancements can be found in Lab Experiment 4.

Time demand: (if functions not yet available)

- Suboptimal procedures: average
The function must be written and then used in the coder instead of the full-pixel search.
- Subpixel accurate prediction: high
Modification of the search functions, interpolation, and corresponding adjustments in the encoder and the decoder.

- Variable block size: very high
New estimation functions, additional side information in the encoder and the decoder.
- Inter / intra decision: average
New functions for estimation and compensation.

6.3.2 De-Blocking Filter Within the Coding Loop

Short description:

After a frame is reconstructed, strong block artifacts caused by block-based quantization of the individual coefficients can usually be observed. A de-blocking filter helps to minimize this effect. If the filter is also used within the coding loop, an increased coding efficiency can be additionally achieved. This is mainly due to the better homogeneity of a frame with less block artifacts, which improves the results of motion compensation.

Time demand: average

Additionally needed functions:

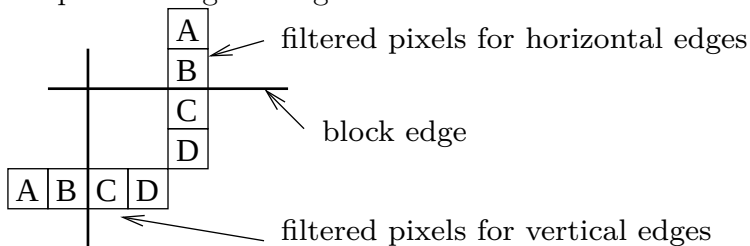
- `filtered_reconstructed_image= filter_image(reconstructed_image, transform_block_size, qp)`

Approach

- Filtering the reconstructed frame in both encoder and decoder

Example filter (cf. H.263 [8])

All pixels along the edges of the transform blocks are filtered:



In each step, the pixels A , B , C , and D are filtered along the edge. Proceed as follows:

1. Calculate the edge strength: $d = (A - 4B + 4C - D)/8$

2. Determine the filter strength *STRENGTH* as per the table below
3. Calculate the scalar $d1 = \text{UpDownRamp}(d, \text{STRENGTH})$;
 where:

$$\text{UpDownRamp}(d, \text{STRENGTH}) = \text{SIGN}(d) * (\text{MAX}(0, \text{abs}(d) - \text{MAX}(0, 2 * (\text{abs}(d) - \text{STRENGTH}))))$$
4. Calculate $d2 = \text{clipd1}(\frac{A-D}{4}, \frac{d1}{2})$; where $\text{clipd1}(x, l)$ means that x is limited to the range $\pm l$, e.g., the lower bound if $(x < -l)$ $x = -l$
5. Calculate the values
 $B_1 = \text{clip}(B + d1)$
 $C_1 = \text{clip}(C - d1)$
 $A_1 = A - d2$
 $D_1 = D + d2$
 where $\text{clip}(x)$ means that x is limited to valid pixel values $[0; 1]$.
6. Replace pixel A with A_1 , B with B_1 , and so on.
7. Process all horizontal edges first, followed by all vertical edges.

Apply the filter to all block edges of the reconstructed image in both encoder and decoder!

Table for **determining the filter strength** as a function of qp:

qp	1	2	3	4	5	6	7	8	9	10	11	12
STRENGTH	1	1	2	2	3	3	4	4	4	5	5	6
qp	13	14	15	16	17	18	19	20	21	22	23	
STRENGTH	6	7	7	7	8	8	8	9	9	9	10	

Remember to adjust the parameter *STRENGTH* according to our range!

6.3.3 Extension for Coding Color Data

Short description:

Coding of luminance and chrominance of YC_bC_r image data.

Note: The PSNR curves always apply to the luminance component. Therefore, this modification will not affect the final D(R) curves. However, it may become considerably noticeable in the subjective test.

Time demand: average

Additionally needed functions: none

Approach

- Apply all functions, except motion estimation, also to the chrominance data.
- Keep in mind that the chrominance data is down-sampled by a factor of 2 both horizontally and vertically.
- Scale the motion vectors of the luminance component to compensate for the chrominance.
- Before performing motion compensation, apply the **floor** operation to the scaled motion vectors.
- Save all data that is part of the chrominance component into a separate file so that it does not affect calculating the rate.

6.3.4 Prediction of the DC Coefficients

Short description:

In Lab Experiment 5, the prediction of the DC component of DCT blocks is described. The quantized DC coefficient of the current block is predicted from the previous block. This procedure can also increase coding efficiency when coding P-frames. The approach is the same as in Lab Experiment 5.

Time demand: low

6.3.5 Dead-Zone Quantizer

Short description:

In Lab Experiment 3, a dead-zone quantizer was introduced.

In video codecs, the run-length of coefficients quantized to zero has considerable influence on the coding efficiency. Therefore, a dead-zone quantizer is often used to increase the number of coefficients that are quantized to zero.

Time demand: low if dead-zone quantizer already exists

6.4 Summary

In this lab experiment, you have built a complete video codec with the functions from the previous experiments. During the second session, you have had the opportunity to add several improvements to your coder with the help of this lab description and the optional parts of previous experiments. In the next lab session, we will compare the performance of some of your codecs in a subjective video test. On the last day of the lab course, the results of the subjective video test will be presented. In addition, each group will get the chance to talk about their codec.

6.5 Literature Recommendations

Current video codecs adapt the coding block structure according to the content. Additionally, non-squared blocks can be chosen by the encoder. Areas with low details (e.g. heaven) are usually coded with larger blocks, whereas high structured areas (e.g. the flowerbed) are coded with smaller blocks. Besides the maximum coding block size has been increasing over the recent years due to the increasing resolution of video data. The separation into blocks and all other details on the coding standards AVC and HEVC can be found in the corresponding overview papers:

T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, “Overview of the H.264/AVC video coding standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, July 2003

G. J. Sullivan, J. Ohm, W. Han, and T. Wiegand, “Overview of the High Efficiency Video Coding (HEVC) standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, Dec 2012

Lab Experiment 7

Subjective Video Test

In the previous lab experiments, different rate-distortion curves have been plotted. While the data rate of a video coder is obviously given by the size of the output file, there are different possibilities of expressing distortion. Until now, the PSNR has been used as a measure for objectively evaluating the video quality. Comparing this measure to subjective perception, evaluating the results, and discussing the different implemented codecs is the goal of this lab experiment.

The subjective test of the video codecs will be performed in the first session of this lab experiment. In the second session, which is also the last day of the lab course, the results of the subjective test will be presented. In addition, each group will get a chance to talk about their codec.

7.1 Viewing Conditions

In the meantime, the developed video codecs of all groups have been used to encode four not yet viewed sequences with five quantization step sizes. The decoded sequences of each run have been saved. The results of five selected groups will be used in a subjective video test in this lab experiment.

There are various possibilities to perform such a test. In order to achieve maximum reproducibility, the tests are usually performed under accurately controlled conditions. This includes that only one viewer be allowed per monitor, and that the illumination, background noise, etc. be accurately logged. Furthermore, the viewers are tested as to

their acuteness of vision beforehand. Thus, the exact execution of a subjective test is impossible within the scope of this lab course.

As an alternative, we will project the decoded sequences onto a screen. In doing so, all viewers will be able to watch and evaluate the sequences simultaneously, assuring approximately the same viewing conditions. Unfortunately, the highly varying distance to the screen cannot be prevented. This parameter is hence recorded in the evaluation forms.

7.2 Subjective Video Test

There are various methods for doing the assessment in a subjective test. The individual methods mainly differ in the way the sequences are viewed: Each sequence can be viewed independently from all others; it could be viewed in comparison to its reference sequence; or two distorted sequences could be viewed simultaneously. Additionally, it is possible to do the rating on a continuous scale, in discrete steps, or by a binary better/worse decision. Depending on the type of video material and the time available for the test, a combination of these methods is chosen. For nearly undistorted TV images, the “Double Stimulus Continuous Quality Scale” (DSCQS)[11] is frequently employed, for instance. With this test procedure, the sequences are viewed two at a time. For each sequence, the viewer is to give a rating on a 10 cm long scale which is labelled from “very good” to “very bad”. The test we want to perform is a so-called “Absolute Category Rating with Hidden Reference Removal”, abbreviated as ACR-HRR [12]. As the name does not contain a “Double Stimulus”, each sequence is assessed separately. The term “Absolute Category Rating” suggests that evaluation is to be done with discrete values, in this case “excellent, good, fair, poor, bad”. The used original sequences are also interspersed in the test. Assessing these sequences will indicate if the reference sequences are actually perceived as sequences of “excellent” quality. Since there can be exceptions, the results for the original sequences are used as reference and all results are subtracted from the reference. This leads to the so-called “Difference Mean Opinion Score” (DMOS).

7.3 Evaluation of a Subjective Video Test

There are many possibilities for evaluating a subjective video test. First off, all individual votes have to be merged into a combined result e.g., the mean value of the objective opinion per sequence. Just like in any other statistical analysis, the result becomes more

accurate with an increasing number of participants. The possibility of incorrectly filled out forms must also be taken into consideration. Afterward, the result of the subjective test can be compared to the objective measurement of the PSNR. This is often done using an x-y-plot. Figure 7.1 depicts an example diagram. If both subjective and objective results perfectly fit together, all points should be located on a diagonal line. Since the employed objective measure is often distorted with respect to the subjective measure, a calibration is applied. The calibration curve for the PSNR measure is also plotted into the figure. A calibration with this curve leads to the diagram in Figure 7.2. In order to

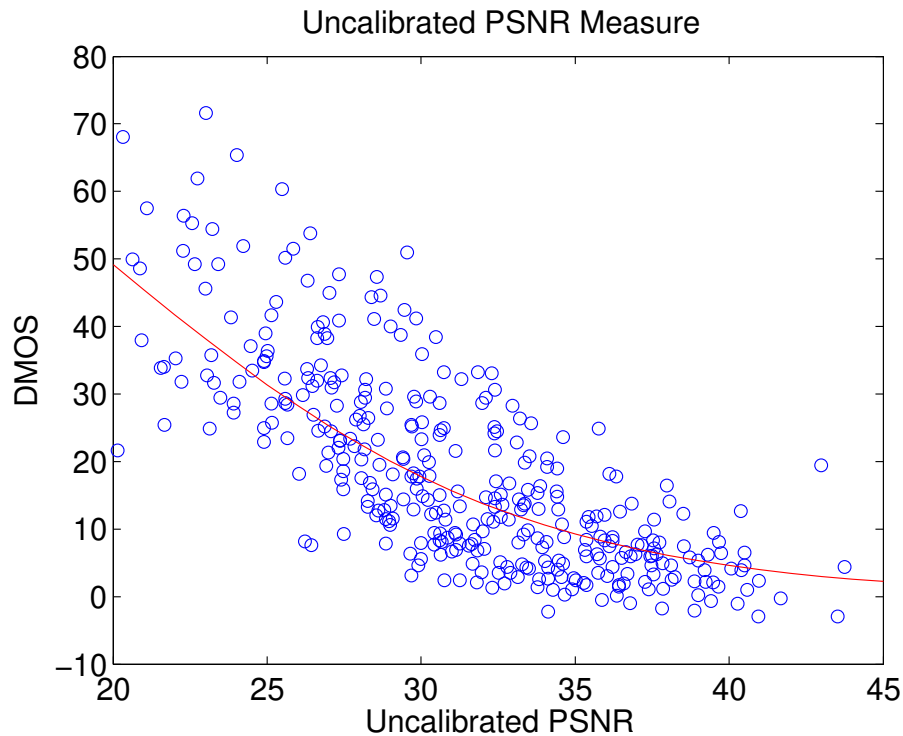


Figure 7.1: PSNR results compared to subjective assessment.

obtain a single measure for the correlation between the subjective and objective measures, the Pearson correlation is often employed. For this purpose, the results of the subjective test are combined into a sequence $a[n]$ and the results of the objective test are combined into another sequence $b[n]$. The Pearson correlation is computed from the z-normalized distribution of these sequences. Normalization to a z-normal distribution for the sequence $a[n]$ is calculated with the mean value $\bar{a}$ and the variance σ_a as

$$z_a = \frac{a[n] - \bar{a}}{\sigma_a}.$$

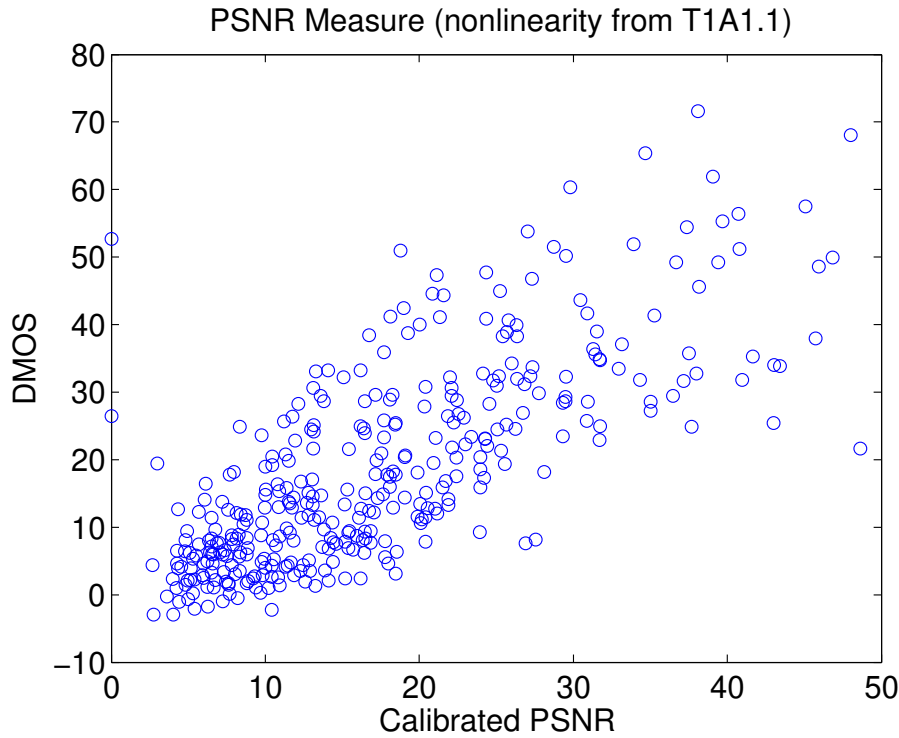


Figure 7.2: Calibrated PSNR results compared to subjective assessment.

From this, the Pearson correlation between two sequences a and b of length N can be computed with

$$r = \frac{\sum_{i=1}^N z_a[i] \cdot z_b[i]}{N}.$$

For the uncalibrated PSNR curve, a measurement of -0.7085 is obtained. The calibrated curve yields 0.7468. Optimally, the Pearson correlation would be 1 or -1 depending on the orientation of the diagonal. The best procedures currently available achieve a correlation of approximately 0.9.

Practical Part: _____

All students attending the lab course also participate in the subjective test. The codecs of five groups have been used to code four sequences, using five different quantization step sizes. This yields 100 sequences. In addition, the four original sequences have been included, totalling 104 sequences to be shown. The viewing order is determined randomly. Each sequence has a duration of approximately 8 seconds, followed by a pause of 4 seconds during which to give the assessment. Blocks containing 10 sequences each will be viewed one after the other. Each of

those blocks is followed by a short intermediate sequence in order to indicate the start of the new viewing block. So the whole test will take about half an hour.

The assessment is done by checking boxes on an evaluation form. This form is divided into blocks of 10 sequences each, to be filled out from top to bottom, left to right. Figure 7.3 shows part of such an evaluation form.

Since the forms are later on analyzed automatically using a computer program,

Bewertungsbogen subjektiver Test

Reihenfolge der Blöcke beachten!

Name: _____
Alter: _____

Block 1	sehr gut	gut	befriedigend	ausreichend	mangelhaft
Sequenz 1	☐	☒	☐	☐	☐
Sequenz 2	☒	☐	☐	☐	☐
Sequenz 3	☐	☐	☒	☐	☐
Sequenz 4	☐	☒	☐	☐	☐
Sequenz 5	☐	☐	☐	☐	☒
Sequenz 6	☒	☐	☐	☐	☐
Sequenz 7	☐	☐	☐	☒	☐
Sequenz 8	☐	☐	☐	☒	☐
Sequenz 9	☐	☐	☐	☐	☒
Sequenz 10	☐	☒	☐	☐	☐

Block 2	sehr gut	gut	befriedigend	ausreichend	mangelhaft
Sequenz 11	☐	☐	☐	☐	☐
Sequenz 12	☐	☐	☐	☐	☐
Sequenz 13	☐	☐	☐	☐	☐

Block 5	sehr gut
Sequenz 41	☐
Sequenz 42	☐
Sequenz 43	☐
Sequenz 44	☐
Sequenz 45	☐
Sequenz 46	☐
Sequenz 47	☐
Sequenz 48	☐
Sequenz 49	☐
Sequenz 50	☐

Block 6	sehr gut
Sequenz 51	☐
Sequenz 52	☐
Sequenz 53	☐

Figure 7.3: Part of the subjective evaluation form (translation of the ratings: sehr gut = excellent; gut = good; befriedigend = fair; ausreichend = poor; mangelhaft = bad).

checking the boxes should preferably be done using a dark-inked marker (or fountain pen) or a pencil. Ball-point pens have proved unsuitable for this analysis. For the evaluation, the number of dark pixels within the boxes is used. Therefore the marks should preferably look like as in the example of Figure 7.3.

The first part of this lab experiment concludes with the end of the subjective test.

7.4 Presentation and Discussion of Results

Practical Part:

The evaluation forms which were filled out in the first session of this lab experiment have been evaluated for the last lab session. The results are presented and discussed by all participants. On one hand, the correlation between the PSNR results and

the subjective test results is shown, on the other hand, a differentiation between the five video codecs used in the test will take place. Finally, each group gets the chance to shortly explain their extensions to their video coders.

This concludes the last lab experiment as well as the Image and Video Compression Lab Course.

7.5 Literature Recommendation

As shown before, the PSNR does not correlate well with the human subjective impression. Thus, *Netflix* introduced a new quality metric called Video Multimethod Assessment Fusion (VMAF) in 2016 that should represent the human impression with a trained neural network. This method is used more and more in current publications to evaluate the coding performance additionally to PSNR.

Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara, “Toward a practical perceptual video quality metric,” Netflix, <https://medium.com/netflix-techblog/toward-a-practical-perceptual-video-quality-metric-653f208b9652>, Tech. Rep., Jun. 2016

Bibliography

- [1] A. Kaup, *Skript zur Vorlesung Image and Video Compression*. Lehrstuhl für Multi-mediakommunikation und Signalverarbeitung, Universität Erlangen-Nürnberg, 2004.
- [2] G. Söder, *Skriptum zum Praktikum Simulation digitaler Übertragungssysteme – Wert-diskrete Informationstheorie*. Lehrstuhl für Nachrichtentechnik Technische Universität München, 2003.
- [3] D. Marpe, H. Schwarz, and T. Wiegand, “Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 620–636, July 2003.
- [4] B. Li, H. Li, L. Li, and J. Zhang, “ λ domain rate control algorithm for high efficiency video coding,” *IEEE Transactions on Image Processing*, vol. 23, no. 9, pp. 3841–3854, Sep. 2014.
- [5] Y. Wang, Y.-Q. Zhang, and J. Ostermann, *Video Processing and Communications*. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2001.
- [6] A. M. Tekalp, *Digital video processing*. University of Michigan: Prentice-Hall, Inc., 1995.
- [7] J. Lainema, F. Bossen, W. Han, J. Min, and K. Ugur, “Intra coding of the HEVC standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1792–1801, Dec 2012.
- [8] G. Sullivan, Ed., *ITU-T Recommendation H.263: Transmission of Non- Telephone Signals; Video Coding for Low Bit Rate Communication*. ITU-T SG16, Jan. 1998.
- [9] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, “Overview of the H.264/AVC video coding standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, July 2003.
- [10] G. J. Sullivan, J. Ohm, W. Han, and T. Wiegand, “Overview of the High Efficiency Video Coding (HEVC) standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, Dec 2012.

- [11] *Methodology for the subjective assessment of the quality of television pictures*, International Telecommunication Union, Question ITU-R 211/11, recommendation ITU-R BT.500-10.
- [12] *Subjective video quality assessment methods for multimedia applications*, International Telecommunication Union, ITU-T Study Group 12, 9 1999, recommendation ITU-T P.910.
- [13] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara, “Toward a practical perceptual video quality metric,” Netflix, <https://medium.com/netflix-techblog/toward-a-practical-perceptual-video-quality-metric-653f208b9652>, Tech. Rep., Jun. 2016.